

Notes on BitVM3: Garbled Circuits, Extractable Signatures, and Adaptor Signatures

Cian Lalor

Important Note: This document (and corresponding codebase) is a work in progress.
Sections may be incomplete or require further review.

Notation

$\{0, 1\}^\ell$	set of bit-strings of length ℓ ; $ \{0, 1\}^\ell = 2^\ell$.
$\{0, 1\}^*$	set of bit-strings of arbitrary finite length (no fixed bound).
κ	security parameter (bit-length of labels/keys; typically 128).
$A \leftarrow B$	A is assigned the value B (or the output of B , if B is an algorithm).
$A \xleftarrow{\$} S$	A is sampled uniformly at random from the set S .
$A : B \rightarrow C$	A is a function with domain B and codomain C .
$L_i^0 \mapsto 0$	inside a table: “the entry L_i^0 maps to the value 0”.
$[n]$	the index set $\{1, 2, \dots, n\}$: a set of n elements.
$A \sqcup B$	disjoint union: $A \cup B$ with $A \cap B = \emptyset$ asserted, so $ A \sqcup B = A + B $.
$\oplus$	bitwise XOR ($\wedge$ in Rust on <code>bool</code> values).
$\parallel$	string concatenation.
1^κ	the security parameter in unary; standard input format for crypto algorithms.
$\text{negl}(\kappa)$	a function shrinking faster than every inverse polynomial (§2.2).
PPT	probabilistic polynomial-time algorithm.
$\perp$	“no valid decoding”; signals an honest evaluation did not produce this output.
π	overloaded: a SNARK <i>proof object</i> (§13) and a <i>permutation mapping</i> of garbled gates.

Contents

Notation	i
Introduction & BitVM3 Overview	1
I Garbling Foundations	2
1 Boolean Circuits	2
2 Garbling Schemes	3
2.1 Syntactic Definition	3
2.2 Correctness and Authenticity	4
2.3 Privacy-Free Variant	5
3 Yao's Construction	6
3.1 Wire Labels	6
3.2 Garbled Gate Tables	6
3.3 Worked Example: 1-Bit AND Gate	8
3.4 Chaining Gates: Multi-Gate Evaluation	10
4 Optimized Garbling: Free-XOR and Half-Gates	11
II Committing and Revealing Labels	13
5 Signature Primitives: What Is Used Where	13
6 Lamport One-Time Signatures	14
6.1 Construction and Unforgeability	15
6.2 Non-Equivocation via Reused Keys	17
7 Winternitz One-Time Signatures	18
7.1 Construction	18
7.2 Checksum and Unforgeability	18
8 Garbled Encoding Extractable Signatures	19
8.1 Syntax and Security	20
8.2 Lamport-over-Input-Labels Instantiation	21
9 Schnorr Adaptor Signatures	22
9.1 Motivation: Scriptless Scripts	23
9.2 Schnorr Refresher	23
9.3 Pre-Signature, Adapt, Extract	23
9.4 Security Properties	24
9.5 Bridge to GES: Two Flavours of Extractability	25
9.6 Realizing GES with Adaptor Signatures	25
9.7 Wide Labels: Mapping Bits to Digits	27
9.8 Application: Atomic Swap	29
III Protocol and On-Chain Realization	30

10 Setup Verification	30
10.1 SNARK or STARK of Correct Setup	30
10.2 Multiple Garblings and Statistical Soundness	31
10.2.1 Cut-and-Choose	31
10.2.2 Verifiable Secret Sharing of Labels	35
10.3 Trusted Execution Environment	37
10.4 Comparison of the Three Routes	37
11 Bitcoin Transactions and the BitVM3 Transaction Graph	38
11.1 Transaction Anatomy	38
11.2 Taproot	39
11.3 Sighash: What a Signature Commits To	40
11.4 Spending Conditions	41
11.5 Signatures and Adaptors in the Witness	42
11.6 The Assert, Disprove, and Timeout Templates	43
12 The SNARK Being Garbled	43
12.1 Components	44
12.2 Garbled Verifier Overview	44
12.3 Public Inputs and Binding	45
12.4 Trusted Setup and Toxic Waste	45
12.5 How the Proof Reaches the Chain	46
12.6 Structuring the Verifier Circuit	46
12.7 Hazards and Constraints	47
13 BitVM3-core Protocol	48
13.1 Role Assignments	49
13.2 Setup, Proving, Disprove	49
13.2.1 Setup (off-chain, once)	49
13.2.2 Proving (on-chain, per claim)	51
13.2.3 Disprove (on-chain, conditional, within the challenge window)	51
13.2.4 Anatomy of the Assert Broadcast	52
13.3 Why the Pieces Compose	54
14 Related Designs and Security History	55
14.1 The RSA Garbling Scheme and Its Break	55
14.2 Argo: the Size Frontier	55
14.3 Glock: Designated-Verifier SNARK-in-Circuit	56
14.4 BABE: Witness Encryption	56
14.5 Comparison: Argo, Glock, BABE	57
14.6 The GSV Lineage	57
14.7 Lombard BSA: TEE-Attested, Sealed Labels	57
14.8 Reality Check	57
(Optional) Further Reading	58
A Bitwise XOR as an Encryption Primitive	58
A.1 Wide-label rows	58
B Merkle Commitments	59

Introduction & BitVM3 Overview

A simplified picture of the bridge scenario this document supports.

1. A burn event happens on the destination chain (e.g. Ethereum).
2. The operator collects the burn transaction, the relevant block header, and Merkle inclusion proofs from Ethereum.
3. The operator runs `Groth16.Prove`($\mathcal{R}$, crs, φ , w), where the witness w includes those Ethereum chain artefacts and the statement φ is the burn fact being claimed.
4. This π is what the on-chain signature σ_π commits to on Bitcoin.
5. A challenger reads only: extract L_π from σ_π , run the garbled circuit, check whether the output decodes to False.

There is no “info injection” by the challenger – the challenger’s job is verification, the prover’s is generation. The SNARK only verifies if π was honestly built from real destination-chain state.

This document aims to outline how, instead of running the trace of the verifier’s execution of a proof compiled to a Boolean circuit in Bitcoin Script on-chain (as in BitVM2), we can compress the programme with the garbled-circuit primitive, reducing the final on-chain data validation by $\sim 1000\times$. Notably, BitVM3 relies on a *privacy-free* variant of the garbled circuit: the obfuscation the garbling primitive would normally provide is of no concern here, so input privacy is obsolete and only authenticity is required.

The key to the virtual machine’s operation is having a SNARK attest to an on-chain claim φ and committing this to the public witness data, which when done reveals a secret. That secret unlocks an *adaptor signature*, completing a native Bitcoin transaction and writing the result back to chain.

The document is organized in three parts. **Part I (Garbling Foundations)** builds Boolean circuits, the garbling-scheme abstraction, Yao’s construction, and the optimized (half-gate) garbling the deployed systems use. **Part II (Committing and Revealing Labels)** covers the machinery that releases input labels: Lamport one-time signatures, the Garbled Encoding Extractable Signature (GES) built on them, and the Schnorr adaptor signatures BitVM3 actually deploys on-chain. **Part III (Protocol and On-Chain Realization)** turns to setup verification (cut-and-choose, VSSS, or a SNARK of setup), the Bitcoin transactions and Assert/Disprove graph that carry the dispute, the BitVM3-core protocol that composes everything, and the related designs and security history that separate the paper’s clean model from what is being built. Cross-references to the accompanying Rust crates are noted throughout.

Part I: Garbling Foundations

1 Boolean Circuits

Definition 1.1 (Boolean circuit). A *Boolean circuit* computing $f : \{0, 1\}^\ell \rightarrow \{0, 1\}^m$ is a finite directed acyclic graph C whose vertices are either *input wires* (indexed $1, \dots, \ell$, in-degree 0), *output wires* (indexed $1, \dots, m$, out-degree 0), or *gates* drawn from a gate set $\mathcal{G}$. The standard set used throughout is

$$\mathcal{G} = \{\text{AND}, \text{XOR}, \text{NOT}\},$$

with fan-in 2 for AND and XOR, and 1 for NOT. Every gate has exactly one output wire (one output bit), though that wire may fan out to several downstream gates.¹²

Remark 1.1 (Graph terminology). For a directed graph, the *in-degree* of a vertex is the number of edges pointing into it, and the *out-degree* the number pointing out. An input wire has in-degree 0 because its value is supplied from outside the circuit (the i -th component of x); an output wire has out-degree 0 because its value is read off as the i -th component of $f(x)$. For a gate, the in-degree is called the *fan-in* – the number of bits the gate consumes – and the out-degree the *fan-out* – the number of downstream gates that read its result. Convention restricts fan-in ≤ 2 because a gate of fan-in k has a truth table of 2^k rows, and the cost of garbling that gate scales with this exponent.

Example 1.2 (Half-adder). The function $f : \{0, 1\}^2 \rightarrow \{0, 1\}^2$ defined by $f(a, b) = (a \oplus b, a \wedge b)$ – the half-adder – is computed by a circuit on six vertices: two input wires a, b (in-degree 0); one XOR gate (fan-in 2, fan-out 1) producing the sum bit; one AND gate (fan-in 2, fan-out 1) producing the carry bit; and two output wires sum, carry (out-degree 0). Both inputs a and b have fan-out 2, since each is consumed by both gates. This concrete f is the running example for the Yao construction in §3.

Definition 1.2 (Evaluation). Given input $x = (x_1, \dots, x_\ell) \in \{0, 1\}^\ell$, the *evaluation* of circuit C on x assigns $\beta_v \in \{0, 1\}$ to each vertex in topological order: $\beta_{v_i} = x_i$ for input wires; for a gate v with operation $g \in \mathcal{G}$ and predecessors $u_1, \dots, u_k$, set $\beta_v = g(\beta_{u_1}, \dots, \beta_{u_k})$. The output is $f(x) = (\beta_{o_1}, \dots, \beta_{o_m})$ read from the output wires.³

The full assignment $\{\beta_v\}$ over *every* vertex (not just the output wires) is the analog of BitVM2’s execution trace, where it can be described as a working buffer which builds up programme state; $f(x)$ is only its final output.

Remark 1.3 (Directed acyclic graphs and topological order). The circuit graph C is a *directed acyclic graph* (DAG): *directed* because each edge (u, v) has an orientation – a wire carries a value from u to v , not the reverse – and *acyclic* because C contains no cycle, i.e. no sequence of edges $v_0 \rightarrow v_1 \rightarrow \dots \rightarrow v_k = v_0$ returning to its start. A DAG on n vertices admits a *topological*

¹Implementation note: in Rust (and the C family) the operator $\wedge$ is bitwise XOR, *not* exponentiation as in mathematical notation. On `bool` values it acts as logical XOR ($a \oplus b$), so $\{\text{AND}, \text{XOR}, \text{NOT}\}$ is directly expressible as `a && b`, `a ^ b`, `!a`.

²In a working implementation the circuit is held in an *intermediate representation* (IR): an in-memory data structure – here a `struct Circuit` carrying wire counts and an ordered list of `enum Gate` variants – that downstream code (evaluator, garbler) walks over. The theoretical f becomes the IR once we want to compute with it. Distinct from a *buffer*: the IR (`Circuit`) carries structure, whereas the buffer (the flat `Vec<bool>` of length `num_wires` that `evaluate` scratches per-wire values into) holds only values.

³In our gate set (Definition 1.1) the fan-in k is 2 for AND/XOR and 1 for NOT, so the predecessors are at most u_1, u_2 . The output count m depends on f : the half-adder of Example 1.2 has $m = 2$ (sum, carry), so $f(x) = (\beta_{o_1}, \beta_{o_2})$; the SNARK verifier circuit in §13 has $m = 1$ (accept or reject proof), so $f(x) = \beta_{o_1}$; this single accept/reject bit is exactly what the BitVM dispute turns on (<https://bitvm.org/snark.html>).

ordering: a permutation $v_{\pi(1)}, \dots, v_{\pi(n)}$ of its vertices such that whenever (u, v) is an edge, u appears before v in the ordering. Definition 1.2 relies on such an ordering to evaluate each gate strictly after its predecessors. We exclude graphs with cycles (which would model sequential or stateful circuits) since they require a different evaluation semantics.⁴

Remark 1.4 (Partial vs. total order). The DAG fixes only a *partial order* (gates are ordered only if a path links them), so many gates have no required relative ordering; a topological sort chooses one *total order* consistent with this partial ordering, of which many are valid; a total order is needed only to assign per-gate nonces ν (see Rmk. 3.3). Gates at the same depth are independent, so they can be evaluated in parallel, i.e., the circuit’s depth, not its gate count, bounds the sequential layers.

2 Garbling Schemes

A garbling scheme transforms a Boolean circuit f (as in §1) into a randomised object F that another party can evaluate on an input it does not know in cleartext. The party that performs the transformation, the *garbler*, chooses, for each wire of f , a pair of random labels representing the bits 0 and 1. The *evaluator* sees only labels, never bits, and runs the garbled object gate by gate to obtain output labels; a separate decoding step converts those labels back to a plaintext bit-string. The construction is interesting only because of what it preserves under a malicious counterparty: an honest evaluator learns $f(x)$ and (in the full scheme) nothing else; a malicious evaluator cannot produce labels that decode to anything but $f(x)$.

In BitVM3 the role split is more specialised. The prover is the garbler; the verifier is the evaluator; the input being garbled is a SNARK proof π , which is already public. Privacy is therefore not required as there is no secret input to hide, but authenticity remains essential: a malicious prover must not be able to produce labels that fool the verifier into accepting $\text{SNARK.Vrfy}(\pi) = \text{True}$ when in fact it is False. The syntactic definition that follows and the properties of §2.2 are designed to capture exactly this.

2.1 Syntactic Definition

Definition 2.1 (Garbling scheme). Let $\mathcal{F}$ be a class of Boolean circuits computing functions $f : \{0, 1\}^\ell \rightarrow \{0, 1\}^m$. A *garbling scheme* for $\mathcal{F}$ is a tuple $\mathbf{G} = (\mathbf{Gb}, \mathbf{En}, \mathbf{Ev}, \mathbf{De})$ of polynomial-time algorithms:

1. $(F, e, d) \leftarrow \mathbf{Gb}(1^\kappa, f)$ – a probabilistic *garbler* taking a security parameter 1^κ and a circuit $f \in \mathcal{F}$. It returns a garbled circuit F , encoding information e , and decoding information d .
2. $L_x \leftarrow \mathbf{En}(e, x)$ – a deterministic *encoder* mapping a plaintext input $x \in \{0, 1\}^\ell$ to encoded labels L_x . In BitVM3 these labels are not delivered directly; the prover releases $\mathbf{En}(e, \pi)$ for the SNARK proof π only by signing it via the GES of §8.
3. $L_y \leftarrow \mathbf{Ev}(F, L_x)$ – a deterministic *evaluator* mapping the garbled circuit and an encoded input to encoded output labels L_y .
4. $y \leftarrow \mathbf{De}(d, L_y)$ – a deterministic *decoder* recovering a plaintext output $y \in \{0, 1\}^m \cup \{\perp\}$ from the encoded output. The symbol $\perp$ denotes “no valid decoding” and signals that L_y was not produced by an honest evaluation.

⁴A topological ordering is a *schedule*: read input wires first (in-degree 0), process each gate when all its inputs are available, read output wires last. Any two topological orderings of the same DAG produce the same $f(x)$ because each gate depends only on its predecessors, never on the relative order chosen between independent (incomparable) gates, which is what makes $f(x)$ a function of (C, x) alone, not of the scheduling choice. We write $\text{eval} : (C, x) \mapsto f(x)$ for this evaluation (universal) mapping that interprets a circuit on an input.

Remark 2.1 (Probabilism and the security parameter 1^κ). *Probabilistic* means **Gb** draws internal randomness, so the same circuit f yields different (F, e, d) on each run, here from freshly sampled wire labels (in contrast to a *deterministic* algorithm, which always yields the same result). The *security parameter* κ is the label length in bits (e.g. 128) and sets the scheme’s *cryptographic strength*; larger κ makes it exponentially harder to break, and the unary form 1^κ lets “polynomial-time in the input” count κ itself.

Remark 2.2 (Why **Gb** returns three things). The three outputs reflect three distinct uses. The garbled circuit F is the public artefact: anyone holding it can run **Ev**. The encoding information e is the only piece needed to translate a plaintext input into the wire labels that **Ev** accepts – whoever encodes inputs must hold e . The decoding information d inverts that translation at the output: whoever interprets the result holds d . In BitVM3-core (§13) the prover holds all three at garbling time. Afterwards (F, d) are sent to the verifier, while e is retained by the prover and handled through the signature scheme of §8: the GES public key *commits* to all of e (both labels per wire, as hashes), and signing an input x *exposes* only $\text{En}(e, x)$, one label per wire. The full e is never revealed, and releasing the complementary labels would equivocate and slash the prover. A destination-chain event (the burn) is what allows a valid proof π to be generated; feeding π to the encoder emits the labels $L_\pi = \text{En}(e, \pi)$, so exactly the partial e matching π (one label per wire) is revealed to the verifier.

Remark 2.3 (Decoding in practice). For the constructions of §3, the decoding information d is a lookup table: for each output wire i the garbler stores the pair $(L_i^0 \mapsto 0, L_i^1 \mapsto 1)$, where L_i^0, L_i^1 are the two labels chosen for that wire at garble time. **De** checks each component of $L_y = (L_1, \dots, L_m)$ against the table: if every L_i matches one of the two stored labels for its wire, it returns the corresponding bit-string; otherwise it returns $\perp$. The $\perp$ case is where authenticity bites. An adversary attempting to convince the verifier of a wrong output must produce labels that are simultaneously *not* the honest evaluation result and yet *still match* entries of d , which, as labels are uniformly random κ -bit strings, succeeds with probability at most $1/2^\kappa$ per output wire.

2.2 Correctness and Authenticity

A garbling scheme is useful only if honest evaluation lands on the right answer and no efficient adversary can land anywhere else accidentally.

Theorem 2.4 (Correctness). For every $f \in \mathcal{F}$ and every $x \in \{0, 1\}^\ell$,

$$\Pr[\text{De}(d, \text{Ev}(F, \text{En}(e, x))) = f(x)] = 1,$$

where the probability is over the randomness of $\text{Gb}(1^\kappa, f) = (F, e, d)$.⁵

Theorem 2.5 (Authenticity). For every PPT adversary $\mathcal{A}$, every $f \in \mathcal{F}$, and every $x \in \{0, 1\}^\ell$, let

$$(F, e, d) \leftarrow \text{Gb}(1^\kappa, f), \quad L_x := \text{En}(e, x), \quad L'_y \leftarrow \mathcal{A}(1^\kappa, f, F, L_x).$$

Then

$$\Pr[L'_y \neq \text{Ev}(F, L_x) \wedge \text{De}(d, L'_y) \neq \perp] \leq \text{negl}(\kappa).$$

Proof sketch. Work in the random oracle model. By the 0^t marker, any non- $\perp$ output is a legitimate output-wire label. The adversary is given only the active labels L_x , the garbled circuit F , and f ; the complementary *inactive* labels are uniform κ -bit strings it never observes.

⁵The equality holds exactly because the honest row’s 0^t marker always matches, provided the evaluator identifies the correct row unambiguously (e.g. via point-and-permute select bits). A *wrong* row passing the marker by coincidence is a 2^{-t} event; it does not affect correctness of the intended row and is accounted for by authenticity below.

To output some $L'_y \neq \text{Ev}(F, L_x)$ that decodes to non- $\perp$, it must either guess an inactive output label, which succeeds with probability at most $2^{-\kappa}$, or produce a ciphertext decrypting to a valid 0^t marker without querying H on the correct $(K_1 \parallel K_2 \parallel \nu)$, which succeeds with probability at most 2^{-t} per attempt. A union bound over the $\text{poly}(\kappa)$ gates and oracle queries gives total advantage at most $\text{poly}(\kappa) (2^{-\kappa} + 2^{-t}) = \text{negl}(\kappa)$. $\square$

Remark 2.6 (PPT adversaries and negligible probability). A *probabilistic polynomial-time* (PPT) algorithm is a randomised algorithm whose runtime is bounded by a polynomial in the security parameter κ . The class captures what any realistic attacker could compute: brute-forcing a κ -bit key requires 2^κ steps and is therefore *not* PPT, while anything an adversary can plausibly do in practice is. A function $\mu : \mathbb{N} \rightarrow \mathbb{R}_{\geq 0}$ is *negligible* if it shrinks faster than every inverse polynomial: for every constant $c > 0$ there exists κ_0 such that $\mu(\kappa) < 1/\kappa^c$ for all $\kappa \geq \kappa_0$. We write $\text{negl}(\kappa)$ for any such function μ . A claim of the form “ $\Pr[\text{some bad event}] \leq \text{negl}(\kappa)$ ” for every PPT adversary $\mathcal{A}$ therefore reads as: no realistic attacker, even given polynomially many tries, can make the bad event happen with non-vanishing probability.

Remark 2.7 (Reading the authenticity bound). The adversary $\mathcal{A}$ is handed exactly what a BitVM3 verifier receives: the garbled circuit F and the encoded input L_x for the operator’s chosen x . The bounded event is that $\mathcal{A}$ produces output labels L'_y that (i) differ from the honest evaluation $\text{Ev}(F, L_x)$ and (ii) still decode to a non- $\perp$ value. The second clause matters because $\perp$ is what makes the verifier reject; “decodes to non- $\perp$ ” is exactly what would make the verifier accept. So this theorem is the formal statement: a malicious prover cannot fabricate output labels that fool the verifier into accepting the wrong y , except with negligible probability.

2.3 Privacy-Free Variant

The Bellare–Hoang–Rogaway framework⁶ defines several security properties beyond correctness: *privacy* (the evaluator learns nothing about x beyond $f(x)$), *obliviousness*⁷ (the evaluator learns nothing about x at all, not even $f(x)$, only L_y), and *authenticity*. A scheme required to satisfy only correctness and authenticity, dropping privacy and obliviousness, is the *privacy-free* variant.

The motivation is direct. In BitVM3-core the input being garbled is a SNARK proof π that the prover has already chosen to publish on chain i.e., it has no secrecy left to preserve. Dropping privacy unlocks constructions (notably the privacy-free half-gates of §4) whose ciphertext-per-AND-gate cost is roughly half that of the full-privacy variant: one ciphertext per AND instead of two, with XOR gates free. Across a SNARK-verifier circuit with millions of AND gates these savings compound: classical Yao’s four ciphertexts per AND gate fall to one, a $4\times$ reduction in the size of the garbled object the operator ships off-chain, of which $2\times$ comes from dropping privacy and $2\times$ from the other optimizations (§4). The construction and the algebra behind the saving are given in §4.

Remark 2.8 (What we lose by dropping privacy). Without privacy, the garbled circuit may leak information about the plaintext input through the structure of labels and ciphertexts. In BitVM3 this is benign: π is public the moment the prover signs it, and the verifier already knows f (the SNARK verifier circuit) i.e., there is nothing for privacy to protect.

Remark 2.9 (Why the labels survive). Privacy is the *only* property dropped; the label pairs are kept for *authenticity*. Evaluating the garbled circuit still yields only the genuine output label, and forging the label of a false output is infeasible (Theorem 2.5). That unforgeable output label, released only once the prover signs the proof (§8), is what an adaptor signature binds to settle the dispute on chain. So privacy-free changes what the labels *hide*, not what they *commit*.

⁶M. Bellare, V. T. Hoang, P. Rogaway, *Foundations of Garbled Circuits*, ACM CCS 2012. ePrint 2012/265, <https://eprint.iacr.org/2012/265>.

⁷Obliviousness here is a *security property* of the garbling scheme and is distinct from *oblivious transfer* (the interactive protocol of Remark 3.9)

Remark 2.10 (What BitVM3 skips). Three steps of the general scheme fall away. The bit-hiding machinery of *privacy* and *obliviousness* is dropped entirely. The per-gate ciphertext shuffle (§3.2) is omitted, replaced by an explicit row pointer. And the encoder’s labels are not delivered directly or by oblivious transfer (Remark 3.9); they are released only by signing the proof (§8).

3 Yao’s Construction

3.1 Wire Labels

Section 2 gave the four-algorithm signature of a garbling scheme but said nothing about how **Gb**, **En**, **Ev**, **De** are actually built. Yao’s construction is the first realisation. Its building block is the *wire label*: a random κ -bit string that stands in for the plaintext bit 0 or 1 on a given wire. The evaluator only ever sees labels, never bits, until the very end. Everything in §3.2–§3.4, covering encrypted truth tables, gate chaining, output decoding, is just bookkeeping on top of these labels.

Definition 3.1 (Wire labels). For a Boolean circuit C with wires indexed $1, \dots, W$, the garbler samples, independently for each wire w and each bit $b \in \{0, 1\}$, a label

$$L_w^b \xleftarrow{\$} \{0, 1\}^\kappa,$$

uniformly at random. The pair (L_w^0, L_w^1) is the *label pair* of wire w . Every wire (inputs, intermediates, and outputs) is assigned a label pair.

At garble time the garbler holds all $2W$ labels. The bit-to-label mapping for input wires is stored in the encoding information e and used by **En** (the encoder, which converts a plaintext input into the labels the evaluator will run on); for output wires it is stored in the decoding information d and used by **De** (the decoder, which translates the final output labels back into a plaintext bit-string). After garbling, the evaluator receives exactly one label per input wire (the one matching the bit being evaluated on) and the decoding information d for output wires; the label pair of an intermediate wire is never revealed in plaintext to either party during evaluation.

Remark 3.1 (Why intermediate wires are never revealed). A bit-to-label mapping is published only at the circuit’s boundary (e for input wires, d for output wires), so an intermediate wire has no decoding, and evaluation surfaces only its *single active label* (§3.2); the complementary one stays locked in the garbled tables. Lacking both a label pair and any mapping, the evaluator cannot tell which bit an intermediate label denotes: this is the *privacy* property of §2.3. The privacy-free variant keeps these structural facts but drops the secrecy: since the input is public (π in BitVM3), the evaluator recomputes f and traces every internal bit, leaving each intermediate label an authenticated token rather than a secret.

3.2 Garbled Gate Tables

A garbled gate is the per-gate encrypted analogue of a *truth table* (the table listing each possible input combination and the resulting output bit; for a fan-in-2 gate this is four rows, one per pair $(i, j) \in \{0, 1\}^2$). Each row of the plaintext truth table is encrypted under the input labels for that row; the evaluator, holding the labels for the actual input, can decrypt exactly one row and recover the output label.

Definition 3.2 (Dual-key cipher). Let $H : \{0, 1\}^* \rightarrow \{0, 1\}^{\kappa+t}$ be a hash function modelled as a random oracle⁸, and let t be a padding length. For keys $K_1, K_2 \in \{0, 1\}^\kappa$, a message

⁸“Modelled as a random oracle” is a standard cryptographic idealisation: H samples an independent, uniformly random value on each new input and returns that same value on repeated queries, i.e. it is a fixed function whose outputs look uniformly random. Real hash functions (e.g. SHA-256) are deterministic, not true random oracles, but proofs “in the random oracle model” are widely accepted when no better assumption is available.

$M \in \{0, 1\}^\kappa$, and a nonce ν , the *dual-key symmetric encryption* of M under both keys is

$$E_{K_1, K_2}^\nu(M) := H(K_1 \parallel K_2 \parallel \nu) \oplus (M \parallel 0^t).$$

Decryption is $D_{K_1, K_2}^\nu(C) := H(K_1 \parallel K_2 \parallel \nu) \oplus C$; the decryptor accepts the result if the last t bits are all zero, and outputs $\perp$ otherwise.⁹ False acceptance under the wrong keys occurs with probability $1/2^t$. Clearly, when $C = E_{K_1, K_2}^\nu(M)$ the decryptor returns $M \parallel 0^t$, recovering the message M and the t zero bits.

Remark 3.2 (Why padding). Decryption under the wrong (K_1, K_2) produces a uniformly random $(\kappa + t)$ -bit string. The t -bit zero pad gives the evaluator a checkable witness that the keys were correct: real decryptions end in 0^t ; random ones almost never do. With $t = 128$, as instantiated throughout this document (Remark 3.5; $t = 80$ is a common classical choice), the false-positive rate 2^{-128} is negligible.

Remark 3.3 (Nonce choice). The nonce ν is not sampled; it is a deterministic per-gate identifier (typically the gate’s index in the circuit’s topological order) and is public. Its role is to ensure that two distinct gates with coincidentally-equal label pairs do not produce identical ciphertexts. The garbler and the evaluator both derive ν from the same public circuit description and its total order (Remark 1.4), so it never needs to be transmitted.

Remark 3.4 (Hash output length and ciphertext size). For E_{K_1, K_2}^ν to be well-typed, H must produce $(\kappa + t)$ output bits, matching the length of $M \parallel 0^t$. In practice one fixes $\kappa + t$ to the natural output length of the hash (e.g. 256 for SHA-256: $\kappa = 128$, $t = 128$), or truncates the hash output. Each ciphertext is therefore $\kappa + t$ bits, and a plain-Yao garbled circuit on $|C|$ fan-in-2 gates has total size $4|C|(\kappa + t)$ bits before any optimisation.

Remark 3.5 (Widened labels force a 48-byte row: SHA-384). The bridge to adaptor signatures (Remark 9.6) widens the label to $\kappa = 256$ bits, so that a label read as a scalar leaves a discrete log with $\approx 2^{124}$ security (Remark 9.7) rather than the $\approx 2^{64}$ a 128-bit label would give. That choice propagates into the ciphertext. A row is $M \parallel 0^t$ with M the $\kappa = 256$ -bit output label and $t = 128$ the marker, so $\kappa + t = 384$ bits = 48 bytes, and H must emit 384 bits to mask it. SHA-256’s 256-bit output no longer spans a row. The accompanying implementation uses SHA-384, whose output is exactly 384 bits = 48 bytes, so the entire hash is the keystream and no bytes are discarded. (SHA-512 would serve equally, its top 128 bits truncated: a fixed-position truncation of a random oracle is itself a random oracle on the bytes retained, so the $t = 128$ marker still coincides with a wrong-key decryption only with probability 2^{-128} , Remark 3.2.) A tempting miscount reasons that the 64 hash bytes “exceed the group order” and so must be trimmed to 48: but the group order constrains only the *label*, which is read as a scalar, whereas the pad is pure keystream, XOR’d onto the row and never interpreted as a field element. The figure 48 follows solely from $\kappa + t = 256 + 128$, and $t = 128$ is fixed by the target 2^{-128} authenticity (Remark 3.2), independently of how wide the label κ is.

Definition 3.3 (Garbled gate table). Let g be a fan-in-2 gate with input wires a, b and output wire c , computing $c = g(a, b)$. Let (L_w^0, L_w^1) denote the label pair of wire w (Definition 3.1). The *garbled gate table* for g is the set of four ($= 2^k$ for fan-in $k = 2$; Remark 3.6) ciphertexts

$$\{ E_{L_a^i, L_b^j}^\nu(L_c^{g(i, j)}) \mid (i, j) \in \{0, 1\}^2 \},$$

shuffled by a uniformly random permutation π_g on four elements. The nonce ν is a fixed per-gate identifier (e.g. the gate’s index in the circuit) and is publicly known.

⁹XOR is self-inverse, so XOR’ing the ciphertext with the same hash used at encryption returns the original $M \parallel 0^t$. See Appendix A for a worked bit-level example.

The evaluator, holding labels L_a^i and L_b^j for the actual input bits (i, j) , attempts $D_{L_a^i, L_b^j}^\nu$ on each of the four ciphertexts. Exactly one ciphertext, the one originally encrypted under (L_a^i, L_b^j) , decrypts to a string ending in 0^t ; the evaluator strips the pad and uses the leading κ bits as the output label $L_c^{g(i,j)}$. The three other ciphertexts produce strings whose last t bits are zero only with probability 2^{-t} , so trial decryption identifies the correct row with overwhelming probability.

Remark 3.6 (Truth-table size and shuffle count). A fan-in- k gate has 2^k truth-table rows, since each of k binary inputs takes one of two values; a fan-in-2 gate (AND, XOR) therefore has 4 rows, NOT has 2 rows. The garbler shuffles these rows by sampling π_g uniformly from the *symmetric group* S_n – the group of all bijections of an n -element set under composition, with $|S_n| = n!$. For a fan-in-2 gate this gives $|S_4| = 24$ possible shuffles; for fan-in-3 it would be $|S_8| = 40320$.

Remark 3.7 (Role of the shuffle). The permutation π_g removes the implicit link between a ciphertext’s array index and the input combination that produced it. Without it, the evaluator could read input bits off the index of the row that decrypted successfully, even when the labels themselves leak nothing. In the privacy-preserving Yao setting the shuffle is therefore essential. In the privacy-free setting of BitVM3 it is not: the prover’s input is a public SNARK proof, so input-bit leakage is harmless. The shuffle is presented here for completeness because it is constitutive of plain Yao; in the optimised constructions of §4 (point-and-permute, half-gates), it is replaced by an explicit per-gate row pointer that is faster and equally suitable for the BitVM3 regime.

3.3 Worked Example: 1-Bit AND Gate

This subsection threads the constructions of §3.1 and §3.2 through a single AND gate, end-to-end. The crux of what follows: throughout the procedure the evaluator handles only labels, which are indistinguishable from uniformly random κ -bit strings, and recovers the plaintext value of the output wire alone (via d); the truth values of input and intermediate wires remain hidden.

1. Setup. The gate has input wires a, b and output wire c . The garbler samples six κ -bit labels:

$$L_a^0, L_a^1, \quad L_b^0, L_b^1, \quad L_c^0, L_c^1 \xleftarrow{\$} \{0, 1\}^\kappa.$$

The encoding information is $e = \{a \mapsto (L_a^0, L_a^1), b \mapsto (L_b^0, L_b^1)\}$; the decoding information is $d = \{L_c^0 \mapsto 0, L_c^1 \mapsto 1\}$.

2. Garbling. The four conceptual rows of the AND truth table, in label form:

(i, j)	$i \wedge j$	row
$(0, 0)$	0	$(L_a^0, L_b^0) \rightarrow L_c^0$
$(0, 1)$	0	$(L_a^0, L_b^1) \rightarrow L_c^0$
$(1, 0)$	0	$(L_a^1, L_b^0) \rightarrow L_c^0$
$(1, 1)$	1	$(L_a^1, L_b^1) \rightarrow L_c^1$

Encrypting each row under its input labels (Definition 3.2), with gate-specific nonce ν :

$$\begin{aligned} C_{(0,0)} &= H(L_a^0 \parallel L_b^0 \parallel \nu) \oplus (L_c^0 \parallel 0^t), \\ C_{(0,1)} &= H(L_a^0 \parallel L_b^1 \parallel \nu) \oplus (L_c^0 \parallel 0^t), \\ C_{(1,0)} &= H(L_a^1 \parallel L_b^0 \parallel \nu) \oplus (L_c^0 \parallel 0^t), \\ C_{(1,1)} &= H(L_a^1 \parallel L_b^1 \parallel \nu) \oplus (L_c^1 \parallel 0^t). \end{aligned}$$

The garbler samples a permutation $\pi_g \xleftarrow{\$} S_4$ and ships the shuffled tuple $(C_{\pi_g(1)}, C_{\pi_g(2)}, C_{\pi_g(3)}, C_{\pi_g(4)})$ as the garbled gate.

3. Encoding the input. For input $x = (a = 1, b = 0)$:

$$\text{En}(e, x) = (L_a^1, L_b^0) =: L_x.$$

The evaluator receives L_x together with the four shuffled ciphertexts and the public nonce ν . Note that the ordering of L_x , that is, first slot for wire a and second slot for wire b , is fixed by the circuit description, which is publicly available. In BitVM3 this generic input x is the public SNARK proof π , so $\text{En}(e, \pi) = L_\pi$ and the prover releases it not directly but by signing π via the GES of §8.

4. Evaluation. The evaluator computes $h := H(L_a^1 \parallel L_b^0 \parallel \nu)$ once, then for each ciphertext C in the shuffled tuple checks whether the last t bits of $h \oplus C$ are zero. Exactly one ciphertext, the one originally constructed as $C_{(1,0)}$, passes the check, since only it was encrypted under keys (L_a^1, L_b^0) ; the other three decryptions produce uniformly random $(\kappa + t)$ -bit strings, which end in 0^t only with probability 2^{-t} each. A union bound over those three rows bounds the chance of a spurious second match by $3 \cdot 2^{-t}$, negligible for $t = 128$ (Remark 3.2, Remark 2.6), so the correct row is identified with overwhelming probability. Stripping the pad from the successful decryption yields the output label

$$L_y = L_c^0.$$

5. Decoding. Finally,

$$\text{De}(d, L_y) = d(L_c^0) = 0,$$

recovering the plaintext output $y = 0 = 1 \wedge 0$, as required.

The procedure above garbles and evaluates a *single* gate; a real circuit has many, often millions, and each step (1–5) is repeated per gate in dependency (topological) order, with intermediate output labels carried forward as inputs to downstream gates; gates at the same depth are independent and may be evaluated in parallel (Remark 1.4). See §3.4. The four roles **Gb**, **En**, **Ev**, **De** are an abstraction of the garbling primitive; in any concrete protocol several collapse onto one party (in Yao’s original two-party protocol, Alice garbles, encodes, and produces the decoding information d , while Bob evaluates and decodes the output using d , obtaining his own input labels via oblivious transfer). For the protocol-level role assignments in BitVM3, which party plays garbler, which plays evaluator, and who adjudicates on-chain, see §13.1.

Remark 3.8 (What the evaluator learned). The evaluator now holds L_a^1, L_b^0, L_c^0 , three uniformly random κ -bit strings. They learned the plaintext output bit 0 via d , but cannot determine the input bits from the labels themselves: every label looks like a fresh random sample. Crucially, the shuffled position of the successful ciphertext within the tuple is a random index in $\{1, 2, 3, 4\}$ (independent of $(i, j) = (1, 0)$), so no information leaks through the row index either. This is the privacy property at work for a single gate; the authenticity property (Theorem 2.5) extends to the gate via the unforgeability of L_c^1 , a fresh random string the evaluator never sees in this evaluation.

Remark 3.9 (Oblivious transfer and BitVM3). In Yao’s classical two-party setting, the garbler holds both labels per input wire, but for the evaluator’s *own* input bits the evaluator must learn only the matching label without revealing the bit to the garbler – solved by 1-out-of-2 *oblivious transfer* (OT). OT is the only interactive step in plain Yao; the other artefacts (ciphertexts, the garbler’s own input labels, d) are one-shot. BitVM3 sidesteps OT entirely: its garbled input is a SNARK proof π that is public on chain, so the evaluator’s input labels (the single active label per input wire, $\text{En}(e, \pi)$) are released by the prover signing π under the *Garbled Encoding Extractable Signature* (GES) of §8, with the complementary half of e never exposed.

3.4 Chaining Gates: Multi-Gate Evaluation

Section 3.3 handled one gate. A general circuit has many; this subsection shows that no new construction is needed to handle them. **Ev** simply iterates the single-gate procedure of §3.3 in topological order: gates are processed by increasing height (a gate’s height is one more than the maximum height of its input wires), so every gate runs only after the gates feeding it. Gates of equal height are independent and may be processed in any order or in parallel (Remark 1.4), so the circuit’s depth, not its gate count, bounds the number of sequential layers.

The chaining mechanism. By Definition 3.1, each wire w of the circuit has exactly one label pair (L_w^0, L_w^1) , sampled once by the garbler. For an intermediate wire w that is the output of gate g and an input to gate g' :

- In g ’s garbled table, the labels L_w^0, L_w^1 appear as *plaintexts*: the messages encrypted by the rows of g ’s truth table.
- In g' ’s garbled table, the same labels L_w^0, L_w^1 appear as *keys*: they form (jointly with g' ’s other input wire’s labels) the keys under which each row of g' is encrypted.

So the label that the evaluator recovers when decrypting the right row of g is, by construction, exactly one of the keys needed to decrypt the right row of g' . The wiring of the plaintext circuit – which gate output feeds which gate input – is mirrored at the label level without any additional structure.

Multi-gate evaluation. For a circuit C with W wires and gates $g_1, \dots, g_G$ in topological order, **Ev** proceeds as follows. Maintain a buffer of recovered labels $B[1], \dots, B[W]$, initialised as unset. For each input wire $i \in \text{inputs}(C)$, set $B[i]$ to the corresponding label from $L_x = \text{En}(e, x)$. Then for each gate g_k in order:

- Read input labels from B at the indices of g_k ’s input wires.
- Run the single-gate trial-decryption procedure of §3.3 on g_k ’s shuffled ciphertexts.
- Write the recovered output label to B at the index of g_k ’s output wire.

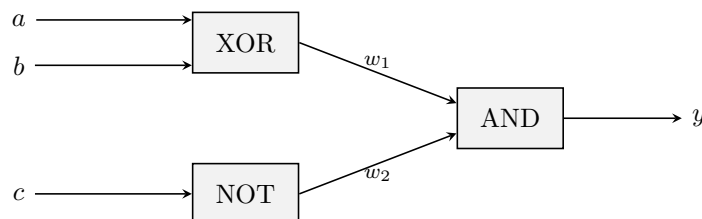
After all gates are processed, the labels $B[o_1], \dots, B[o_m]$ at the output-wire indices form L_y . The decoder $\text{De}(d, L_y)$ recovers the plaintext output.

What the evaluator does and does not learn. The evaluator learns the labels for input wires (from En) and discovers labels for every internal and output wire as **Ev** progresses. Only the output-wire labels are looked up in d ; intermediate labels are used as keys for downstream gates and otherwise discarded. By Definition 3.1, every label looks like a uniformly random κ -bit string, so the evaluator never recovers any plaintext intermediate bit. The decoding table d contains only the entries for output wires (e.g. 2 entries for a single-bit output, regardless of circuit size), which is what keeps $|d|$ bounded.

Example 3.10 (A 3-gate circuit). Consider the function

$$y = (a \oplus b) \wedge (\neg c).$$

Implemented with three gates over $\{\text{AND}, \text{XOR}, \text{NOT}\}$:



Six wires in total: three input wires a, b, c , two intermediate wires $w_1 = a \oplus b$ and $w_2 = \neg c$, and one output wire $y = w_1 \wedge w_2$. Topological order of gates: XOR, NOT, AND.

(i) **What the garbler produces.** For each of the six wires, sample a label pair, yielding 12 uniformly random κ -bit strings. For each gate, build the corresponding garbled gate table per §3.2: 4 ciphertexts for the XOR (2^2 rows), 2 ciphertexts for the NOT (2^1 rows), 4 ciphertexts for the AND. Shuffle each gate’s ciphertexts independently with permutations $\pi_{g_1}, \pi_{g_2}, \pi_{g_3}$ drawn from S_4, S_2, S_4 respectively. Package:

- F : the three shuffled gate tables (10 ciphertexts total).
- e : the input-wire mapping $\{a \mapsto (L_a^0, L_a^1), b \mapsto (L_b^0, L_b^1), c \mapsto (L_c^0, L_c^1)\}$.
- d : the output-wire mapping $\{L_y^0 \mapsto 0, L_y^1 \mapsto 1\}$ – a 2-entry table, independent of circuit size.

(ii) **What the evaluator gets.** For input $x = (a, b, c)$, the garbler sends:

- $L_x = (L_a^a, L_b^b, L_c^c)$: three input labels, one per input wire (the matching one of each pair).
- F : the 10 ciphertexts in shuffled form, with public gate-index nonces ν_1, ν_2, ν_3 .
- d : the two-entry output-decoding table.

The evaluator runs each gate in topological order: trial-decrypts g_1 ’s 4 ciphertexts using (L_a^a, L_b^b) to recover $L_{w_1}^{a \oplus b}$; trial-decrypts g_2 ’s 2 ciphertexts using L_c^c to recover $L_{w_2}^{\neg c}$; trial-decrypts g_3 ’s 4 ciphertexts using $(L_{w_1}^{a \oplus b}, L_{w_2}^{\neg c})$ to recover $L_y^{(a \oplus b) \wedge (\neg c)}$. Finally, $\text{De}(d, L_y) = y$.

(iii) **Misc.** The intermediate labels $L_{w_1}^{a \oplus b}$ and $L_{w_2}^{\neg c}$ are uniformly random κ -bit strings to the evaluator; they reveal nothing about the plaintext values $a \oplus b$ or $\neg c$. The other halves of every wire pair (L_w^{1-b} for each w and the relevant b) are never transmitted; they remain with the garbler and are essential to the authenticity argument of Theorem 2.5 – the evaluator cannot fabricate a valid label for the bit they did not receive without breaking the pseudorandomness of H .

4 Optimized Garbling: Free-XOR and Half-Gates

Plain Yao (§3.2) ships four ciphertexts per gate. A standard sequence of optimizations cuts this sharply, and BitVM3 ships the *privacy-free* end of that sequence, so it is the only one we derive in full (the “Privacy-free AND gate” below). The rest we state only at a high level. What separates them is not just how much they save but *what they require of the data and which security properties they keep*:

- **Point-and-permute** (Beaver–Micali–Rogaway, 1990), *performance only*: each wire label carries a *select bit*, the two labels of a wire holding complementary bits in an order that reveals nothing, and the two input labels’ select bits jointly index the single ciphertext to decrypt, removing trial decryption (§3.2). Ciphertext count and every security property are unchanged, so this is an implementation speed-up, optional for our purposes.
- **Free-XOR** (Kolesnikov–Schneider, 2008), *size*: a global offset Δ fixes $L_w^1 = L_w^0 \oplus \Delta$ on every wire, so XOR gates cost zero ciphertexts. The data restriction is structural, one secret Δ shared circuit-wide (and a correlation-robust hash), while correctness, authenticity, and privacy are all preserved.
- **Half-gates** (Zahur–Rosulek–Evans, 2015), *size*: each AND gate garbles to two ciphertexts with privacy still preserved; the second ciphertext exists only to hide the input bit.

- **Privacy-free variant** (§2.3), *size, by giving up privacy*: dropping the requirement to hide input bits deletes that second ciphertext, taking each AND to a *single* one. This is sound only when the input carries no secret, which is exactly BitVM3’s setting (the garbled input is an on-chain SNARK proof), and it is the variant BitVM3 ships. We derive it in full next.

Privacy-free AND gate, end to end. With Free-XOR every wire w carries labels W_0 and $W_1 = W_0 \oplus \Delta$ for one global secret offset Δ , and an XOR gate costs no ciphertext (its output label is the XOR of the two input labels). A privacy-free AND gate $c = a \wedge b$ is a *single* ciphertext. At garble time, before any input is fixed, the garbler sets the output false-label $C_0 := H(B_0)$ (so $C_1 = C_0 \oplus \Delta$) and transmits

$$T_E = H(B_0) \oplus H(B_1) \oplus A_0.$$

Note that T_E depends on *both* labels of b , so a single ciphertext already encodes both possible values of the unknown input. At evaluation time the evaluator knows the real bit b (legitimate in the privacy-free setting, since the input is public) and holds the labels A_a, B_b ; it outputs

$$\begin{cases} H(B_0) = C_0, & b = 0, \\ H(B_b) \oplus T_E \oplus A_a, & b = 1. \end{cases}$$

A direct check confirms the one ciphertext is correct for all four inputs:

- $b = 0$: output = C_0 for either a , matching $a \wedge 0 = 0$.
- $b = 1, a = 0$: here $A_a = A_0$, so output = $H(B_1) \oplus T_E \oplus A_0 = H(B_0) = C_0$, matching $0 \wedge 1 = 0$.
- $b = 1, a = 1$: here $A_a = A_0 \oplus \Delta$, so output = $C_0 \oplus \Delta = C_1$, matching $1 \wedge 1 = 1$.

The saving is therefore *not* “keep only the row for the known input.” At garble time no input is fixed, and T_E is not a stored truth-table row but a gadget correct for every (a, b) . The full-privacy construction needs a *second* ciphertext, a *generator half-gate* $T_G = H(A_0) \oplus H(A_1) \oplus r\Delta$ built from a secret random bit r , whose sole job is to hide b ’s value behind a permuted color bit. Knowing b makes that half-gate unnecessary: the privacy-free variant deletes T_G and keeps only the evaluator half-gate T_E , halving the per-AND cost from two ciphertexts to one. This is the construction behind the size claim of §2.3.

Total size, accounted. The garbled object is dominated by *ciphertexts*, not labels: with Free-XOR a wire’s label pair is just W and $W \oplus \Delta$, so labels are never shipped in bulk (only one active input label per input wire is released, via §8). Total size is thus the number of ciphertexts times the row size. Plain Yao charges every gate 4 ciphertexts, giving $4|C|(\kappa + t)$ (Remark 3.4). Free-XOR drops XOR gates to zero, so only the $|C_\wedge|$ AND gates are charged; row-reduction and half-gates take each AND from 4 to 2; and the privacy-free deletion of T_G takes it from 2 to 1. The shipped circuit therefore shrinks to $|C_\wedge|\kappa$ bits: a half-gate row is a bare label, carrying no 0^t marker, since the evaluator selects its row rather than searching by trial decryption, and the hash here emits κ bits, a narrower object than the dual-key H of Definition 3.2. This is a $4\times$ reduction per AND gate ($2\times$ from the optimizations, $2\times$ from dropping privacy), on top of XOR gates becoming free. The label-pair count never changes; only the ciphertext tables do.

Remark 4.1 (What BitVM3 actually garbles with, and the size frontier). The BitVM3 paper leaves the garbling scheme *abstract*: any privacy-free, correct, and authentic garbler suffices, and it fixes a concrete construction only for its cost estimates, namely a Yao-style Boolean garbled circuit for the Groth16 verifier whose non-free-gate count is read as half-gates (≈ 40

GB, one 16-byte ciphertext per non-free gate). The half-gate scheme of this section is therefore what the paper benchmarks. The garbling primitive is, however, the least settled part of the design. An earlier BitVM3 whitepaper used an *RSA-based* garbling scheme that was broken: a malicious evaluator forges wire labels using the extended Euclidean algorithm on coprime public exponents (Futoransky, Larotonda, Barbàra, ePrint 2025/1291), which retracted it. The current size frontier is *Argo* (Eagen and Lai, ePrint 2026/049), which garbles *arithmetic* circuits with homomorphic elliptic-curve MACs and claims a further $\sim 1000\times$ reduction (a garbled Groth16 verifier from tens of GB to an estimated ~ 25 MB), though it is not yet fully specified on Bitcoin. We teach half-gates because it is what the reference paper and the public implementations use; §14 returns to the RSA break and the Argo direction.

Part II: Committing and Revealing Labels

5 Signature Primitives: What Is Used Where

Three primitives recur, each doing a distinct job; this section aims to map their respective use cases, and subsequent sections aim to address each construction. To begin, a high level summary: *adaptor signatures reveal the proof’s garbled labels*, *Winternitz commits values into script*, and *cut-and-choose or a SNARK certifies the garbling*.

Primitive	Description	Role
Lamport (§6)	simplest hash-OTS	clean-model GES; pedagogy
Winternitz (§7)	compact hash-OTS <i>bit commitment</i>	committing <i>values</i> to Bitcoin script
GES (§8)	a <i>role</i> : signature to release $\mathbf{En}(e, x)$	proof’s input-label reveal
Schnorr adaptor (§9)	deployed realization of GES role	releasing proof labels on-chain

For a garbled SNARK-verifier circuit whose input is the proof π and whose output is the verdict, we can construct four basic categories to describe components:

1. **Input (proof labels):** $\mathbf{En}(e, \pi)$ is released by the GES: a Lamport hash-OTS in the clean model, a Schnorr adaptor signature in deployment (§9.6).
2. **Output (verdict):** The circuit is evaluated off-chain; a wrong verdict is punished by revealing the false-output label through an on-chain *hashlock* (§11.4), *not* a signature: the label is the preimage of the hash baked into the Bitcoin locking script’s hash check.
3. **Setup honesty:** That the garbling itself is faithful is certified by cut-and-choose (§10.2.1) or a SNARK-of-setup (§10.1); the resulting one-versus-two-SNARK distinction is unpacked in Remark 5.1.
4. **Auxiliary values:** Any other values which the protocol must pin to Bitcoin’s stateless script, e.g., light-client epoch commitments, transaction-graph connectors, program inputs, are a *Winternitz bit commitment* (§7). Auxiliary values are any value beyond the input (proof labels revealed by adaptors) and the output (hashlock gated verdict).

Remark 5.1 (Two-SNARK distinction). The proof π is itself an application Groth16 proof, and its *verifier* is the circuit being garbled (§12); this SNARK is always present. It is distinct from the *SNARK-of-setup*, which proves the garbling was done correctly (§10.1). That second SNARK is *optional*: it appears only in the SNARK-of-setup variant, in lieu of the cut-and-choose option (§10.2.1) which certifies the garbling using a two-party (sub-)protocol with no second SNARK at all. Neither SNARK is the GC encoding information e : e is produced by garbling, and the setup-SNARK (or cut-and-choose) merely certifies it.

Each of the above signing primitives is *one-time*: equivocation (attesting to two separate conditions, revealing two separate preimages), i.e., revealing multiple conflicting commitments, leaks a slashable secret. This single idea is realized in two forms: hash-OTS signature preimage equivocation (Lamport, Winternitz, and the bit commitments built on them; §6.2) and Schnorr nonce reuse (adaptors; §9.4). That self-punishing equivocation is what binds each party to a single value across the transaction graph.

6 Lamport One-Time Signatures

Lamport’s one-time signature is the clearest illustration of hash-based one-time signing, which is why we build the GES (§8) on it. Initial public testnet versions of BitVM3 implement *Schnorr adaptor signatures* as the on-chain mechanism to reveal GES labels instead of hash-based one-time signature; the reasons for that choice are discussed in (§9). Despite this, topics in this sections still carry over to the GES construction unchanged.

Lamport’s one-time signature¹⁰ is the simplest hash-based one-time signature, built from only a preimage-resistant hash function H , with fixed-length messages signed bit by bit. More compact hash-based generalizations exist, but Lamport is used here for its conceptual clarity. Its importance here is that Lamport *combines with* the garbled circuit of §3 to produce the *Garbled Encoding Extractable Signature* (GES) of §8: each garbled wire’s label pair (L_w^0, L_w^1) serves directly as the Lamport secret pair for that input bit, so a Lamport signature on an input $x \in \{0, 1\}^\ell$ is the garbled encoding $\text{En}(e, x)$. In BitVM3 (§13) this lets the prover release the input-wire labels through a single on-chain signature on the SNARK proof, i.e., the operators (prover) can provide all input-wire labels to challenge by posting just a single signature. The Lamport construction below therefore carries over to §8 unchanged, with the wire labels themselves playing the role of Lamport secrets.

Remark 6.1 (Where hash-based one-time signatures still appear in BitVM3). The garbled-label reveal uses Schnorr adaptor signatures (§9), not a hash-based one-time signature.¹¹ But Lamport and its more compact generalization *Winternitz* (§7, which signs several bits per hash chain) are not absent from the deployed protocols: they remain the general on-chain *bit-commitment* primitive for parts outside the label reveal, for example a light client committing each epoch-boundary block hash and timestamp (roughly every 2016 blocks),¹² and program-input signing in the BitVMX line.¹³ Winternitz trades signature size against hashing cost, but at roughly 25 witness bytes per signed bit¹⁴ it is far heavier on-chain than the adaptor encoding, which is why the label reveal itself moved away from it.

¹⁰L. Lamport, *Constructing Digital Signatures from a One-Way Function*, SRI International CSL-98 (1979).

¹¹The adaptor-signature encoding of the proof is BitVM3 §8.1 (<https://bitvm.org/bitvm3.pdf>); see also Glock (Eagen, Alpen Labs, 2025) and Mosaic (ePrint 2026/812, <https://github.com/alpenlabs/mosaic>). Reference implementation: `BitVM/garbled-snark-verifier` (`src/cut_and_choose/vsss/adaptor.rs`).

¹²The light-client epoch-boundary commitment via an on-chain Lamport/Winternitz signature is described in the BitVM3 paper’s light-client construction (<https://bitvm.org/bitvm3.pdf>).

¹³BitVMX is a separate BitVM variant that verifies arbitrary program execution on an emulated CPU (RISC-V style) by optimistic challenge-response, narrowing a dispute to a single instruction step checked on-chain, rather than by garbled circuits; its Winternitz-signed program inputs are treated in ESSPI (Lerner, Jonas, Futoransky, <https://arxiv.org/abs/2503.02772>). For hash-based one-time signatures as stateful-script bit commitments more generally, see <https://www.rootstocklabs.com/blog/exploring-lamport-and-winternitz-signatures-for-stateful-bitcoin-scripts/>.

¹⁴ESSPI reports ≈ 25 witness bytes per signed bit for Winternitz (<https://arxiv.org/abs/2503.02772>); BitVM2 measurements give ≈ 26 bytes per input bit (Bitlayer, <https://medium.com/@Bitlayer/bitvm-and-its-optimization-considerations-007da599d8ac>).

6.1 Construction and Unforgeability

The scheme operates on fixed-length n -bit messages using a hash function $H : \{0, 1\}^\kappa \rightarrow \{0, 1\}^\kappa$, assumed preimage-resistant.

Definition 6.1 (Preimage resistance). H is *preimage-resistant* (one-way) if, for $y = H(u)$ with $u \xleftarrow{\$} \{0, 1\}^\kappa$, every PPT adversary given y finds some u' with $H(u') = y$ only with probability $\text{negl}(\kappa)$. Every one-time-signature reduction in this part rests on this single assumption.

Definition 6.2 (Lamport one-time signature). $\text{Lamport} = (\text{KeyGen}, \text{Sign}, \text{Vrfy})$ for n -bit messages:

1. $(sk, pk) \leftarrow \text{KeyGen}(1^\kappa, n)$: for n -bit message, sample $2n$ independent secrets $s_{i,b} \xleftarrow{\$} \{0, 1\}^\kappa$ for $i \in \{0, \dots, n-1\}$ and $b \in \{0, 1\}$. Set $sk := (s_{i,b})_{i,b}$ and $pk := (H(s_{i,b}))_{i,b}$.
2. $\sigma \leftarrow \text{Sign}(sk, m)$: for $m = (m_0, \dots, m_{n-1}) \in \{0, 1\}^n$, output $\sigma := (s_{0,m_0}, s_{1,m_1}, \dots, s_{n-1,m_{n-1}})$.
3. $b \leftarrow \text{Vrfy}(pk, m, \sigma)$: parse $\sigma = (\sigma_0, \dots, \sigma_{n-1})$ and accept iff $H(\sigma_i) = pk_{i,m_i}$ for every $i \in \{0, \dots, n-1\}$.

Remark 6.2 (Signing is selection, not computation). A Lamport signature is literally a *subset* of the secret key: of the $2n$ secrets it reveals the n selected by the message, one per position (s_{i,m_i}) . Nothing is computed at signing time; the signer just picks. Verification is equally cheap: hash each revealed secret and check it against the matching public-key commitment, $H(s_{i,m_i}) = pk_{i,m_i}$. The n unrevealed secrets (the complementary bits) stay secret, which is what keeps the scheme one-time.

Example 6.3 (Lamport on a 3-bit message). Take $n = 3$ and message $m = 101$, so $m_0 = 1$, $m_1 = 0$, $m_2 = 1$.

KeyGen samples six independent κ -bit secrets $s_{i,b}$ for $i \in \{0, 1, 2\}$, $b \in \{0, 1\}$. The public key is the three pairs of their hashes,

$$pk = ((H(s_{0,0}), H(s_{0,1})), (H(s_{1,0}), H(s_{1,1})), (H(s_{2,0}), H(s_{2,1}))).$$

$\text{Sign}(sk, 101)$ reveals one secret per position, selected by the bit value:

$$\sigma = (s_{0,1}, s_{1,0}, s_{2,1}).$$

$\text{Vrfy}(pk, 101, \sigma)$ then checks three hashes at the slots of pk indexed by m :

$$H(\sigma_0) = pk_{0,1}, \quad H(\sigma_1) = pk_{1,0}, \quad H(\sigma_2) = pk_{2,1}.$$

All three hold by construction of pk , and the verifier accepts.

Remark 6.4 (Structure of pk). pk is best viewed as an $n \times 2$ array of hashes: row i is the pair $(pk_{i,0}, pk_{i,1}) = (H(s_{i,0}), H(s_{i,1}))$, with $pk_{i,b}$ the hash committed for bit-value b at position i . Both columns are committed at keygen, *before any message is chosen*, because the signer cannot yet predict m_i . The bit m_i then acts at verification time as a column index or bit selector, i.e., denoting which of the two pre-committed hashes $H(\sigma_i)$ must match. The total size of pk is $2n\kappa$ bits.

Theorem 6.5 (Correctness). For every $(sk, pk) \leftarrow \text{KeyGen}(1^\kappa, n)$ and every $m \in \{0, 1\}^n$,

$$\text{Vrfy}(pk, m, \text{Sign}(sk, m)) = 1.$$

Proof. $\text{Sign}(sk, m)$ returns $\sigma_i = s_{i,m_i}$ for each i . Then $H(\sigma_i) = H(s_{i,m_i}) = pk_{i,m_i}$ by construction of pk , satisfying the verifier's check at every position. $\square$

Theorem 6.6 (One-time unforgeability). For every PPT adversary $\mathcal{A}$, let

$$(sk, pk) \leftarrow \text{KeyGen}(1^\kappa, n), \quad m^* \leftarrow \mathcal{A}(pk), \quad (m', \sigma') \leftarrow \mathcal{A}(pk, \text{Sign}(sk, m^*)).$$

Then

$$\Pr[m' \neq m^* \wedge \text{Vrfy}(pk, m', \sigma') = 1] \leq \text{negl}(\kappa).$$

Proof. We argue by contraposition ($P \Rightarrow Q \iff \neg Q \Rightarrow \neg P$, where P is “ H is one-way” and Q is “Lamport is unforgeable”): rather than bound the forger directly, we show that a successful PPT forger yields a PPT algorithm inverting H . Since H is preimage-resistant (Definition 6.1), no such inverter exists, forcing the forging probability to be negligible.

Let $\mathcal{A}$ be a PPT forger with success probability

$$\varepsilon(\kappa) = \Pr[m' \neq m^* \wedge \text{Vrfy}(pk, m', \sigma') = 1] \quad (6.1)$$

where κ is the security parameter, i.e. the bit-length of the secrets and the hash.

As $m' \neq m^*$, the messages differ in at least one position, i^* , where $m'_{i^*} = 1 - m^*_{i^*}$. The single sign-query on m^* revealed $s_{i^*, m^*_{i^*}}$ but not the complementary secret $s_{i^*, m'_{i^*}}$, so $\mathcal{A}$ has seen only its image $pk_{i^*, m'_{i^*}}$. Verification of the forgery nonetheless requires

$$H(\sigma'_{i^*}) = pk_{i^*, m'_{i^*}}, \quad (6.2)$$

so any accepting σ'_{i^*} is a preimage under H of a value whose preimage $\mathcal{A}$ was never given.

From the forger $\mathcal{A}$ we build a PPT algorithm $\mathcal{B}$ that inverts H : given a challenge image $y = H(u)$, $u \xleftarrow{\$} \{0, 1\}^\kappa$, $\mathcal{B}$ must output a preimage of y . Treating $\mathcal{A}$ as a black-box subroutine (an algorithm it runs internally on inputs of its own choosing, reading off whatever $\mathcal{A}$ outputs), $\mathcal{B}$ draws $(i, b) \xleftarrow{\$} \{0, \dots, n-1\} \times \{0, 1\}$, samples the remaining $2n-1$ secrets and their images honestly, and sets $pk_{i,b} := y$; it then runs $\mathcal{A}$ on pk . To $\mathcal{A}$'s single query m^* it responds honestly when $m^*_i \neq b$, and aborts when $m^*_i = b$ (that query alone would require the unknown u). If $\mathcal{A}$ returns an accepting forgery with $(i^*, m'_{i^*}) = (i, b)$, then $H(\sigma'_i) = y$ and $\mathcal{B}$ outputs σ'_i .

The pair (i, b) is uniform and independent of $\mathcal{A}$'s view, so it coincides with the forgery position (i^*, m'_{i^*}) with probability $1/(2n)$. On that event $m^*_i = 1 - b \neq b$, so $\mathcal{B}$ does not abort and presents $\mathcal{A}$ with a perfectly simulated game. Hence

$$\Pr[\mathcal{B}(y) \in H^{-1}(y)] \geq \frac{\varepsilon(\kappa)}{2n}. \quad (6.3)$$

Preimage resistance bounds the left-hand side by $\text{negl}(\kappa)$, and since $2n$ is polynomial in κ while a polynomial times a negligible function is again negligible,¹⁵

$$\varepsilon(\kappa) \leq 2n \cdot \text{negl}(\kappa) = \text{negl}(\kappa). \quad (6.4)$$

□

Remark 6.7 (Why one-time). If the signer reuses pk to sign a second message $m'' \neq m^*$, every position i where $m^*_i \neq m''_i$ now exposes *both* preimages of pk_i : both $s_{i,0}$ and $s_{i,1}$ are public. With both preimages in hand for any position, an adversary can mint a valid signature on any third message that agrees with the union of revealed bits, breaking Theorem 6.6. The honest mitigation is one fresh keypair per signature; BitVM3 (§13) takes that route. §6.2 turns the same failure mode into a feature, via deliberately reused keys.

¹⁵Recall (Remark 2.6) that μ is *negligible* if for every constant $c > 0$ there is a κ_0 with $\mu(\kappa) < 1/\kappa^c$ for all $\kappa \geq \kappa_0$. Such a μ times a fixed polynomial p of degree d stays negligible: given c , invoke the bound at exponent $c + d + 1$; since $p(\kappa) \leq \kappa^{d+1}$ for large κ , we get $p(\kappa)\mu(\kappa) < \kappa^{d+1}/\kappa^{c+d+1} = 1/\kappa^c$.

6.2 Non-Equivocation via Reused Keys

The same Lamport keypair used twice was the failure mode of §6.1. Reversing perspective turns it into a feature: a verifier who can produce two signatures under the same pk on contradictory messages $m \neq m'$ holds cryptographic evidence that the key was reused, since the two signatures jointly reveal both preimages of pk_i at every position i where $m_i \neq m'_i$. The leaked preimages can be replayed as a fraud-proof witness (see Remark 6.8); BitVM3 (§3.4, “Non-Equivocation Signatures”) uses this to slash an operator who tries to claim two contradictory burn-events.

Definition 6.3 (Equivocation evidence). Let $(sk, pk) \leftarrow \text{KeyGen}(1^\kappa, n)$, and let σ, σ' be valid Lamport signatures under pk on messages $m, m' \in \{0, 1\}^n$ with $m \neq m'$. The tuple (m, σ, m', σ') is an *equivocation pair*¹⁶; for every position i with $m_i \neq m'_i$ it exposes the secret pair $\{s_{i,0}, s_{i,1}\} = \{\sigma_i, \sigma'_i\}$.

Remark 6.8 (One-bit equivocation suffices). A single position where the two messages disagree is enough to leak both preimages at that position, and one such leaked pair is enough to drive the slashing transaction: the fraud-proof demonstrates *some* reused pair, not a full key recovery. Larger Hamming distance between m and m' does not strengthen the proof, only its forensic value.

Remark 6.9 (Constant-size punishment). Verification of an equivocation pair is one hash equality per leaked position, so the on-chain fraud-proof is constant-size regardless of n . This is what makes the construction attractive when the off-chain commitment graph is large but the on-chain footprint of pk is fixed.

Remark 6.10 (Naive Lamport sizes in BitVM3). The Lamport public key is $2n\kappa$ bits in total: two κ -bit hashes per bit position, n positions. For BitVM3 with $\kappa = 128$ and n the bit-length of the 128-byte Groth16 proof ($n = 1024$), the BitVM3 paper’s evaluation prices the Lamport reveal at 74 leaf-script bytes plus a 20-byte witness preimage per bit, roughly 25 kB on-chain against 2.4 kB for the adaptor encoding (BitVM3 §8.1). The much-quoted 7.6 MB reveal arises in a different setting: Citrea’s post-quantum analysis garbles a verifier for a hash-based proof of ~ 14 kB at 80-bit security, and Lamport-signing every bit of that proof yields the 7.6 MB figure.¹⁷ This is large but Taproot-feasible: SegWit/Taproot witness elements carry the bulk, sidestepping both the legacy 520-byte single-push limit and the 10,000-byte legacy script cap that would block a pre-SegWit scriptSig from holding the key.

Remark 6.11 (Discrete-log analogue: extractable one-time signatures). The equivocation-to-slashing mechanism has a discrete-log *analogue*: the same structural idea (equivocation yields an extractable secret, which is the fraud proof) realized with a different primitive. Where a reused Lamport key leaks both hash *preimages*, a Schnorr key reused across two messages under one nonce leaks the *private key* itself (the nonce-reuse of §9.4); this is an *extractable one-time signature* (EOTS). Babylon’s Bitcoin Staking uses EOTS for exactly this: a finality provider that double-signs two blocks at the same height must reuse its nonce, exposing its key and enabling slashing.¹⁸

¹⁶The canonical treatment is Ruffing, Kate, and Schröder, *Liar, Liar, Coins on Fire! Penalizing Equivocation by Loss of Bitcoins*, ACM CCS 2015 (<https://dl.acm.org/doi/epdf/10.1145/2810103.2813686>), which introduces *accountable assertions* (a primitive whose equivocation reveals the party’s secret credentials) and *non-equivocation contracts* that turn this into a Bitcoin penalty.

¹⁷Citrea engineering write-up, <https://www.blog.citrea.xyz/post-quantum-citrea/>; adaptor and Lamport costs from the BitVM3 paper, <https://bitvm.org/bitvm3.pdf>, §8.1.

¹⁸Babylon, *Bitcoin Staking: Unlocking 21M Bitcoins to Secure the Proof-of-Stake Economy* (litepaper), which specifies EOTS and the double-sign slashing mechanism: [https://docs.babylonlabs.io/papers/btc_staking_litepaper\(EN\).pdf](https://docs.babylonlabs.io/papers/btc_staking_litepaper(EN).pdf).

7 Winternitz One-Time Signatures

Winternitz is the compact generalization of Lamport (§6). Lamport spends a secret *pair* per message bit and reveals one of the two; Winternitz instead signs w bits at a time using a single hash chain (Def. 7.1) per w -bit digit. The digit's *value* selects a *position* along that chain: to sign a digit d the signer reveals the secret sitting d steps in, $H^d(s)$, and the verifier hashes it the remaining $2^w - 1 - d$ times to reach the public chain tip $H^{2^w-1}(s)$. So with $w = 2$ the digit $d = 3$ (i.e. 11) is signed by revealing $H^3(s)$, which the verifier identifies as position 3 because 0 further hashes already land on the tip. One chain thus covers all 2^w values of the group, trading signature size against the cost of building and walking chains of length $2^w - 1$. Its role in BitVM3, the bit-commitment primitive rather than the label reveal, is introduced in §5; here we give the construction.

Definition 7.1 (Hash chain). A *hash chain* of length ℓ from a seed $s \in \{0, 1\}^\kappa$ is the sequence

$$s, H(s), H^2(s), \dots, H^\ell(s),$$

where H^k denotes H applied k times (so $H^2(s) = H(H(s))$), each element the hash of the one before. Moving *forward* from $H^k(s)$ to $H^{k'}(s)$ costs $k' - k$ hash evaluations; moving *backward* would invert H and is infeasible, so $H^k(s)$ determines every later element while hiding every earlier one. The cost of walking a chain is thus the *number* of steps, each a single hash of a fixed κ -bit value, so it grows with the chain length (2^w here), not with any per-hash input size.¹⁹

7.1 Construction

Fix a Winternitz parameter $w \geq 1$ (the number of message bits signed per chain) and let H be a preimage-resistant hash. We work in base 2^w (i.e. each digit holds one of 2^w values, from 0 to $2^w - 1$). To sign a message m , hash it to a fixed digest $H(m)$ and split that digest into w -bit chunks, reading each chunk as a base- 2^w digit. This gives ℓ_1 digits $d_1, \dots, d_{\ell_1}$, where the count ℓ_1 is how many w -bit chunks the digest occupies ($\ell_1 = \lceil |H(m)|/w \rceil$); each digit $d_i \in \{0, \dots, 2^w - 1\}$ is signed on its own chain.

Definition 7.2 (Winternitz one-time signature).

1. **KeyGen**: for each chain i sample a secret $sk_i \xleftarrow{\$} \{0, 1\}^\kappa$ and publish the chain end $pk_i = H^{2^w-1}(sk_i)$ (H applied $2^w - 1$ times).
2. **Sign**: for each digit d_i reveal $\sigma_i = H^{d_i}(sk_i)$ (walk the chain d_i steps from the secret).
3. **Vrfy**: accept iff $H^{2^w-1-d_i}(\sigma_i) = pk_i$ for every i (walk the remaining steps to the committed end).

Example 7.1 (Toy example: one chain). Take $w = 3$, so one digit ranges over $\{0, \dots, 7\}$ and a single chain has $2^3 = 8$ nodes joined by $2^3 - 1 = 7$ hashes: the secret $s = H^0(s)$ signs 000, its first hash $H^1(s)$ signs 001, and so on up to the tip $H^7(s) = pk$ signing 111. To sign 011 (digit 3) the signer reveals $H^3(s)$, and the verifier applies H the remaining $7 - 3 = 4$ times to recover pk ; every one of the 8 combinations is thus a distinct position on the same chain.

7.2 Checksum and Unforgeability

Remark 7.2 (Checksum). A *checksum* is a small value computed from data so that any change in the data changes the value. It is usually a tool for *error detection*, flagging accidental corruption in storage or transmission. Winternitz instead uses it explicitly for *forgery prevention*: the

¹⁹Hash chains originate with Lamport, *Password Authentication with Insecure Communication*, CACM 24(11):770–772 (1981): <https://lamport.azurewebsites.net/pubs/password.pdf>.

checksum C below is a signed function of the digits whose value moves opposite to any message digit, so a forger cannot raise one digit without being forced to lower another.

The chains alone are forgeable: since $\sigma_i = H^{d_i}(sk_i)$, anyone can hash forward to $H^{d_i+k}(sk_i)$, a valid signature for the *larger* digit $d_i + k$. To block this the signer appends a *checksum*

$$C = \sum_{i=1}^{\ell_1} (2^w - 1 - d_i),$$

a single integer, then *positionally encoded* back into base 2^w as ℓ_2 checksum digits $c_1, \dots, c_{\ell_2}$, where $C = \sum_{j=1}^{\ell_2} c_j (2^w)^{\ell_2-j}$ and each $c_j \in \{0, \dots, 2^w - 1\}$. The string actually signed is the concatenation of the message and checksum digits,

$$d_1, \dots, d_{\ell_1} \parallel c_1, \dots, c_{\ell_2},$$

one chain per digit, for $\ell_1 + \ell_2$ chains in total. The verifier does not receive C : it recomputes C from the message digits d_i , re-derives the c_j , and then checks *every* chain, message and checksum alike, by the **Vrfy** rule of Definition 7.2. Recomputation is what gives the checksum its force: the c_j are a fixed function of the message, not free input, so a forger cannot simply supply convenient checksum digits. Incrementing any message digit d_i *decreases* C , which forces *some* recomputed checksum digit c_j to *decrease*, and signing that lower digit means revealing an earlier chain node $H^{c_j-k}(sk)$, i.e. inverting H , which is infeasible. So forgery-by-chain-extension is blocked, and one-time unforgeability reduces to preimage resistance (Definition 6.1), exactly as for Lamport (§6.1).

Example 7.3 (Worked example: message and checksum combined). Take $w = 2$ (base 4, digits 0–3) and a toy 4-bit digest 1101. Split into 2-bit chunks: 11, 01, so $d_1 = 3$, $d_2 = 1$ on $\ell_1 = 2$ chains. The checksum is $C = (3 - 3) + (3 - 1) = 2$; its maximum is $2 \cdot 3 = 6$, needing $\ell_2 = 2$ base-4 digits, and $C = 2$ encodes as 02_4 , i.e. $c_1 = 0$, $c_2 = 2$. The signed string is $3, 1 \parallel 0, 2$: four digits, four chains, signature $(H^3(sk_1), H^1(sk_2), H^0(sk_3), H^2(sk_4))$. Now attempt a forgery: bump d_2 from 1 to 2 by hashing σ_2 once forward, which is free. The verifier recomputes $C = (3 - 3) + (3 - 2) = 1 = 01_4$, so the forger must drop c_2 from 2 to 1, i.e. produce $H^1(sk_4)$ from $H^2(sk_4)$: a preimage, and the forgery is stuck.

Example 7.4 (Checksum sizing). With $w = 4$ (base 16) and a 256-bit digest, the message occupies $\ell_1 = 64$ chains, each of length $2^4 - 1 = 15$. The checksum is at most $64 \cdot 15 = 960$, which fits in $\ell_2 = 3$ base-16 digits, giving 67 chains in total. Raising w to 8 halves the message chains to 32 (plus 2 checksum) but lengthens each to 255 hashes: the size-versus-time trade of Remark 7.5.

Remark 7.5 (Size vs. time, and the Lamport limit). Larger w packs more bits per chain, giving fewer chains and a shorter signature, at the cost of chains of length 2^w and thus more hashing per sign and verify. At $w = 1$ each chain has length 1 and one secret covers a single bit; this is the bit-granular regime of Lamport (§6), though not identical to it, since the checksum stands in for Lamport’s second per-bit secret. The one-time restriction still binds: signing a second message reuses chain secrets and, as in Lamport (Remark 6.7), leaks enough to forge. The GES of §8 is instantiated over the Lamport case for clarity; it carries over to Winternitz with chains in place of secret pairs.

8 Garbled Encoding Extractable Signatures

A *garbled encoding extractable signature* (GES) keys a signature scheme to one specific garbled circuit’s input encoding e , so that signing an input x and releasing its garbled encoding $\mathbf{En}(e, x)$

become a single act. It is the construction that *combines* the garbling scheme of §3 with the Lamport signature of §6: the label pair (L_i^0, L_i^1) that §3 assigns to the i -th input wire is reused as the Lamport secret pair for the i -th message bit, so a signature on x is the list of input labels $\mathbf{En}(e, x)$. A verifier holding such a signature can extract those labels and run $\mathbf{Ev}$ on them directly, with no separate label-delivery step. In BitVM3 (§13) this is what lets the operator release the input of a garbled SNARK-verifier circuit through a single on-chain signature on the proof π , revealing the labels for π and nothing else.²⁰

8.1 Syntax and Security

The instantiation works because Yao’s encoder is *projective*: the encoding information splits into one label pair per input wire, and $\mathbf{En}$ selects one label from each pair according to the corresponding input bit.

Definition 8.1 (Projective garbling scheme). A garbling scheme (Definition 2.1) is *projective* if, for a circuit of input length ℓ , the encoding information has the form $e = ((e_{i,0}, e_{i,1}))_{i=1}^\ell$ of one label pair per input wire, and the encoder selects componentwise:

$$\mathbf{En}(e, x) = (e_{1,x_1}, e_{2,x_2}, \dots, e_{\ell,x_\ell}).$$

Remark 8.1 (Yao is projective). The wire-label construction of §3.1 is projective: e is exactly the input-wire label pairs (Definition 3.1), and $\mathbf{En}$ returns the matching label of each pair (one per input bit). This per-bit structure is the hinge on which the GES instantiation rests, because it lines up one-to-one with Lamport’s per-bit secrets (2 committed values per position, one revealed). A non-projective scheme, whose encoding mixes input bits together, would have no such alignment.

Definition 8.2 (Garbled encoding extractable signature). Let $\mathbf{G} = (\mathbf{Gb}, \mathbf{En}, \mathbf{Ev}, \mathbf{De})$ be a projective garbling scheme, f a circuit of input length ℓ , and fix an honestly generated $(F, e, d) \leftarrow \mathbf{Gb}(1^\kappa, f)$. A *garbled encoding extractable signature scheme* for $\mathbf{G}$ is a tuple $\mathbf{GS} = (\mathbf{KeyGen}, \mathbf{Sign}, \mathbf{Vrfy}, \mathbf{Extract})$ of deterministic polynomial-time algorithms:

1. $(pk, sk) \leftarrow \mathbf{KeyGen}(1^\kappa, e)$: from the encoding information e , output a key pair. The secret key sk holds the input labels of e ; the public key pk commits to them. Determinism is well-defined because e already fixes the labels (the randomness lived in $\mathbf{Gb}$).
2. $\sigma \leftarrow \mathbf{Sign}(sk, x)$: for an input $x \in \{0, 1\}^\ell$, output a signature σ .
3. $b \leftarrow \mathbf{Vrfy}(pk, \sigma)$: output $b \in \{\text{True}, \text{False}\}$, equal to True iff σ is a valid signature under pk . There is no message argument.
4. $L_x \leftarrow \mathbf{Extract}(pk, \sigma)$: from a valid signature σ , output the encoded input L_x that σ reveals.

Remark 8.2 (Why $\mathbf{Vrfy}$ takes no message). Plain Lamport verifies against a supplied message: $\mathbf{Vrfy}(pk, m, \sigma)$ (Definition 6.2). A GES signature is instead *self-describing*. Each component reveals a preimage of one of the two committed hashes at its position, and which of the two it matches is the corresponding input bit. Validity is therefore checkable from (pk, σ) alone, and the message is not an input to verification but an output, recovered by $\mathbf{Extract}$ followed by $\mathbf{De}$. The signature does not attest to an externally chosen m ; it is the encoding of whichever x it commits to.

A secure GES satisfies three properties, stated for an honestly generated e and a fixed input length ℓ .

²⁰Definitions in this section follow the BitVM3 paper, §3.3 and Appendix B.4 (Linus Woll et al.), <https://bitvm.org/bitvm3.pdf>.

Theorem 8.3 (Correctness). For every $x \in \{0,1\}^\ell$, with $(pk, sk) \leftarrow \text{KeyGen}(1^\kappa, e)$ and $\sigma \leftarrow \text{Sign}(sk, x)$,

$$\Pr[\text{Vrfy}(pk, \sigma) = \text{True}] = 1.$$

Theorem 8.4 (Extractability). For every $x \in \{0,1\}^\ell$, with $(pk, sk) \leftarrow \text{KeyGen}(1^\kappa, e)$ and $\sigma \leftarrow \text{Sign}(sk, x)$,

$$\Pr[\text{Extract}(pk, \sigma) = \text{En}(e, x)] = 1.$$

That is, every honestly generated signature reveals exactly the garbled encoding of the signed input.

Theorem 8.5 (Unforgeability). For every PPT adversary $\mathcal{A}$, with $(pk, sk) \leftarrow \text{KeyGen}(1^\kappa, e)$ and $\sigma^* \leftarrow \mathcal{A}(1^\kappa, pk)$,

$$\Pr[\text{Vrfy}(pk, \sigma^*) = \text{True}] \leq \text{negl}(\kappa).$$

Remark 8.6 (Reading GES unforgeability). The game hands $\mathcal{A}$ only pk , with no signing oracle, and asks for *any* verifying signature. Since each position of pk is a pair of hashes whose preimages are the secret labels, producing a valid σ requires inverting H at every position, which is infeasible under preimage resistance. This is weaker than the EUF-CMA notion of §6, but it is what BitVM3 needs: the encoding stays hidden behind pk until the operator chooses to sign. The one-time discipline of §6.1 (one fresh key per proof) then stops a second signature from leaking the complementary labels, and the equivocation view of §6.2 is the same fact read forward, namely that a second signature under one pk is precisely what slashes a cheating operator.

8.2 Lamport-over-Input-Labels Instantiation

The canonical GES is a Lamport signature (§6) whose secrets are not freshly sampled but *taken to be* the input-wire labels of the garbled circuit. To align with Lamport’s subscript notation $s_{i,b}$, write the label that §3.1 denoted L_i^b (the bit- b label of the i -th input wire) as $L_{i,b}$ throughout this subsection. Let $H : \{0,1\}^\kappa \rightarrow \{0,1\}^\kappa$ be the preimage-resistant hash of §6, which is *not* the dual-key hash $H : \{0,1\}^* \rightarrow \{0,1\}^{\kappa+t}$ of Definition 3.2. The scheme:

1. **KeyGen** $(1^\kappa, e)$: parse the projective encoding $e = ((L_{i,0}, L_{i,1}))_{i=1}^\ell$. Set $sk := e$ and $pk := ((H(L_{i,0}), H(L_{i,1})))_{i=1}^\ell$.
2. **Sign** (sk, x) : output $\sigma := (L_{1,x_1}, L_{2,x_2}, \dots, L_{\ell,x_\ell})$.
3. **Vrfy** (pk, σ) : parse $\sigma = (\sigma_1, \dots, \sigma_\ell)$ and accept iff $H(\sigma_i) \in \{pk_{i,0}, pk_{i,1}\}$ for every $i \in \{1, \dots, \ell\}$.
4. **Extract** (pk, σ) : output σ itself.

The three properties of §8.1 are now immediate. By projectivity (Definition 8.1), $\text{Sign}(sk, x) = (L_{i,x_i})_{i=1}^\ell = \text{En}(e, x)$, so **Extract** returns $\text{En}(e, x)$ verbatim: extractability (Theorem 8.4) holds by construction, with **Extract** doing no work beyond parsing. Correctness (Theorem 8.3) holds because $H(L_{i,x_i}) = pk_{i,x_i} \in \{pk_{i,0}, pk_{i,1}\}$ at every position. Unforgeability (Theorem 8.5) is Lamport’s preimage argument (Theorem 6.6) read per position: a forger given only pk must exhibit a preimage of $pk_{i,0}$ or $pk_{i,1}$ to pass **Vrfy** at position i , and both are images of secret labels under H .

Example 8.7 (GES over a 2-input AND gate). Reuse the garbled AND gate of §3.3, with input wires a, b taken as positions 1, 2. The encoding is $e = ((L_{1,0}, L_{1,1}), (L_{2,0}, L_{2,1}))$, so

$$pk = ((H(L_{1,0}), H(L_{1,1})), (H(L_{2,0}), H(L_{2,1}))).$$

Signing the input $x = 10$ (that is $a = 1, b = 0$) reveals one label per position:

$$\sigma = (L_{1,1}, L_{2,0}).$$

Verification checks $H(L_{1,1}) = pk_{1,1}$ and $H(L_{2,0}) = pk_{2,0}$, both committed slots, and accepts. Extraction returns

$$\text{Extract}(pk, \sigma) = (L_{1,1}, L_{2,0}) = \text{En}(e, 10) = L_x,$$

which are the very labels the evaluator fed into Ev in §3.3 (there written (L_a^1, L_b^0)). One signature, and the challenger holds the garbled input ready to evaluate.

Remark 8.8 (Instantiation in BitVM3). In BitVM3-core (§13) the signed input is the SNARK proof, $x = \pi$, so ℓ is the proof bit-length and pk_{GS} is the 2ℓ committed hashes of Remark 6.10. Posting $\sigma_\pi \leftarrow \text{Sign}(sk_{GS}, \pi)$ on-chain releases $L_\pi = \text{En}(e, \pi)$; any challenger extracts L_π , evaluates the garbled verifier $L_y \leftarrow \text{Ev}(GC, L_\pi)$, and decodes $y \leftarrow \text{De}(d, L_y)$, posting a fraud proof if $y = \text{False}$. The on-chain predicate is written $\text{CheckGS}_{pk}(\sigma)$ for consistency with Bitcoin Script.

Remark 8.9 (Label width is forced by the adaptor flavour, not this one). The two GES instantiations react differently to the label width κ . The *adaptor* GES (§9.6) reads a label as a scalar and commits it as $T = [\ell]G$, so a narrow label leaves a kangaroo-recoverable discrete log and κ must be pushed near the field width (Remark 9.6, Remark 9.7). The *Lamport* GES above is indifferent to κ : $\text{KeyGen}, \text{Sign}, \text{Vrfy}, \text{Extract}$ treat a label only as a preimage of $H : \{0, 1\}^\kappa \rightarrow \{0, 1\}^\kappa$, so widening κ merely lengthens each secret label and its committed hash, changing no step of the construction and costing nothing beyond the extra bytes. Its security rests on preimage resistance, which is already full at $\kappa = 128$ (recovering a label is a 2^{128} preimage search, with no interval structure for kangaroo to exploit, contrast Remark 9.5); any extra width it carries is defensive slack inherited for the sibling adaptor flavour, not a demand of its own. This is exactly why, in code, a Lamport-over-labels GES whose secret type *is* the garbling label needs no edit when that label grows: the hash absorbs any input length, and only the byte count changes.

9 Schnorr Adaptor Signatures

An *adaptor signature* is a Schnorr signature deliberately offset by a secret t : the signer publishes a *pre-signature* $\tilde{\sigma}$ that does not yet verify under the standard Schnorr equation; whoever holds t can *adapt* it into a normal signature σ that does; and from the pair $(\tilde{\sigma}, \sigma)$ anyone can *extract* t by subtraction. This turns “publishing a signature on chain” into “atomically releasing a secret off chain”, which is the workhorse of scriptless scripts, atomic swaps, and Lightning channels – without any extra on-chain script. It is also what BitVM3 actually deploys to reveal garbled input labels on-chain: for the 128-byte Groth16 proof, a Lamport reveal (§6) costs roughly 94 witness bytes per proof bit (about 25 kB), Winternitz roughly 25, and the adaptor encoding about 8 (one 65-byte signature per 8-bit digit, about 2.4 kB in total), a 3–10× saving that is why the deployed systems moved to it (BitVM3 §8.1, Remark 6.10). The reveal is not free: even the adaptor encoding is a 65:1 expansion over the raw proof.

Components. Three algorithms on top of plain Schnorr (§9.2):

- $\tilde{\sigma} \leftarrow \text{PreSign}(sk, m, T)$ – pre-signature on message m bound to public statement $T = [t]G$.
- $\sigma \leftarrow \text{Adapt}(\tilde{\sigma}, t)$ – inject the witness, producing a real Schnorr signature.
- $t \leftarrow \text{Ext}(\tilde{\sigma}, \sigma)$ – recover the witness from any matching pre-/full-signature pair.

9.1 Motivation: Scriptless Scripts

A *scriptless script* enforces a spending condition without expressing it in on-chain script. The carrier is an ordinary signature: the transaction looks like a plain payment, yet completing and publishing that signature also discloses a secret to a designated party. Adaptor signatures are the mechanism. Because the condition lives in the signature algebra rather than in Bitcoin Script, the on-chain footprint is a single signature and the logic (atomic swaps, payment channels, and the BitVM3 label reveal of §13) is invisible to observers.

9.2 Schnorr Refresher

Fix a group of prime order n with generator G , written additively, so $[k]G$ denotes the k -fold sum of G . The signer holds a private key $x \in \{1, \dots, n-1\}$ and publishes the public key $P = [x]G$. The construction below is the (R, s) form of Schnorr; it is the baseline that the adaptor variant of §9.3 modifies, and it is implemented from scratch in `zkrareskillscourse/crypto/src/schnorr`.

Signing (message m):

1. sample a nonce $k \xleftarrow{\$} \{1, \dots, n-1\}$ and set $R = [k]G$;
2. compute the challenge $e = H(R \parallel P \parallel m) \pmod{n}$;
3. set $s = k + ex \pmod{n}$ and output the signature (R, s) .

Verification of (R, s) on m :

1. recompute $e = H(R \parallel P \parallel m)$;
2. accept iff $[s]G = R + [e]P$.

Correctness is the single identity the whole section rests on: expanding $s = k + ex$,

$$[s]G = [k + ex]G = [k]G + [e][x]G = R + [e]P. \quad (9.1)$$

Remark 9.1 (The homomorphism behind the verification identity). Both non-trivial steps of (9.1) are the single fact that scalar multiplication is a group homomorphism. Let $\varphi : \mathbb{Z}_n \rightarrow \mathbb{G}$, $k \mapsto [k]G$, map the additive group of scalars mod the prime order n into the curve group $\mathbb{G}$; it is a homomorphism, and since G generates a prime-order group, an isomorphism onto $\mathbb{G} = \langle G \rangle$ (the whole group is cyclic, generated by G). Additivity $\varphi(a + b) = \varphi(a) + \varphi(b)$ gives the first step $[k + ex]G = [k]G + [ex]G$, so a sum of scalars becomes a sum of points. The scalar action then gives $[ex]G = [e][x]G = [e]P$, since ex copies of G are e copies of $[x]G$. Here n is the group *order* (a prime), distinct from the base-field prime over which the curve is defined, and all scalar arithmetic ($k + ex$, ex) is mod n . This is the only structure the section uses; the adaptor of §9.3 and the Taproot tweak of §11.2 (Remark 11.2) reuse the same φ , so shifting or splitting a statement at the scalar level mirrors exactly on the points.

Remark 9.2 (The nonce point is what the adaptor tweaks). Verification (9.1) pins a signature to its nonce point R : the pair (R, s) verifies only against that specific R . The adaptor construction exploits exactly this, shifting R by a public statement T so that the pre-signature verifies against R while the *completed* signature verifies against $R + T$. This is why the (R, s) form, which exposes R , is the natural one here. (The $s = k + ex$ used above is equivalent to the $y = [-x]G$ convention where verification reads $R = [s]G + [e]y$; the sign choice only moves the negation.)

9.3 Pre-Signature, Adapt, Extract

An adaptor signature binds a signature to a *statement* $T = [t]G$, where the scalar t is the *witness*. The signer knows its key x and the public T , but not t .

Definition 9.1 (Schnorr adaptor signature). For a statement $T = [t]G$ and message m :

1. $\tilde{\sigma} = (R, \tilde{s}) \leftarrow \text{PreSign}(x, m, T)$: sample $k \xleftarrow{\$} \{1, \dots, n-1\}$, set $R = [k]G$, compute the challenge against the *shifted* nonce $e = H(R+T \parallel P \parallel m)$, and output $\tilde{s} = k + ex \pmod{n}$.
2. $\text{PreVrfy}(P, m, T, \tilde{\sigma})$: with $e = H(R+T \parallel P \parallel m)$, accept iff $[\tilde{s}]G = R + [e]P$.
3. $\sigma = (R+T, s) \leftarrow \text{Adapt}(\tilde{\sigma}, t)$: set $s = \tilde{s} + t \pmod{n}$.
4. $t \leftarrow \text{Ext}(\tilde{\sigma}, \sigma)$: output $t = s - \tilde{s} \pmod{n}$.

The subtlety is in **PreSign**: the challenge e is computed against the shifted nonce $R+T$, not R alone. The signer can do this without knowing t , since T is public and $R+T = [k]G + [t]G$. This is precisely what makes the adapted signature verify.

Proposition 9.3 (Adaptability and extractability). If $\tilde{\sigma} = (R, \tilde{s})$ passes **PreVrfy** for the statement $T = [t]G$ on message m , then $\sigma = \text{Adapt}(\tilde{\sigma}, t)$ is a valid Schnorr signature on m , and $\text{Ext}(\tilde{\sigma}, \sigma) = t$.

Proof. The adapted signature is (R', s) with $R' = R+T$ and $s = \tilde{s} + t$. Its verification challenge is $e' = H(R' \parallel P \parallel m) = H(R+T \parallel P \parallel m) = e$, the same challenge **PreSign** used. Then, using **PreVrfy** (i.e. $[\tilde{s}]G = R + [e]P$) for the middle equality,

$$[s]G = [\tilde{s}]G + [t]G = (R + [e]P) + T = (R+T) + [e]P = R' + [e']P,$$

so (R', s) verifies. Extraction is immediate: $s - \tilde{s} = (\tilde{s} + t) - \tilde{s} = t$. $\square$

9.4 Security Properties

Beyond correctness, a secure adaptor signature scheme satisfies three properties.

1. **Pre-signature adaptability.** Any pre-signature valid under **PreVrfy** for a statement T can be completed into a valid signature given the witness t (Proposition 9.3).
2. **Witness extractability.** From a valid pre-signature and a valid *adapted* signature for the same (m, T) , the witness t is recovered (Proposition 9.3). Publishing the completed signature therefore leaks t to anyone holding the pre-signature.
3. **Pre-signature unforgeability.** No PPT adversary, even given a signing oracle, can produce a pre-signature that passes **PreVrfy** without the secret key x ; this is the adaptor analogue of EUF-CMA. Since **PreVrfy** is the Schnorr verification equation with the challenge merely hashed over $R+T$ instead of R , forging a pre-signature is essentially forging a plain Schnorr signature; the intuition is that unforgeability rests on Schnorr's, hence on discrete-log hardness, while the statement T only adds the extractable witness. The full result (adaptor AEUF-CMA) also needs Schnorr's *strong* unforgeability and hardness of the statement relation.²¹²²

Remark 9.4 (One-time discipline). As with plain Schnorr, the nonce k must be fresh per signature: two signatures under the same R expose x . Explicitly, for a wire whose two pre-signatures share nonce k (so $R = [k]G$) under key x and message m , with statements T_0, T_1 and challenges $e_b = H(R + T_b \parallel P \parallel m)$,

$$\tilde{s}_0 = k + e_0x, \quad \tilde{s}_1 = k + e_1x \quad \implies \quad x = \frac{\tilde{s}_0 - \tilde{s}_1}{e_0 - e_1},$$

²¹The statement relation is discrete log (recover t from $T = [t]G$): generically $O(\sqrt{n})$ group operations via Pollard's rho (n the group order), or $O(\sqrt{L})$ by the kangaroo variant when t is confined to an interval of width L , so a b -bit witness gives only $\approx b/2$ -bit security.

²²Formal treatment in *Foundations of Adaptor Signatures*, ePrint 2024/1809.

since k cancels in the difference and both e_b are publicly computable (T_0, T_1, P, m are public). Two openings under one R therefore hand any observer the key. In BitVM3 this hazard is turned into a feature, mirroring the equivocation-slashing of §6.2: a second opening is exactly what slashes a cheating operator.

9.5 Bridge to GES: Two Flavours of Extractability

The GES of §8 and the adaptor signature above are the same idea in two settings: publishing a signature on-chain releases a payload off-chain. They differ in what the payload is and who can read it.

	payload released	who extracts
GES (§8)	$\text{En}(e, x)$, the input labels	any observer of the chain
Adaptor (§9.3)	t , a discrete-log witness	the holder of the pre-signature

In GES the signature *is* the encoding, so extraction is a parse; in the adaptor case the signature and its pre-signature jointly *determine* t by subtraction. BitVM3-core uses GES in its clean model but realizes the on-chain reveal with adaptor signatures (§13), so both flavours appear in the deployed protocol.

9.6 Realizing GES with Adaptor Signatures

This is the deployed reveal mechanism, stated here as a construction rather than an aside: it is one scheme instantiated two ways, not two schemes. The Lamport GES of §8 releases each input label as a hash preimage; the deployed GES keeps the same interface (§8.1) but swaps the release mechanism to the adaptor signature of this section. Since an adaptor signature is a Schnorr signature whose nonce is shifted by a statement $T = [t]G$, and completing it releases the witness t (§9.3), taking each input label to *be* a witness turns “post the claim signature” into “release the encoding.”

Definition 9.2 (Adaptor-GES over input labels). Let $\mathbf{G}$ be a projective garbling scheme with input-wire label pairs $e = ((\ell_{i,0}, \ell_{i,1}))_{i=1}^\ell$ (each label read as a scalar), let x be the operator’s Schnorr key (§9.2), let m be the Assert transaction, and let the signed input be the proof π .

1. **KeyGen**(e): for each input wire i and bit $b \in \{0, 1\}$, publish the statement $T_{i,b} = [\ell_{i,b}]G$, committing *both* labels of the wire (as the Lamport pk_{GS} commits both hashes, §8.2).
2. **Sign**(x, m): pre-sign both statements of every wire, $\tilde{\sigma}_{i,b} \leftarrow \text{PreSign}(x, m, T_{i,b})$ (Definition 9.1), and publish them.
3. **Post**(x, m, π): to claim on π , complete only the selected pre-signatures, $\sigma_i \leftarrow \text{Adapt}(\tilde{\sigma}_{i,\pi_i}, \ell_{i,\pi_i})$, and place them in the Assert witness.
4. **Extract**: recover $\ell_{i,\pi_i} = s_i - \tilde{s}_{i,\pi_i}$ for every i , i.e. the encoding $\text{En}(e, \pi)$.

Remark 9.5 (Why a point, not a hash). Committing a label as $T = [\ell]G$ is a *group homomorphism* $\ell \mapsto [\ell]G$ from the scalar field into the elliptic-curve group, so the label’s additive structure is preserved: statements can be shifted ($R + T$) and the witness recovered by scalar subtraction ($s - \tilde{s} = \ell$). The discrete-logarithm assumption hides ℓ while leaving this arithmetic available. A hash commitment $H(\ell)$ is instead *one-way* with no algebraic structure: it supports only reveal-and-compare, which is why the Lamport GES (§8.2) can *open* a label but never *adapt* it.

Remark 9.6 (Encoding labels as witnesses). The construction reads each label as a scalar, so the map from label bytes to $\mathbb{Z}_q$ must be *injective and reversible*: **Extract** has to return the exact label the evaluator will feed to **Ev**, so a hash (irreversible) will not do. A 128-bit label embeds

injectively (it is below q), but then the witness lives in a 128-bit sub-range, and the discrete log of $T = [\ell]G$ is recoverable in $\approx 2^{64}$ by Pollard’s kangaroo (Remark 9.7), well under the intended level. Full security therefore requires labels near the field width, mapped injectively (canonical reduction, or rejection sampling to stay below the group order so the map inverts). Widening the label is a garbling-parameter choice; a cleverer byte-to-scalar map cannot add entropy a 128-bit label does not have. The accompanying implementation widens to 32-byte labels with the top byte forced to zero: witnesses then range over a 2^{248} interval, costing $\approx 2^{124}$ kangaroo steps to recover, four bits of margin (security level in bits) shy of the $\approx 2^{128}$ that rejection sampling below the group order would give.

Remark 9.7 (Pollard’s kangaroo and interval discrete log). *Pollard’s kangaroo* (or lambda) method solves discrete log when the exponent is known to lie in an interval of width L , in $\approx \sqrt{L}$ group operations, against the $\approx \sqrt{n}$ of a full-range search over a group of order n . Two pseudo-random walks, a *tame* kangaroo started from a known point and a *wild* one from the target $T = [\ell]G$, hop across the interval until their trajectories collide; the accumulated offset at collision then yields ℓ . The consequence used above: interval width, not group order, sets a clamped label’s security, so a b -bit interval gives only a $\approx b/2$ -bit margin (security level in bits). Confining a witness to a narrow interval, as reading a 128-bit label directly as a scalar does, is therefore self-defeating; labels must span close to the full width (Remark 9.6).

Extractability (§8) holds verbatim: completing the selected pre-signatures to post the claim is the *same act* as releasing $\text{En}(e, \pi)$, and any challenger holding the public pre-signatures runs **Extract**. Completing *both* statements of a wire would reveal both its labels: that is equivocation, and the reused nonce then leaks the operator’s key, the slashing lever (non-equivocation). Correctness is immediate from the adaptor construction; unforgeability rests on the pre-signature unforgeability of §9.4. In deployment the positions are carried under a single public key with `OP_CODESEPARATOR` (§11.4).

Remark 9.8 (Who must hold the signing key). Definition 9.2 signs under the operator’s own key x , and this is a real gap, inherited from the paper’s underspecified deployment sketch. GS soundness needs *every* valid signature to extract the encoding, even one produced by the holder of sk ; the Lamport GES has this unconditionally (a valid signature *is* the preimages; Proposition 9.9), but an adaptor GES under an operator-known x does not: the operator can satisfy each `OP_CHECKSIGVERIFY` with a fresh vanilla BIP340 signature (witness data is covered by no sighash, §11.3), posting a valid Assert that reveals nothing. The repair is a slot key with no surviving signing exponent: generated jointly at setup, used only to issue the pre-signatures, its shares then deleted, mirroring the covenant committee’s deletion assumption, so that completing a pre-signature with a label is the only way any slot signature can ever exist. The BitVM3 paper asserts the extract-from-any-valid-signature property but does not specify this key arrangement for its adaptor encoding, and its formal extractability definition quantifies only over honestly generated signatures, weaker than what its own soundness proof uses.

Proposition 9.9 (Universal extractability of the Lamport GES). Let $pk = ((H(L_{i,0}), H(L_{i,1})))_{i=1}^\ell$ be a Lamport-GES public key (§8.2) with H collision-resistant. Then every σ , however produced, with $\text{Vrfy}(pk, \sigma) = \text{True}$ satisfies $\text{Extract}(pk, \sigma) = \text{En}(e, x)$ for some $x \in \{0, 1\}^\ell$, except with negligible probability.

Proof. Fix a position i . Acceptance requires $H(\sigma_i) = pk_{i,b_i} = H(L_{i,b_i})$ for some $b_i \in \{0, 1\}$. If $\sigma_i \neq L_{i,b_i}$, the pair (σ_i, L_{i,b_i}) is a collision for H , contradicting collision resistance. Hence $\sigma_i = L_{i,b_i}$ at every position; setting $x := (b_1, \dots, b_\ell)$ gives $\sigma = (L_{i,x_i})_{i=1}^\ell = \text{En}(e, x)$, which is what **Extract** returns. The acceptance predicate ranges over label tuples and nothing else: no signer, honest or malicious, has any satisfying assignment that is not an encoding. $\square$

The adaptor variant fails at exactly this point when a signing exponent survives setup. A slot’s

consensus check accepts *any* pair (R', s') with

$$[s']G = R' + [e']P, \quad e' = H(R' \parallel P \parallel m),$$

and the operator, knowing x , mints one afresh: $k' \xleftarrow{\$} \{1, \dots, n-1\}$, $R' = [k']G$, $s' = k' + e'x$. This verifies, yet is no completion of either pre-signature: $R' \neq R_i + T_{i,b}$ except with negligible probability, and the challenger's subtraction returns

$$s' - \tilde{s}_{i,b} = (k' - k_i) + (e' - e_{i,b})x,$$

a scalar carrying a fresh random nonce, not a label. The acceptance set is parameterized by the free nonce k' and strictly contains the two completions; extractability holds only on that two-element subset, while membership in the set at large requires only x . Deleting every share of x once the pre-signatures are issued collapses the *computable* acceptance set to exactly those completions, which restores extraction.

9.7 Wide Labels: Mapping Bits to Digits

Definition 9.2 releases one label per *wire*: 1024 signatures per claim (π 128 bytes or 1014 bits). To reduce the on chain assert transaction costs (data/witness size) deployment instead groups the 1024 input wires into 128 digits of 8 bits and spends one 65-byte signature $(R, \tilde{s})$ per digit (the script gadget carrying the 128 slots: §11.4, Remark 13.4). A signature releases a single scalar, but a digit's assignment selects 8 bit-labels; the gap is closed by releasing a *key* rather than the labels themselves: the labels sit published but encrypted in a lookup row, and the signature's scalar, the *wide label*, unlocks them.

Definition 9.3 (Wide-label reveal, single instance). Fix the Assert transaction and write m_d for slot d 's sighash: the BIP342 script-spend digest committing to the entire Assert (inputs and outputs, §11.3), the executing tapleaf, and the opcode position of slot d 's preceding OP_CODESEPARATOR. The separator position is the only field varying across slots, so there are exactly 128 pairwise-distinct messages, each shared by its slot's 256 candidate pre-signatures (Remark 13.7). For each digit $d \in [128]$ and candidate value $v \in \{0, \dots, 255\}$ the garbler:

1. *samples the key*: $W_{d,v} \xleftarrow{\$} \mathbb{Z}_q$, with statement $T_{d,v} = [W_{d,v}]G$;
2. *pre-signs slot d against it*: with a fresh nonce $k_{d,v}$ (distinct for every pair (d, v)), $R_{d,v} = [k_{d,v}]G$ and

$$\tilde{s}_{d,v} = k_{d,v} + e_{d,v}x, \quad e_{d,v} = H(R_{d,v} + T_{d,v} \parallel P \parallel m_d),$$

the PreSign of Definition 9.1 under the slot key x , with $P = [x]G = pk_{slot}$ the one key every CHECKSIGVERIFY of the leaf checks; no operator signature gates the spend, and the slot key's shares are deleted after these pre-signatures are issued (Remark 9.8), so completions are the only signatures that can ever exist. The statement enters through the challenge, so this pre-signature completes only with $W_{d,v}$ itself (Remark 13.6) and verifies only against slot d 's message;

3. *locks the labels behind the key*: having garbled the circuit, the garbler holds every bit-label; it derives the symmetric key $H(W_{d,v})$, a hash of the scalar's canonical serialization, and publishes $\text{row}_{d,v} = \text{Enc}_{H(W_{d,v})}(L_{8(d-1)+1, v_1}, \dots, L_{8d, v_8})$, the 8 bit-labels selected by v 's bits under that key (worked byte-level in Appendix A).

The tuples $(T_{d,v}, R_{d,v}, \tilde{s}_{d,v}, \text{row}_{d,v})$ are published off-chain and are the entire reveal state. At claim time the operator completes one pre-signature per slot, $s = \tilde{s}_{d,v_d} + W_{d,v_d}$; the on-chain witness carries only these 128 completed signatures (≈ 2.4 kB). Setup therefore issues $128 \times 256 = 32,768$ pre-signatures per Assert template (under cut-and-choose, one such set

per evaluation instance; Remark 10.8). Under the sharing of §10.2.2 the statements need not be published, being derivable from the Feldman coefficient commitments, and the quadruple shrinks to the triple $(R, \tilde{s}, \text{row})$.

Remark 9.10 (Why full-width sampling is what makes this secure). $W_{d,v}$ is sampled uniformly over all $\sim 2^{256} \approx 2^{3.33 \times 76.9} \approx 10^7$ scalars, represented by 64 hex characters. The discrete log of $T_{d,v}$ is not known to lie in any interval smaller than the full range $[0, q)$, so Pollard’s kangaroo (Remark 9.7), whose cost is the square root of the *interval width*, degenerates to the generic $\sim \sqrt{q} \approx 2^{128}$ bound; contrast the per-wire adaptor witness, a 128-bit bit-label read as a scalar, whose 2^{128} -wide interval falls in $\approx 2^{64}$.

Conversely, committing a scalar to a label is never an issue: W is never reduced or truncated into a label; it enters only as the preimage of the symmetric key $H(W)$ for a 256-bit (or more) hash H , and the bit-labels are sampled independently of W (then concatenated and XORed with the keystream of this hash digest, see App. A.1). Consequently no scalar-to-label map exists whose non-injectivity could induce collisions; the injectivity requirement on a label-to-scalar encoding arises only in the per-wire construction (Remark 9.6), where the label itself is the witness.

Before a claim each $W_{d,v}$ is hidden by discrete log and each row by the cipher; at the claim, a slot signature is consensus-valid only if it completes a pre-signature with exactly the committed scalar (Remark 13.6), so a valid Assert necessarily releases genuine keys. Revealing two values for one digit is equivocation, and it is punished through evaluation rather than key extraction: the two revealed keys decrypt two rows, giving that digit’s labels under two bytes, and challengers splice them, encoding a mutated π' differing from π in that digit alone. By SNARK soundness π' is invalid bar negligible probability, so evaluation hands the challenger the false-output label, the Disprove preimage, and the bond is slashed; the sibling completion need never confirm on-chain (the bonded output is spent once), possession of the material alone sufficing, and the nonce-reuse leak of Remark 9.4 plays no role since each of a slot’s 256 pre-signatures carries a distinct nonce, a shared nonce leaking the signing key from the published $\tilde{s}$ already at setup (§9.4).

Example 9.11 (Challenger decoding, one digit). The Assert witness holds 128 signatures. For slot d , from public data alone:

1. read (R', s) at position d (position identifies d : slot sighashes are pairwise distinct, Remark 13.7);
2. identify the value: test $R' = R_{d,v} + T_{d,v}$ over the 256 published candidates; the unique match is v_d , the digit’s revealed value;
3. extract the key: $W = s - \tilde{s}_{d,v_d}$;
4. decrypt: derive the symmetric key $H(W)$ from the extracted scalar’s canonical serialization, exactly as the garbler did at setup; $\text{Dec}_{H(W)}(\text{row}_{d,v_d})$ yields the 8 bit-labels of wires $8(d-1)+1, \dots, 8d$. The row already *contains* the labels, concatenated at setup; $H(W)$ merely unlocks them (keystream mechanics worked byte-level in Appendix A).

Concatenating over all 128 slots gives exactly $\text{En}(e, \pi)$. The value v_d is revealed *positionally*, by which statement matches; no arithmetic on the scalar encodes it. This flow serves any single-instance setup certification (§10.1, §10.3); under cut-and-choose the extracted scalar is additionally a Shamir share, and interpolation precedes step (iv) per kept instance (§10.2.2).

Remark 9.12 (Anatomy of the 65 bytes). Each completed signature is $32 + 32 + 1$ bytes: the nonce point serialized *x-only*, the scalar s in full, and a sighash-type flag (omitted under `SIGHASH_DEFAULT`, giving 64). X-only serialization is sound because a point’s y -coordinate is redundant up to sign. The curve is defined over the prime field $\mathbb{F}_p$, $p = 2^{256} - 2^{32} - 977$ (the

base-field prime, distinct from the group order n), by the short Weierstrass equation $y^2 = x^3 + 7$; for any valid x this has exactly the two mirror solutions y and $p - y$, negatives of one another: $(x, p - y) = -(x, y) = [-1](x, y)$, group negation reflecting a point over the x-axis, so dropping y loses only this one sign bit, which the parity convention restores. As p is odd, y and $p - y$ have opposite parity: for suitable $n, k, \tilde{k} \in \mathbb{Z}_{\geq 0}$, let $p = 2n + 1$, if $y = 2k$ then $p - y = 2(n - k) + 1$, and if $y = 2\tilde{k} + 1$, then $p - y = 2(n - \tilde{k})$. BIP340 fixes the convention of taking the even root y ; the verifier (every validating node's script interpreter,²³ and any challenger re-checking off-chain) recomputes y from x and selects it: derive y from x and return point (x, y) if y even, else $(x, p - y)$. The choice is always well defined: $y = p - y$ would force $y = 0$, i.e. a $g \in G$ with $g = -g$, so $2g = e$ and g has order two; however, via Lagrange, element orders divide the group order, and secp256k1's order n is a large odd prime, so no such g exists.

Remark 9.13 (One signature, eight labels). A completed adaptor signature releases one scalar of at most 256 bits, but a digit's bit-labels total $8 \times 256 = 2048$ bits: no scalar can carry them. The wide label resolves this by indirection: the label material sits published but encrypted in the row, and the released scalar is only its key. This is also why the encoding is byte-granular; a per-bit encoding would need eight signatures per digit, forfeiting the witness compression of §13.

Remark 9.14 (The digit-width dial). Byte granularity is a tuning choice: w -bit digits need $1024/w$ on-chain signatures but $(1024/w) 2^w$ wide labels, pre-signatures, and lookup rows (and, under the sharing of §10.2.2, polynomials), so each doubling of w halves the witness while multiplying off-chain state exponentially. $w = 8$ is the knee of the curve, the same signature-size-versus-exponential-work trade as Winternitz (Remark 7.5).

9.8 Application: Atomic Swap

Example 9.15 (Cross-chain atomic swap). Alice holds coin A and Bob holds coin B on a second chain, and they wish to exchange atomically: either both transfers happen or neither does. Bob samples a witness t and publishes the statement $T = [t]G$. On each chain a spending transaction is prepared, and each party produces a *pre-signature* on the other's transaction bound to the same T ; neither pre-signature is yet a valid signature. Bob, who knows t , adapts the pre-signature that pays him coin A and broadcasts the completed signature, claiming A . The instant it is on-chain, Alice runs **Ext** on it together with her pre-signature and recovers t (witness extractability); with t she adapts the pre-signature that pays her coin B . One broadcast reveals the witness and the second transfer follows atomically, with no hash-timelock or on-chain contract: the coordination lives entirely in the signatures.

Remark 9.16 (Staggered refund timelocks). Each locked coin also carries a refund branch, spendable by its original owner after a timeout, in case the counterparty goes silent. These two timeouts must *not* share a deadline. The party who claims *second*, using the just-revealed witness t (Alice, on coin B), needs a strictly later refund timeout than the coin claimed *first* (Bob, on coin A). Otherwise a malicious first mover reveals t at the last moment and races his own refund against the honest second claim: if both refunds opened together, Bob could claim A and refund B , taking both coins. Staggering guarantees a window in which only the honest second claim, not a refund, can succeed. This is the same asymmetric-timelock requirement as in hash-timelocked contracts (HTLCs), and the refund branch is the timeout leaf of the Assert/Timeout templates in §11.6.

²³Bitcoin Core v28.0: `CheckSchnorrSignature`, <https://github.com/bitcoin/bitcoin/blob/v28.0/src/script/interpreter.cpp#L1673>, which also handles the 64-versus-65-byte sighash-flag cases, calls `XOnlyPubKey::VerifySchnorr`, <https://github.com/bitcoin/bitcoin/blob/v28.0/src/pubkey.cpp#L232>, whose x-only key parse performs the even- y lift inside libsecp256k1.

Part III: Protocol and On-Chain Realization

10 Setup Verification

The authenticity property of §2.2 stops a malicious *evaluator* from forging output labels, but says nothing about a malicious *garbler*. Since the operator garbles its own circuit, nothing so far prevents it from garbling one that computes the *wrong* verdict, for instance a circuit that outputs True on every proof. Setup verification closes this gap: it convinces the verifiers that the *GC*, the committed labels, and the Assert transaction are a faithful garbling of the agreed `SNARK.Vrfy` public circuit. This is a load-bearing architectural choice, and this integrity is achieved through either one of following methods: a *cryptographic* route (garble one copy, prove it correct; §10.1), a *statistical* route (garble many copies, open a random subset; §10.2),²⁴, and a *hardware* route (garble one copy inside an attested enclave; §10.3).

Previously, theorem 2.4, we established that a garbling (GC, e, d) of a function f is *correct* if $\text{De}(d, \text{Ev}(GC, \text{En}(e, x))) = f(x)$ for every input x . This is the property the protocol needs of the garbled verifier: that it computes the agreed public $f = \text{SNARK.Vrfy}$, and in particular yields the false-output label exactly on invalid proofs. Correctness holds automatically for an honest garbler (Bellare–Hoang–Rogaway); the difficulty is that the operator garbles its own circuit and could instead commit a corrupt GC which outputs True on every proof.

Additionally, correctness itself cannot be checked directly: it quantifies over all 2^{1024} inputs, and would involve inspecting a garbled circuit by opening its seed, which exposes every wire’s label pair and thus destroys its usability for binding evaluation, since a circuit whose full encoding info e is public is no longer secure (see § 12.2, point 4.).

Definition 10.1 (Soundness of setup verification). A setup-verification procedure has *soundness error* ε if, for every (possibly adversarial) garbler, a setup that is *not* correct (Theorem 2.4) is accepted with probability at most ε . Equivalently, an accepted setup is correct except with probability ε .

A verification procedure therefore certifies correctness only *indirectly*: it accepts or rejects a committed garbling without evaluating it on all inputs, and its guarantee is probabilistic, that an incorrect garbling is accepted with probability at most ε . This bound ε is the procedure’s *soundness error* (Definition 10.1), and certifying correctness reduces to making it negligible.

Correctness is thus what we want; yet only soundness is what we can aim to obtain, and it is the axis on which the three routes of §10.4 differ: cut-and-choose attains $\varepsilon \approx 2^{-40}$ statistically (tunable via n), a SNARK-of-setup attains a cryptographic (or computational) $\varepsilon \approx 2^{-100}$, and a TEE replaces ε with a trust assumption of hardware attestation. Throughout, *integrity* and *faithfulness* are informal synonyms for correctness; we reserve *soundness* for this catch-probability guarantee.

10.1 SNARK or STARK of Correct Setup

This is one of the two methods proposed in the BitVM3 paper. It is a hypothetical cryptographic alternative to the more feasible statistical guarantee of soundness explored in section §10.2.²⁵

²⁴BitVM3 paper, §4.2, <https://bitvm.org/bitvm3.pdf> (ePrint 2026/933). The deployed cut-and-choose line is developed in Glock (ePrint 2025/1485) and Mosaic (ePrint 2026/812).

²⁵Alternative here in regards to the scope of statistical usage rather than mutually exclusive paradigms, give SNARK soundness is itself partly statistical. Groth16’s QAP divisibility check is verified at a secret random point, sound by the Schwartz–Zippel lemma with error at most d/q for polynomial degree d and field size $q \approx 2^{255}$; STARKs likewise draw random challenges for FRI’s low-degree test. The practical gap is the *size* of the error

The operator garbles a *single* instance and produces a SNARK (or STARK) attesting that the setup was performed correctly: that GC is a faithful garbling of the agreed circuit under the committed labels, and that pk_{GS} commits to those labels. One proof then certifies the execution of the entire garbling computation that produced GC . Formally, the setup SNARK proves membership in the relation

$$\mathcal{R}_{\text{setup}} = \left\{ \left(\underbrace{GC, \{C_i\}_i, h, f}_{\text{statement}}; \underbrace{r}_{\text{witness}} \right) : (GC, e, d) = \text{Gb}(1^\kappa, f; r) \wedge C_i = \text{commit}(e_i) \forall i \wedge h = H(L_{\text{out}}^{\text{False}}) \right\},$$

where $L_{\text{out}}^{\text{False}}$ is the false output label of the decoding information d returned by the same Gb call, so the published Disprove hashlock h (§11.6) is certified to open exactly for the genuine false-output label; without this conjunct a faithful GC could be proven while an unrelated h makes Disprove unsatisfiable (Remark 13.3, step 3). Here $f = \text{Circuit}(\text{SNARK.Vrfy})$ is the public verifier circuit (§12), r is the garbling randomness that seeded the labels (and exist within the private part of witness from which the operator computes a proof that the SNARK-of-setup verifier accepts, without r itself ever being revealed), and $\{C_i\}$ are the published label commitments (to which pk_{GS} is also bound). A valid proof certifies that GC really decrypts to f under exactly those committed labels.

Concretely, this is a *different* SNARK from the one being garbled (§12): here Groth16 proves the garbling correct, rather than being the circuit that is garbled.

Remark 10.1 (Why this is out of reach today). Note what the witness of $\mathcal{R}_{\text{setup}}$ is: the *entire garbling computation*. The prover must arithmetize Gb itself and, inside that arithmetization, re-execute one cipher call per garbled row and hash the full output to match the commitments. The BitVM Alliance verifier circuit has 2.7×10^9 non-free gates and its tables total ≈ 41.2 GB; garbling it *natively* takes 353.7 s (16-core Ryzen 7 7840U; BitVM3 §8.2). Arithmetizing Gb turns each of those rows into $\sim 10^3$ – 10^4 constraints (arithmetization-dependent; lookup-based systems sit at the low end), a setup circuit of $\sim 10^{12}$ – 10^{13} constraints; at the $\sim 10^4$ – 10^5 constraints per second a large prover sustains that is months to years, and the 10^{12} -plus-element witness is terabytes past a single prover’s memory. The obstruction is feasibility, not speed. Below, the Cut-and-choose paradigm replaces all this with native re-garbling plus a byte comparison, at the price of 181 instances, 288.4 GB ($= 7 \times 41.2$) kept-set storage, and a 3 h 24 m parallelized setup²⁶. That asymmetry, not preference, is why the deployed line (GSV, Glock, Mosaic) chose cut-and-choose.

10.2 Multiple Garblings and Statistical Soundness

Rather than prove one enormous circuit correct, this approach garbles n copies of it and certifies them *statistically*: a random subset is opened and re-derived (cut-and-choose, §10.2.1), and the confidence gained transfers to the copies left closed. The price is that the surviving copies’ labels must still reach the chain; naively that is one reveal per survivor, so a second mechanism (verifiable secret sharing, §10.2.2) collapses all of them into a single on-chain value. The two subsections are the two halves of one pipeline: *verify* the garblings, then *reveal* the survivors’ labels cheaply.

10.2.1 Cut-and-Choose

The operator garbles n independent instances of the same circuit, each from a fresh seed, gates, wiring and topology remain identical, but every label pair for each wire (inputs, outputs and

(roughly 2^{-40} for the paper’s cut-and-choose parameters versus far below 2^{-100} here), plus the computational assumptions (pairings and knowledge assumptions for Groth16, hash security for STARKs) that cut-and-choose does without.

²⁶BitVM3 §8.2 measured benchmark, <https://bitvm.org/bitvm3.pdf>.

intermediary) is sampled anew, and commits to all of them. A verifier then draws a uniformly random partition

$$[n] = C \sqcup E, \quad |E| = f, \quad |C| = n - f,$$

one of the $\binom{n}{f}$ equally likely choices: C is the *check set*, E the *evaluation set*.²⁷ For each instance in C the operator reveals its seed; a challenger deterministically re-garbles it and checks the result against the commitment (Remark 10.2). The instances in E stay closed and carry the real evaluation.

Remark 10.2 (Checking an opened instance). Write the garbler as $\mathbf{Gb}(1^\kappa, f; \text{seed})$: all internal randomness is drawn from the seed, so the same seed on the same public circuit reproduces the instance byte for byte, every label and every ciphertext. The garbling code is public. To check an opened instance $i \in C$, a challenger runs $\mathbf{Gb}(1^\kappa, f; \text{seed}_i)$ themselves and compares the output against instance i 's setup commitments (the ciphertext-hash digest foremost, alongside the label and table commitments).²⁸ A match proves the instance was honestly garbled; any mismatch aborts the setup. The check needs no semantic understanding of the circuit, not even a notion of what the true and false output labels are: it is reproduction and comparison, which is what makes this verification cheap relative to *proving* the same statement (Remark 10.1).

Remark 10.3 (Soundness of cut-and-choose). A malicious garbler's attack vector requires all the garbled instances in the check set to be honest and the bad instances selected by the verifier to be in the evaluation set. Challengers will evaluate *all* f instances in the garbled circuit after an operator's assert tx is broadcast as the one on-chain reveal extends to every kept instance's labels by the secret sharing of §10.2.2. Therefore a single *correct* garbled circuit in the evaluated set is sufficient and establishes the verdict honestly, and achieves setup soundness and, thus, the attack only succeeds if *every* evaluated instance is bad while the check set stays clean, which forces the bad set to coincide exactly with the evaluation set: probability at most $1/\binom{n}{f}$ (see Prop. 10.4 for more rigours arguement beyond the intuition of "possible choices of success from the all the ways a verifier can build the evaluation set").

The paper's concrete choice $(n, f) = (181, 7)$ gives $1/\binom{181}{7} \approx 2^{-40}$. (A looser per-instance estimate $(f/n)^k$ is sometimes quoted; Remark 10.5 derives the exact product and shows the heuristic errs in the attacker's favour.) The parameters n and f are derived from the single chosen input, the target $\varepsilon = 2^{-40}$: a conventional *statistical* security level (Remark 10.7). In the BitVM3 paper $f = 7$ is fixed by storage (each kept instance costs 41.2 GB forever, and one honest one suffices), and 181 is the smallest n with $\binom{n}{7} \geq 2^{40}$: $\binom{181}{7} \approx 2^{40.03}$ while $\binom{180}{7} \approx 2^{39.97}$.

A tempting intuition runs the other way: open as many instances as possible to "annul the risk". But checking has no protective power of its own; opened instances are burned and contribute nothing at claim time, and the only thing standing between a false claim and the bridge is that at least one *kept* instance is honest, an event of probability $1 - 1/\binom{n}{f}$. Decreasing the ratio of $f : n$ improves an attacker's probability of success: (181, 7) gives 2^{-40} , while opening 180 of 181 gives $1/181 \approx 2^{-7.5}$. The soundness error is exactly the probability of the event $E = B$ under the uniform draw (Prop. 10.4), so it is set by the number of equally likely partitions, $\binom{n}{f}$, and by nothing else. Opening the check set contributes no probability of its own; its role is structural: it makes any bad instance outside E detectable, which is what collapses the attacker's options to the single event $E = B$. Without checking, a bad instance anywhere would survive; with it, only a bad set equal to E does. Less risk is bought by garbling a larger

²⁷The letter f for the evaluation-set size follows the BitVM3 paper's notation, as in its concrete choice $(n, f) = (181, 7)$; it is unrelated to the function f being garbled in the garbling sections of this document. Context disambiguates.

²⁸Concretely, in GSV (the BitVM Alliance's **garbled-snark-verifier** reference implementation, §14.6): an instance's 41.2 GB of tables are committed as one digest, an accumulating hash computed over the ciphertext stream as it is generated, and never stored; checking re-streams the regenerated ciphertexts through the same accumulating hash and compares digests. Protocol steps in `docs/gsv-vsss.md` of that repository, <https://github.com/BitVM/garbled-snark-verifier>.

n or keeping more instances, never by opening more (or less) of a fixed n past the optimum of $n/2$, midway. As protocol requirement include the storage of this data, it is kept to a minimum to meet the security requirement (40 bits as per Rmk. 10.7).

Proposition 10.4 (Cut-and-choose soundness, formally). Let $B \subseteq [n]$ be the set of corrupted instances, fixed at commitment time, and let the evaluation set E be drawn uniformly from the f -subsets of $[n]$, independently of B . The attack succeeds iff $E = B$, hence

$$\Pr[\text{undetected false claim}] = \Pr[E = B] = \begin{cases} 1/\binom{n}{f} & \text{if } |B| = f, \\ 0 & \text{otherwise.} \end{cases}$$

Proof. If $B \not\subseteq E$, then there exists an index $i \in B$ with $i \in C = [n] \setminus E$. As all instances in the check set C are re-garbled from its own revealed seed and compared against its commitment (Remark 10.2), instance i 's mismatch is detected and the setup aborts. If $E \not\subseteq B$, then there exists an index $j \in E \setminus B$. The on-chain reveal extends to instance j 's input labels by the secret sharing of §10.2.2, so any challenger computes

$$\text{De}(d_j, \text{Ev}(GC_j, \text{En}(e_j, \pi))) = \text{SNARK.Vrfy}(\pi) = \text{False}$$

by correctness (Theorem 2.4), recovering instance j 's false-output label, and the Disprove witness.

Therefore, given success requires $E \subseteq B$ and $B \subseteq E$, then $E = B$. Since E is uniform over the $\binom{n}{f}$ subset of size f , and B one such a subset when $|B| = f$, $\Pr[E = B] = 1/\binom{n}{f}$; when $|B| \neq f$ the event $E = B$ is impossible and the probability is 0.

The independence of B from E is exactly the commit-then-challenge ordering: the commitments bind B before the partition randomness is drawn: without it the bound is vacuous, since a malicious garbler who knows in advance which instances will be selected into E corrupts exactly those instances and succeeds with probability 1. $\square$

Remark 10.5 (The $(f/n)^k$ heuristic, and its issue). A rough estimate sometimes used for the above Proposition 10.4 treats each of k malformed garbled circuits as independently “hiding” in the evaluation set with probability f/n , giving $(f/n)^k$. By Prop. 10.4, if $k \neq f$ the probability of an undected false claim succeeds with probability 0.

The exact probability that all f bad instances land in E is the without-replacement product

$$\frac{f}{n} \cdot \frac{f-1}{n-1} \cdots \frac{1}{n-f+1} = \frac{f!(n-f)!}{n!} = \frac{1}{\binom{n}{f}},$$

read as placing the bad instances one at a time: the first must land in one of f evaluation slots among n positions, and each success *consumes* a slot, until the last factor is $1/(n-f+1)$, returning the originally derived result.

Were this heuristic instead samples with replacement, every factor stays f/n , i.e., as if all f slots were fresh each time, and hence the product is $(7/181)^7 \approx 2^{-32.9}$ against the true $(n, f) = (181, 7) \approx 2^{-40.03}$. Resultingly, the heuristic error overstates the success probability and is thus the attacker's favour.

Remark 10.6 (More survivors, harder lottery: monotonicity in f). The claim decreasing the ratio $f : n$ improves the attackers success rate in Rmk. 10.3 can be made exact. Writing $P_f = 1/\binom{n}{f}$ for the success probability at survivor count f , first we compare the two splits directly, expanding each as its without-replacement product,

$$P_1 = \frac{1}{181}, \quad P_7 = \frac{7}{181} \cdot \frac{6}{180} \cdot \frac{5}{179} \cdot \frac{4}{178} \cdot \frac{3}{177} \cdot \frac{2}{176} \cdot \frac{1}{175},$$

$$\frac{P_7}{P_1} = \frac{6}{180} \cdot \frac{5}{179} \cdot \frac{4}{178} \cdot \frac{3}{177} \cdot \frac{2}{176} \cdot \frac{1}{175} \approx 2^{-32.5},$$

increasing the evaluation set from 1 to 7 multiplies the attacker’s odds by $\approx 2^{-32.5}$, which grows per extra malformed garbled circuit which must land in a shrinking evaluation set.

We can generalize this comparison to every pair of splits at once:

$$\frac{P_{f+k}}{P_f} = \frac{\binom{n}{f}}{\binom{n}{f+k}} = \prod_{i=1}^k \frac{f+i}{n-f-i+1},$$

noting that terms in the product increase in i . Thus every term in the product lies below 1 iff $f+k < n-f-k+1$, i.e. $f+k \leq \frac{n-1}{2}$; and when analysing an incremental increase of the evaluation set by $k=1$, we obtain $f+1 < n-f$, i.e. $f < \frac{n-1}{2}$.

The case $f=1$, $k=6$ reproduces the result above. So P_f is strictly decreasing in f until $f=n/2$, where the binomial coefficient peaks. For fixed f , P_f also falls as n grows. The theoretical optimum $f \approx n/2$ (here 90) is nonetheless never deployed: each survivor costs 41.2 GB of permanent storage, so the concrete (181, 7) trades security bits against storage, see the “derived, not chosen” arithmetic of Remark 10.3.

Remark 10.7 (Why 2^{-40} : statistical vs. computational security). The target is a *statistical* soundness level: it bounds a single unrepeatable random event and therefore holds even against computationally unbounded adversaries, whereas a *computational* parameter such as 2^{-128} bounds PPT adversaries (Remark 2.6) limited only by their number of offline computations ($\sim 2^{128}$ hash evaluations), attempts that are repeatable, unobservable, and parallelizable. Forty bits is the benchmark at which the cut-and-choose literature compares protocols,²⁹ and it suffices here because each cheating attempt is one *Bernoulli trial*: a single experiment with two outcomes, success with probability p and failure with $1-p$, where success is the event $E=B$ of Proposition 10.4 and $p=1/\binom{n}{f}$. The trial is online (the partition randomness is drawn fresh, after commitment), costs a full n -instance setup, and is publicly attributable on failure, so attempts cannot be repeated offline or concealed. Even k setups succeed at least once with probability $1-(1-p)^k \approx kp$, and the *profit of corruption* $p \cdot V$, for V the bridged value a successful claim would steal, loses to the *cost of corruption* for any plausible V . The per-attempt cost is purely computational, garbling and committing all n instances, and a protocol can raise it further by escrow: an operator bond forfeited on any detected attempt. At $p=2^{-40}$, even a nominal per-attempt cost puts the break-even V in the trillions.

Because challengers are permissionless, the check/evaluation split cannot be chosen by any single party; a colluding operator would simply steer its bad instances into E . The partition must instead come from a public randomness beacon or a coin-tossing protocol among a committee of verifiers.

Remark 10.8 (Per-instance artefacts; one slot key serves all). Each evaluation instance carries its own artefacts: its own label commitments, hashlock $H(L_i^*)$, pre-signature set, and presigned Assert/Disprove/Timeout templates, so the transaction graph grows linearly in f (the on-chain label reveal itself is collapsed to one by the secret sharing of §10.2.2). The slot key (Remark 9.8), however, can be one key across all instances: instance i ’s Assert commits instance i ’s hashlock, so each instance’s Assert is a different transaction with a different sighash, and a pre-signature completion for one instance fails CHECKSIGVERIFY in any other. The same message-binding argument that pins a signature to its slot (Remark 13.7) pins it to its instance.

²⁹Y. Lindell, *Fast Cut-and-Choose Based Protocols for Malicious and Covert Adversaries*, CRYPTO 2013. ePrint 2013/079, <https://eprint.iacr.org/2013/079>: prior works send 125–128 circuits for cheating probability 2^{-40} ; Lindell achieves 2^{-s} with s circuits, so 40 suffice. His mechanism (“cheating recovery”) engineers the property BitVM3’s setting has natively: cheating must corrupt *every* evaluated circuit, not a majority. Pairing $\kappa=128$ computational with 40 statistical bits is the standard MPC convention (Evans, Kolesnikov, Rosulek, *A Pragmatic Introduction to Secure MPC*, Ch. 2).

10.2.2 Verifiable Secret Sharing of Labels

Cut-and-choose leaves f evaluated instances, each with its own input labels; revealing all f label sets on-chain would cost f separate reveals. Verifiable secret sharing collapses this to one. The primitive is Shamir secret sharing made checkable; the BitVM3 paper writes *VSSS* for it, expanded either as “verifiable Shamir secret sharing” or generically as “verifiable secret sharing scheme.”

Definition 10.2 (Shamir secret sharing). To share a secret s so that any $t+1$ shares reconstruct it while t or fewer reveal nothing, the dealer samples a degree- t polynomial over the scalar field $\mathbb{Z}_q$,

$$P(z) = s + a_1z + \cdots + a_tz^t, \quad a_1, \dots, a_t \xleftarrow{\$} \mathbb{Z}_q,$$

so that $P(0) = s$, and gives party i the *share* $P(i)$. Any $t+1$ shares determine P by Lagrange interpolation, hence recover $s = P(0)$; any t shares leave s information-theoretically uniform.

Remark 10.9 (Verifiability, after Feldman). On its own, secret sharing trusts the dealer to hand out genuine shares. Feldman’s variant makes them checkable: the dealer also publishes commitments to the coefficients, $C_k = [a_k]G$ for $k = 0, \dots, t$ (so $C_0 = [s]G$). A holder verifies its share against the committed polynomial by checking $[P(i)]G = \sum_{k=0}^t [i^k]C_k$. A share that passes is a genuine point on the committed P , so a cheating dealer cannot slip in inconsistent shares.³⁰ The check is not limited to one’s own share: the right-hand side is publicly computable for *every* index i , so anyone can verify any share that is later revealed (an opened check-set share, the on-chain reveal), while an unrevealed share is checkable only by its holder, since the check consumes the share value itself.

Certification is route-agnostic about the reveal mechanism (§9.7); in the statistical route the two couple, because one reveal must serve f kept instances. The claim reveals wide labels for only one instance, yet challengers must evaluate all f kept instances. VSSS closes this gap: the labels are arranged so that the one on-chain reveal is the final missing share of a committed polynomial system, from which challengers compute every other kept instance’s labels themselves. The construction below is the implemented one (GSV, as cited in Remark 10.2); the Shamir polynomial is instantiated at degree $t = n - f$, tying the reconstruction threshold to the check-set size.

Definition 10.3 (Wide-label sharing). For each digit d and value v of §9.7, the garbler samples a polynomial of degree $n - f$ over $\mathbb{Z}_q$,

$$P_{d,v}(z) = a_0 + a_1z + \cdots + a_{n-f}z^{n-f}, \quad a_k \xleftarrow{\$} \mathbb{Z}_q,$$

defines the instance keys $W_{d,v}^{(j)} := P_{d,v}(j)$ for every $j \in [n]$, and publishes Feldman commitments $C_k = [a_k]G$ to the coefficients. Instance j ’s lookup row (Definition 9.3) is keyed by $W_{d,v}^{(j)}$; each instance carries its own rows, since each was garbled with fresh labels. The $128 \times 256 = 32,768$ polynomials and their commitments are the entire sharing state; the bit labels themselves appear only inside rows. Of the three certification routes, this is the only one where the published tuple shrinks: the statements are derivable from the coefficient commitments, so the quadruple of Definition 9.3 becomes the triple $(R, \tilde{s}, \text{row})$, whereas the single-instance routes (§10.1, §10.3) publish all four elements.

Definition 10.4 (Shares and their three acquisition modes). The *share for instance j* is the point $(j, P_{d,v}(j))$. A challenger comes to hold shares three ways:

³⁰P. Feldman, *A Practical Scheme for Non-interactive Verifiable Secret Sharing*, FOCS 1987. Overview: https://en.wikipedia.org/wiki/Verifiable_secret_sharing.

1. *Opened* (check set): for $i \in C$ the garbler sends $P_{d,v}(i)$ directly; it is accepted only if it lies on the committed polynomial,

$$[P_{d,v}(i)]G = \sum_{k=0}^{n-f} i^k C_k.$$

2. *Extracted* (asserted instance j^*): the completed adaptor signature releases $W_{d,v}^{(j^*)} = \mathbf{Ext}(\tilde{\sigma}, \sigma) = P_{d,v}(j^*)$ (Definition 9.1). The adaptor statement is itself derivable from the coefficient commitments, $T_{d,v} = [P_{d,v}(j^*)]G = \sum_k [(j^*)^k] C_k$, which is what binds each pre-signature to the committed polynomial.
3. *Interpolated* (all remaining kept instances): holding $n - f + 1$ points $\{(x_i, y_i)\}$, Lagrange interpolation

$$P_{d,v}(z) = \sum_i y_i \prod_{m \neq i} \frac{z - x_m}{x_i - x_m}$$

yields the polynomial, evaluated at the remaining indices of E and verified against the same commitment equation before use.

Remark 10.10 (Why degree $n - f$). Opening the check set necessarily publishes $n - f$ shares of every polynomial. Hiding requires that these do not determine the kept instances' labels, forcing degree $\geq n - f$; the single on-chain reveal must then complete reconstruction, forcing degree $\leq n - f$. The degree is exactly the check-set size: the reveal is the tipping point by construction.

Example 10.11 (One digit, five instances). Take $n = 5$, $f = 2$, $C = \{1, 2, 3\}$, $E = \{4, 5\}$, and fix one (d, v) , so $\deg P_{d,v} = 3$ and four points determine it. Opening C gives $P(1), P(2), P(3)$: one short, and instances 4, 5's wide labels stay information-theoretically hidden. The Assert on instance 4 releases $W^{(4)} = s - \tilde{s} = P(4)$; the challenger checks $[P(4)]G = \sum_k 4^k C_k$ and now holds four points. The interpolation is explicit: evaluating the Lagrange basis of $\{1, 2, 3, 4\}$ at $z = 5$,

$$\ell_1(5) = \frac{(5-2)(5-3)(5-4)}{(1-2)(1-3)(1-4)} = -1, \quad \ell_2(5) = 4, \quad \ell_3(5) = -6, \quad \ell_4(5) = 4,$$

so

$$P(5) = -P(1) + 4P(2) - 6P(3) + 4P(4) \pmod{q} :$$

a fixed integer combination of known shares, computed without ever writing down P 's coefficients. Concretely, $W^{(4)} = P(4)$ and $W^{(5)} = P(5)$ are the two kept instances' keys: deriving $H(W^{(j)})$ and decrypting $\text{row}^{(j)}$ (Definition 9.3, item (iii)) yields each instance's own 8 bit-labels for this (d, v) . The on-chain label is itself a share; no distinguished index is involved.

Remark 10.12 ((WIP) The coding-theoretic view: Shamir as Reed–Solomon). The sharing of Definition 10.3 is a codeword of an *error-correcting code*. Encoding a polynomial of degree at most $n - f$ by its evaluations at n fixed points is the Reed–Solomon code of length n and dimension $k = n - f + 1$; the share vector $(P_{d,v}(1), \dots, P_{d,v}(n))$ is one codeword, and Shamir sharing *is* this encoding (McEliece and Sarwate³¹). The code is maximum-distance-separable: any k symbols determine the codeword, since the $k \times k$ Vandermonde system $P(x_i) = \sum_{j=0}^{k-1} a_j x_i^j$ has determinant $\prod_{i < j} (x_j - x_i) \neq 0$ for distinct evaluation points; and any $k - 1$ symbols leave every unseen symbol uniform, since for each candidate secret s exactly one polynomial passes through the $k - 1$ points and $(0, s)$. The two halves of Remark 10.10 are these two facts read in protocol order: the opened check set is exactly $k - 1$ symbols (hiding), and the on-chain reveal is symbol k (reconstruction), as Example 10.11 works concretely. The kept

³¹R. J. McEliece, D. V. Sarwate, *On Sharing Secrets and Reed–Solomon Codes*, Communications of the ACM 24(9), 1981. <https://dl.acm.org/doi/10.1145/358746.358762>

instances' unrevealed shares are *erasures*, missing symbols at known positions, and Lagrange interpolation is the erasure decoder; their count, $f - 1$, is exactly the code's erasure-correction limit $n - k$. Corrupted shares would be *errors*, wrong symbols at unknown positions. These are correctable by the Berlekamp–Welch method: find an error-locator polynomial E of degree e and a polynomial Q of degree at most $n - f + e$ satisfying $Q(x_i) = y_i E(x_i)$ at every received point, whence $P = Q/E$; the linear system is determined only while $n \geq k + 2e$, bounding correction at $e \leq \lfloor (f - 1)/2 \rfloor$ (three at $(n, f) = (181, 7)$). Against a dishonest dealer even that fails: error decoding presumes the symbols came from *some* codeword, which is precisely what a dealer distributing inconsistent shares denies. The Feldman commitments close this gap by *detection* rather than correction: every share is verifiable against the committed polynomial (Remark 10.9), so a bad share is identified and discarded, and the challenger faces only erasures, never errors.

The chain in full: the Assert reveals the asserted instance's wide labels; each is one share, and interpolation as above converts them into the corresponding wide labels of *every* kept instance; each instance's wide labels open that instance's own lookup rows, yielding its own bit-labels for the *same* committed π ; challengers then evaluate all f kept circuits, each with its own genuine keys. One on-chain reveal, f evaluations.

Remark 10.13 (Protocol flow). Commit (coefficient, share, row, and circuit commitments) $\rightarrow$ partition $[n]$ and pick the assert instance $j^* \in E$ (in GSV's two-party protocol the evaluator chooses, sound because the chooser is the party the check protects; the permissionless bridge instead draws the partition from the public randomness of §10.2.1) $\rightarrow$ the garbler opens C 's seeds and shares, and everything is rechecked against the commitments $\rightarrow$ the Assert releases j^* 's wide labels on-chain $\rightarrow$ extract, interpolate, decrypt rows, evaluate every kept instance; any invalid-output label opens the Disprove hashlock.

10.3 Trusted Execution Environment

A third route replaces cryptographic or statistical certification with hardware. A single instance is garbled *inside* a trusted execution environment (an enclave, e.g. an AWS Nitro instance), and the enclave's remote *attestation*, a signed measurement of the exact code that ran, stands in for cut-and-choose or a setup proof: verifiers accept the garbling as honest because they trust the measured code and the hardware root of trust that signed the measurement, not because they re-derived or proved anything. This removes both the n -fold storage of the statistical branch and the prover cost of the cryptographic one, at the price of a hardware trust assumption (enclave integrity, no side channels, an uncompromised manufacturer key). The Lombard BSA redesign (§14.7) is a concrete instance, pairing enclave attestation with ECIES-sealed labels; it is a genuinely different trust model, not an instantiation of BitVM3-core.

10.4 Comparison of the Three Routes

A garbled circuit can be *checked* or *used*, but not both: verifying it means opening it, which exposes both labels of every wire and burns it for any binding evaluation. The three routes escape this differently: the cryptographic route *proves* correctness without opening, the statistical route opens *copies* it can afford to destroy, and the hardware route has an enclave *attest* that the single unopened instance was honestly produced. All three routes exist to protect *depositors and challengers* from a malicious *garbler*; the garbler itself needs none of them, since it knows its own garbling is honest.

	cryptographic (SNARK-of-setup)	statistical (cut-and-choose)	hardware (TEE)
garbled circuits	1	n copies (e.g. 181)	1
off-chain cost	one garbling + one proof	$n \times$ garbling and storage	one garbling
certified by	a proof over the garbling relation	opening a random subset	enclave attestation
soundness error ε	computational; DL security of the proving curve: $\approx 2^{-128}$ (BLS12-381) or $\approx 2^{-100}$ (BN254)	statistical, $1/\binom{n}{f} \approx 2^{-40}$	none: conditional trust
trust assumptions	the SNARK's (setup, hardness)	the garbling scheme only	enclave integrity, manufacturer key
status	infeasible today (Remark 10.1)	deployed (GSV, Glock, Mosaic)	alternative model (§14.7)

The three soundness errors are not of one kind, and each is derived differently. Cut-and-choose's is *statistical*: it is exactly $1/\binom{n}{f}$, the chance the random check set misses every planted bad instance, which for $(n, f) = (181, 7)$ is $\approx 2^{-40}$ (Remark 10.3); it is tunable, shrinking as n grows. The SNARK's is *computational*: Groth16 knowledge-soundness reduces to a discrete-log-type hardness assumption in the pairing groups, so ε is the cost of the best attack on the curve, $\approx 2^{-128}$ for BLS12-381 ($\approx 2^{-100}$ for BN254). It is *not* the Schwartz–Zippel error of the QAP divisibility check, whose statistical component ($\approx d/q \approx 2^{-225}$ for degree $d \approx 2^{30}$ and field size $q \approx 2^{255}$) is far smaller and not the binding term. A TEE gives no probabilistic ε : cheating probability is 0 conditional on honest hardware and an uncompromised attestation key, a trust assumption rather than a bound. The ordering inverts the deployment choice: the statistical route is the *weakest* (2^{-40}) yet is what ships, because a cheap 2^{-40} check beats a 2^{-128} proof that is infeasible to produce (Remark 10.1).

11 Bitcoin Transactions and the BitVM3 Transaction Graph

Everything so far has been off-chain cryptography. This section covers the on-chain half: the Bitcoin transactions that lock the operator's bond and either release it on an unchallenged claim or slash it on a disproof. It is also the theory backing the Rust implementation of the Assert/Disprove graph (`rust-bitcoin`).

11.1 Transaction Anatomy

A Bitcoin transaction consumes *inputs*, each referencing a previous unspent transaction output (UTXO) by its txid and output index (*vout*), and creates new *outputs*, each locking an amount under a script.³² Since SegWit,³³ the unlocking data (signatures, script preimages) lives in a separate *witness* field rather than in the input's `scriptSig`; this removes transaction malleability and discounts witness bytes, which is what makes BitVM3's large witnesses (a revealed garbled label, a per-digit adaptor signature) affordable. A transaction is named by its *txid*, a

³²A Bitcoin *address* is just a locking script: it encodes, in a standardised human-transferable form, the variable data of one of several accepted script templates (a public-key hash for P2PKH/P2WPKH, a script hash for P2SH, or a witness program for P2WSH/P2TR), which a wallet expands back into the actual `scriptPubKey` opcodes. The encoding (Base58Check for legacy, Bech32/Bech32m for SegWit and Taproot) folds in a version prefix and a checksum, so a mistyped address fails locally rather than sending coins astray. The address itself never appears on-chain; only the expanded script does, and it is that script the network's consensus rules enforce.

³³Segregated Witness, the 2017 Bitcoin soft-fork upgrade that moved the unlocking data out of the input into a separate witness field committed outside the txid, fixing transaction malleability and discounting witness bytes to 25% the block-weight limit.

double-SHA-256 over its non-witness serialization. The txid is handled internally in little-endian byte order but displayed by block explorers big-endian; the two are hex-byte reversals of one another.³⁴

11.2 Taproot

A Taproot output (BIP341) locks coins to a single tweaked public key Q , built from an internal key P and the Merkle root m of a *taptree* of alternative spending scripts, where each script is a *leaf* and m the single hash they all compress into (§B), via the tweak

$$Q = P + t \cdot G \text{ where tweak } t = H(P \parallel m).$$

The scalar t is *public*, recomputable by anyone from P and m and commits both the public key and the entire script tree into one point Q , so a single output *multiplexes* many spending conditions yet reveals only the branch taken, i.e. the many scripts sit as leaves under the one root m , not many tweaks t satisfying the relation; for fixed P the root fixes a single t and hence a single Q (see Rmk. 11.2).

A UTXO locked to a Taproot public key Q can be spent in one of two ways (and not with two separate keys). Firstly, the *key path* produces one BIP340 Schnorr signature verifying under Q : this needs the discrete log $q = p + t$ (the map $k \mapsto [k]G$ is a group homomorphism, so $[p + t]G = [p]G + [t]G = P + tG = Q$), and since t is public it needs only the internal secret p . It reveals nothing about the scripts, on-chain indistinguishable from an ordinary payment. Secondly, the *script path* signs nothing under Q : it reveals one *tapleaf* (a script and its leaf version) together with a *control block* (the internal key P and a Merkle authentication path to get from a leaf to the root), then the network recomputes m from the leaf and path and checks the tweak equation to confirm the leaf was committed, i.e., checking the elliptic curve point addition of these two points ($P + tG$) is Q , and not a discrete-log proof of Q , after which all validating nodes then execute the leaf, whose own condition (a signature under some *other* key, a hashlock, a timelock, or in general any predicate expressible in Bitcoin Script opcodes) is what actually authorises the spend. BitVM3 uses the script path for the challenge and timeout branches, and the cheaper key path for cooperative settlement.

Remark 11.1 (The control block). The control block is a Merkle *proof of membership* for one leaf: the sibling hashes needed to climb from that leaf to the root, plus the key material to confirm that root is the one Q commits to. It is *per leaf*, not shared and not the root itself. BIP341 fixes its length at $33 + 32m$ bytes, where m is the leaf’s depth: the first byte holds the leaf version and the parity of Q ’s Y -coordinate, the next 32 bytes are the internal key P , and the remaining $32m$ bytes are the Merkle authentication path: one sibling hash per level from the leaf up to the root. The verifier hashes the revealed leaf, folds in the sibling hashes to recompute the root (children are sorted before each **TapBranch** hash, so no left/right flags are needed), and accepts iff $Q = P + \text{hash}(P \parallel \text{root})G$. Because the path differs per leaf, each spendable leaf carries its own control block. A taptree may commit up to 2^{128} leaves (depth 128), a vast space of alternative spending scripts locked under one Q at setup; a spend still opens exactly *one* of them, and the control-block size tracks the tree *depth* ($\log_2$ of the leaf count), not the count itself. BitVM3’s Assert output is deliberately minimal, a two-leaf tree ($m = 1$): each control block is 65 bytes with its single path element the *other* leaf’s hash, and a spend takes one of just two branches (Disprove or Timeout). The large combinatorial space in BitVM3 lives not in this output taptree but in the Assert *input*, whose witness commits the garbled proof labels across many per-digit signatures (§13.2).

³⁴Little-endian is the native word order of the x86 hardware Satoshi built on, so a 256-bit hash could be read straight off as an integer for the proof-of-work target comparison with no byte-swap. The big-endian display was layered on afterwards only for human readability (largest byte leftmost); see <https://learnmeabitcoin.com/technical/general/byte-order/>.

Remark 11.2 (Why the tweak binds). At a glance, one can misread $Q = P + H(P \parallel \text{root}) \cdot G$ as if many roots (hence many script trees) could satisfy it for one fixed Q , so the output commits to nothing in particular. The opposite holds: once a spender reveals (P, root) , the tweak $t = H(P \parallel \text{root})$ is *deterministic* and $Q = P + tG$ is a single point. As group elements this is rigid: $tG = Q - P$ is the *unique* curve point solving the equation, and since G generates a prime-order group $t \mapsto tG$ is a bijection, so exactly one scalar t matches it. For fixed P the map $\text{root} \mapsto Q$ is therefore a *function*, so distinct roots give distinct Q (bar a negligible tweak hash collision). The many taptrees that exist in the abstract are not many trees openable against *one* Q ; only the committed one opens.

Binding closes the two ways an attacker might spend Q through a script s that was never committed. (i) *Fix P , hunt a root.* Find a $\text{root}' \neq \text{root}$ whose tree contains s yet still yields Q ; this needs $H(P \parallel \text{root}') = H(P \parallel \text{root})$, a tap-hash collision (or second preimage). (ii) *Vary the key too.* Set $P' = Q - t'G$ for a $t' = H(P' \parallel \text{root}')$ of one's choosing: this satisfies the equation, but now no discrete log of P' is known, so the *key path* is dead and the *script path* still demands a committed leaf, back to (i). Either route is a hash collision, not a spend. (The Merkle machinery the root relies on is in §B; removing route (ii) outright via a NUMS key is Remark 11.3.)

Remark 11.3 (NUMS internal keys). A *nothing-up-my-sleeve* (NUMS) internal key turns route (ii) of Remark 11.2 into a feature. Picking P with no known discrete log (e.g. a hash-to-curve point) makes the key path permanently unspendable, forcing every spend through a committed leaf. This does not loosen the commitment: it *removes* the key-path escape, leaving the collision barrier as the only attack surface, so NUMS makes Q strictly harder to spend illegitimately, never easier; it is not that “millions of preimages open Q .” Variants of BitVM3 puts a NUMS key on the Assert output so its two leaves (Disprove, Timeout) are the *only* ways it can move.

11.3 Sighash: What a Signature Commits To

A Bitcoin spend has three layers. The *transaction data* is the set of input UTXOs to be destroyed and output UTXOs to be created; it contains no authorization of its own. The *sighash* is a commitment to that data: a tagged SHA-256 digest (tag `TapSighash`, BIP341) over the fields the one-byte sighash flag selects. The *signature* is the authorization: it signs the sighash, and it verifies only under the public key that the UTXO being spent named in its locking script, so validity of the signature under the demanded key is what authorizes exactly that destruction and creation. The flag is the policy, the sighash is the bytes actually signed. The default, `SIGHASH_ALL`, covers every input and output, so the signature authorises exactly one transaction paying exactly those outputs; change any output and the sighash changes, invalidating the signature.

For a Taproot key-path spend the digest commits, in order, to: the sighash flag; the transaction version and locktime; the inputs being spent, i.e. their outpoints (which coins), amounts, scripts, and sequences; and the outputs (who is paid). Other flags relax this: `SIGHASH_ANYONECANPAY` drops the inputs, and `SIGHASH_SINGLE`/`SIGHASH_NONE` drop the outputs. BitVM3 designs generally sign with `SIGHASH_ALL`, which is what “binds the signature to all inputs and outputs” means in Remark 11.5.³⁵

Remark 11.4 (Sighash flags as a fee lever). Bitcoin circuit-style protocols exploit these flags deliberately: the signed *outputs* are kept a fixed set (sometimes plus one output returning fees) so the covenant emulation holds, while the *input* side may be left open, e.g. via `ANYONECANPAY`,

³⁵Field ordering follows BIP341 (§11.2). In Bitcoin Core it is `SignatureHashSchnorr` in `src/script/interpreter.cpp`: the function opens at line 1493, and lines 1521–1552 serialise the message in this order: epoch byte, `hash_type`, `nVersion/nLockTime`, the four input hashes (skipped under `ANYONECANPAY`), `sha_outputs` (under `SIGHASH_ALL`), `spend_type`, and the input index. Accessible walkthrough of flag versus signed digest: learn me a bitcoin, <https://learnmeabitcoin.com/technical/keys/signature/#sighash>.

so that a fee-paying input can be attached at broadcast time. Pre-signed graphs thereby gain a dynamic fee mechanism without precommitting fees at setup for every state transition.

Two consequences of *what* the digest covers matter for the graph. The digest commits to each input's `nSequence`, so a pre-signed timeout transaction has its relative timelock frozen in: lowering it afterwards would change the sighash and invalidate the committee's signatures. Witness data, by contrast, is *not* covered by any sighash: this is precisely why the disprover can attach the preimage L^* to an already-pre-signed Disprove transaction (Remark 11.5) at dispute time without invalidating anything.

Remark 11.5 (Covenant emulation by pre-signing). A *covenant* would let an output constrain *where* it may be spent, restricting the spending transaction to a specific output or locking script. Bitcoin Script has no such opcode, so a script cannot bind the coins it locks to a designated next output. BitVM3 emulates one by pre-signing: at setup the N -of- N committee uses Partially Signed Bitcoin Transactions (PSBTs) to collaboratively sign every transaction of the dispute graph (Assert, Disprove, timeout, payout), each under `SIGHASH_ALL`, committing the graph's possible end states and binding each signature to that transaction's exact inputs and outputs. Two facts make the graph constructible in advance. Signing requires the funding outpoint fixed first, since each signature commits to its input outpoints; and because a txid covers only the non-witness serialization (§11.1), every txid in the graph is computable at setup, each child referencing its parent's txid before anything is broadcast, while witnesses attached at claim time (adaptor completions, revealed labels) alter no txid. Pre-SegWit malleability would have broken exactly this chain. Once at least one honest signer deletes its key share, no alternative transaction can ever be signed, so the pre-signed graph is the only way the bonded output can move. This is what pins the off-chain verdict to a fixed on-chain outcome.

11.4 Spending Conditions

Bitcoin Script defines around a hundred opcodes (many disabled or reserved), interpreted in `interpreter.cpp`; the tapleaf scripts in BitVM3 use only the handful below:

- **Hashlock** (`OP_SHA256` $\langle h \rangle$ `OP_EQUALVERIFY`): spendable only by revealing a preimage hashing to h .³⁶ This gates Disprove on the false-output label: $h = H(L^*)$, and the witness must supply L^* .
- **Relative timelock** ($\langle t \rangle$ `OP_CHECKSEQUENCEVERIFY`): the output cannot be spent until t blocks after its confirmation, giving challengers a window before the operator's timeout path opens.
- **Signature check** ($\langle pk \rangle$ `OP_CHECKSIGVERIFY`): a BIP340 Schnorr signature under pk ; e.g. the Assert timeout leaf pairs this with the relative timelock above, so only the operator holding pk_P can reclaim the bond, and only once the challenge window has elapsed.
- **Code separator** (`OP_CODESEPARATOR`): changes which part of the script the next signature's sighash covers. BitVM3 uses it to encode the SNARK proof, split into 128 digits of 8 bits, one 65-byte Schnorr adaptor signature per digit, with `OP_CODESEPARATOR` between successive `OP_CHECKSIGVERIFY` operations so a single public key can authenticate every digit.

Script execution is a stack machine. The interpreter holds five pieces of state between opcodes: a main stack, an alternate stack, the program counter, the conditional-execution flags of `OP_IF`, and the variable `m_codeseparator_pos`. All five are declared or initialized at the head of

³⁶As a *standalone* tapleaf this must end in `OP_EQUAL` (which leaves `true` on the stack), not bare `OP_EQUALVERIFY` (which consumes both operands and leaves the stack empty, failing the spend); `OP_EQUALVERIFY` is correct only when followed by further script that leaves a truthy result.

EvalScript, the interpreter’s entry point (Bitcoin Core v28.0, src/script/interpreter.cpp, L406–L433):

```
bool EvalScript(
    std::vector<std::vector<unsigned char>>& stack, // (1) main stack
    const CScript& script, // the opcodes being executed
    unsigned int flags, const BaseSignatureChecker& checker,
    SigVersion sigversion,
    ScriptExecutionData& execdata, // holds (5) m_codeseparator_pos
    ScriptError* error)
{
    CScript::const_iterator pc = script.begin(); // (2) program counter
    ConditionStack vfExec; // (3) OP_IF flags
    std::vector<valtype> altstack; // (4) alternate stack
    execdata.m_codeseparator_pos = 0xFFFFFFFFUL; // (5) init: none
        executed
    ...
    case OP_CODESEPARATOR:
        execdata.m_codeseparator_pos = opcode_pos; // the opcode’s
            entire effect (L1054)
    ...
}

// In a separate function, the sighash serializer SignatureHashSchnorr
// (L1555–1560), for tapscript spends:
ss << execdata.m_tapleaf_hash; // which leaf is executing
ss << key_version;
ss << execdata.m_codeseparator_pos; // where within it: committed
    into the digest
```

Witness items are pushed onto the stack first; opcodes then execute strictly left to right. The sentinel 0xFFFFFFFF (the suffix UL in the listing is C++ compiler syntax, unsigned long, not part of the value) is set *once*, at script start; only the first signature check ever reads it, and each executed separator overwrites the single variable with its own position. Exactly one opcode writes `m_codeseparator_pos` and only signature checks read it, serializing its current value into the sighash preimage. Three slots of the CheckGS leaf:

idx	opcode	m_codeseparator_pos	effect
0	$\langle pk_{slot} \rangle$	0xFFFFFFFF	push key
1	CHECKSIGVERIFY	0xFFFFFFFF	verify σ_1 against m_1
2	OP_CODESEPARATOR	→ 2	update the variable
3	$\langle pk_{slot} \rangle$	2	push key
4	CHECKSIGVERIFY	2	verify σ_2 against m_2
5	OP_CODESEPARATOR	→ 5	update

Each signature committed to a position at signing time, the position being a field of the message it signed; at verification each slot recomputes its message from the variable’s current value and compares. Agreement per slot, checked locally and independently, is what makes only the correctly ordered witness pass, and the 128-signature requirement itself is enforced by the leaf containing 128 CHECKSIGVERIFY opcodes, each aborting the script on failure. The position value is committed, never stored: it appears nowhere in the transaction bytes and exists only as a field of the hash input at verification time.

11.5 Signatures and Adaptors in the Witness

Two kinds of signature populate the witnesses. Plain BIP340 Schnorr signatures authorize the cooperative key-path settle and the timeout-leaf spend (though some BitVM variants disable the

cooperative key path entirely, forcing every spend through a committed script leaf; cf. the NUMS key of Remark 11.3). Schnorr *adaptor* signatures (§9) carry the proof encoding: the operator posts a pre-signature bound to the proof digits, and completing it on-chain simultaneously authorizes the Assert transaction and reveals the encoded labels to challengers, with no separate label-delivery step (worked one wire at a time in §13.2).

11.6 The Assert, Disprove, and Timeout Templates

The dispute reduces to three transaction templates (BitVM3 paper Tables 3–4).

- **Assert** spends the bonded output. Its input is gated by the on-chain check $\text{CheckGS}_{pk}(\sigma)$, so posting it publishes the encoded proof. Its output is a taptree with two leaves: a challenge leaf $\text{Hashlock}(H(L^*))$ and a timeout leaf $\text{CheckSig}_{pk_P} \wedge \text{RelTimeLock}(t)$.
- **Disprove** spends the Assert output through the challenge leaf. Its witness is the recovered false-output label L^* ; the script checks $H(L^*)$ against the committed hash and, on success, redirects the operator’s bond (the slashing).
- **Timeout** spends the Assert output through the timeout leaf once t blocks have elapsed with no valid Disprove, letting the operator reclaim the bond. This is the honest-operator path.

Remark 11.6 (CheckGS is a script gadget, not an opcode). The two checks in the Assert template are gated differently. CheckSig_{pk_P} in the timeout leaf is a single real opcode (OP_CHECKSIGVERIFY , §11.4) verifying one BIP340 signature under the operator’s key pk_P . $\text{CheckGS}_{pk}(\sigma)$ has no opcode of its own: it is shorthand for the per-digit gadget of §13.2, a chain of $\langle pk_{slot} \rangle$ OP_CHECKSIGVERIFY separated by OP_CODESEPARATOR , one slot per proof digit (pk_{slot} : a joint key no party can sign under after setup, not the operator’s own; §13.2). Consensus checks only those ordinary signatures under pk_{slot} ; the “GS” semantics, that passing releases $\text{En}(e, \pi)$, come from the adaptor pre-signatures each bound to a label statement $T_i^b = [L_i^b]G$ (Remark 13.6), not from any special validation rule. So both checks reduce to OP_CHECKSIGVERIFY under an operator key; CheckGS differs only in that its signatures are adaptor completions that leak the labels. Completing them requires the wire labels L_i^b themselves, the garbling secret $e = sk_{GS}$ that only the operator (the garbler) holds, so only the operator can produce a valid Assert witness, and posting it necessarily releases the active labels $\text{En}(e, \pi)$. Because pk_{slot} is fixed at setup, the whole Assert is bound to one operator’s instance, while the audited verifier circuit is reused across operators (Remark 12.3).

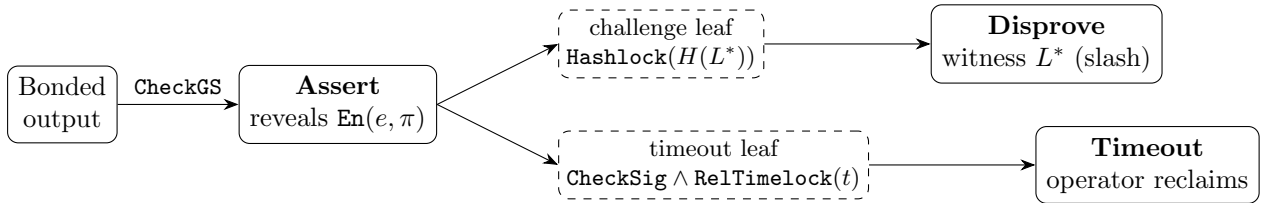


Figure 1: The Assert output carries two tapleaves; the verdict decides which is spendable. A false verdict lets a challenger open the hashlock (Disprove, slash); otherwise the timelock lets the operator reclaim the bond (Timeout).

12 The SNARK Being Garbled

The circuit the operator garbles is the *verifier* of the application proof, $\text{SNARK.Vrfy}(vk, \varphi, \pi)$, instantiated here with Groth16. This section fixes what that verifier consumes and, crucially,

how its *public inputs* bind a proof to one specific claim. It is distinct from the *setup* SNARK of §10.1: there the SNARK proves the garbling honest; here the SNARK is the thing *inside* the garbled circuit.

12.1 Components

A Groth16 instance is fixed by a *common reference string* $\text{CRS} = (pk, vk)$ from a trusted setup:

- the *proving key* pk lets the operator turn a witness w into a proof $\pi = (A, B, C) \in \mathbb{G}_1 \times \mathbb{G}_2 \times \mathbb{G}_1$ (the proof’s size is independent of the statement being proved: always exactly three group elements, regardless of circuit size).³⁷ pk is consumed entirely at proving time and never appears in verification.
- the *verifying key* vk is the circuit-specific subset of the CRS that, with π and the public inputs, drives the bilinear pairing check of elliptic curve group (ECG) elements. It is vk that is compiled into the garbled verifier: verification uses only vk , never pk , which is what lets one audited vk (Remark 12.2) check proofs from any operator holding the corresponding pk .

The CRS entries are group elements, commitments to the setup’s secret scalars (the “toxic waste”): usable for verification, but revealing nothing that would let a party forge a proof.

Remark 12.1 (Why the CRS entries are group elements). Encoding the setup scalars as points $[\tau]G, [\tau^2]G, [\tau^3]G, \dots$ (the powers of τ of §12.4)³⁸ rather than as the scalars themselves does two jobs at once. First, *hiding*: the scalars are the toxic waste, and publishing any of them would let a party forge proofs, so they are locked behind the discrete-log problem, one can compute with the point but not recover the scalar, which is what makes the CRS safe to publish. Second, *computability*: proving and verifying are linear combinations of CRS points followed by a bilinear pairing over them (§12.6), and the scalar-to-point homomorphism (Remark 9.1) lets the prover form $\pi = \sum_i [w_i] (\text{CRS point})$ without knowing the underlying scalars. A plain hash commitment could hide the scalars but not support this arithmetic; a group point is one-way *and* algebraically usable (the same distinction as Remark 9.5). The proof π is itself group elements built from the CRS, but that is a consequence of the encoding, not its reason.

Remark 12.2 (Why the vk is fixed at setup and audited once). The verifying key vk is *compact*: a handful of group elements (the images of $\alpha, \beta, \gamma, \delta$ and one public-input base ψ_i per public input). The bulk of the ceremony output, the powers of τ whose length scales with the circuit’s constraint count, sits in the *proving key*, not vk . Because the circuit `SNARK.Vrfy` is fixed, its vk is fixed too, so it can be audited once and reused across every instance: this is the “one audited verifier” of §12.3, with only the per-UTXO public inputs distinguishing deposits (Remark 12.3).

12.2 Garbled Verifier Overview

Four facts fix the shape of everything in §13; they are collected here once, before the protocol uses them.

1. Input side. The GC is itself the SNARK verifier. It takes a proof $\pi = (A, B, C)$, with A, C of size 32 bytes and B of size 64 bytes, so π totals 128 bytes = 1024 bits, meaning the GC has 1024 input wires, each with two labels: 2048 labels total, 2^{1024} possible input

³⁷Here w is the *SNARK witness*: the prover’s private input (the destination-chain burn transaction, block header, and Merkle proofs). It is unrelated to the *Bitcoin transaction witness*, the on-chain unlocking field (§11.5) that carries signatures and released labels.

³⁸ $[k]G$ is scalar multiplication: the point G added to itself k times under the elliptic-curve group law (§9.2). See https://en.wikipedia.org/wiki/Elliptic_curve_point_multiplication.

combinations.³⁹ The verifier’s public inputs are hard-coded into the circuit at setup (§12.3), which is why the proof bits are the only input wires.

2. Output side. The GC has one output wire with two labels, $L_{\text{out}}^{\text{True}}$ and $L_{\text{out}}^{\text{False}}$: either the evaluation result is the preimage of the on-chain hash $h = H(L_{\text{out}}^{\text{False}})$, or it is not; looking it up in the decoding info d also reveals the plaintext verdict. Only the False label is ever hash-committed on-chain (developed in Remark 13.3).

3. Commitment power, not forging power. Holding the input labels gives the operator the ability to *commit* any proof π to the garbled verifier, not the ability to *forge* one: an accepting π without a valid witness happens with negligible probability, exactly as for any other verifier. The map input $\rightarrow$ output is the deterministic public `SNARK.Vrfy`, so freedom over inputs is worthless when no accepting input exists.

4. Why e stays secret. The encoding info e (the input label pairs) must nonetheless be kept secret from challengers: anyone holding e could encode a fake proof, run it through the GC, and evaluate their way to the preimage of the hash, stealing the bond from an honest operator’s Assert. Authenticity (Theorem 2.5) protects the honest operator only while challengers hold nothing beyond the one released encoding $\text{En}(e, \pi)$.

12.3 Public Inputs and Binding

The statement φ is a vector of *public input values*, supplied per proof, that bind π to a specific assertion; the verifier must hold them, or a proof would verify without attesting to *anything*. In a bridge these encode circuit-format constants plus a commitment to the destination-chain event (peg-out txid, a light-client/SPV inclusion proof, accumulated chain work).

The binding is mechanical, i.e. it is established automatically as a consequence of the verification arithmetic, not by any separate check or assumption. The circuit-specific phase of setup fixes, for each public-input wire i , a group element $\psi_i \in vk$. At verification the public-input scalar x_i multiplies its ψ_i , and the term $\sum_i [x_i] \psi_i$ enters the pairing equation, so a proof verifies only for the exact x_i it was made against. In BitVM3 the ψ_i are identical across all UTXOs (one audited verifier), while the x_i are *per-UTXO scalars hard-coded at garble time*, which is what pins a garbled instance to one deposit.

Remark 12.3 (Pooled-UTXO hazard). Given the verifier circuit must be identical across UTXOs to be audited once, the *public inputs* are the only thing distinguishing instances. If they do not bind per-UTXO, an operator holding a valid proof for one deposit could replay it against another, draining a co-depositor. The per-UTXO scalar x_i , bound through ψ_i , is precisely what prevents this cross-claim.

12.4 Trusted Setup and Toxic Waste

Groth16’s setup runs in two phases. *Powers of Tau* encodes powers of a secret τ as group elements, producing a *structured reference string* (SRS) that is *universal*, reusable across circuits. The *circuit-specific* phase takes the SRS and the circuit and produces the CRS (pk, vk) , reusable across all proofs for that circuit. Both phases generate secret scalars (the toxic waste) that must be destroyed: whoever retains them can forge proofs. In practice the universal phase is a public ceremony reused by many protocols (e.g. the one SP1 relies on), so only the circuit-specific phase is protocol-specific (a fuller treatment of Groth16 is in the companion notes, <https://github.com/CLGAP/Crypto/blob/main/docs/theory.pdf>).

³⁹ B lives in $\mathbb{G}_2$, whose points have coordinates in the extension field $\mathbb{F}_{p^2}$ (the sextic twist allows $\mathbb{G}_2$, natively in $\mathbb{F}_{p^{12}}$, to be represented in $\mathbb{F}_{p^2}$: an efficiency win), hence twice the size of a $\mathbb{G}_1$ point. A and B must sit in different groups for the pairing $e : \mathbb{G}_1 \times \mathbb{G}_2 \rightarrow \mathbb{G}_T$; symmetric variants exist but are insecure or impractical at useful parameter sizes.

12.5 How the Proof Reaches the Chain

Bitcoin never runs `SNARK.Vrfy`. The operator garbles it, and the proof π enters as the garbled input: its encoding $\text{En}(e, \pi)$ is committed at garble time and released on-chain only by signing π through the garbled-encoding extractable / adaptor signatures of §9.6. So the on-chain object committing the proof is the adaptor signature, and the public-input binding above is what ties that commitment to a specific claim.

Remark 12.4 (What a “trace” is, and why BitVM2 commits it). An *execution trace* of `SNARK.Vrfy` is the full step-by-step record of running it: every intermediate value the computation produces, gate by gate (or instruction by instruction), not just the final accept/reject bit. It is the same object as the complete wire assignment $\{\beta_v\}$ of Definition 1.2, taken over *every* vertex rather than the output wires alone. BitVM2 settles disputes over this trace directly: the verifier’s computation is compiled to a circuit, the whole trace is bit-committed on-chain, and a challenger who disagrees *bisects* to a single step and has Bitcoin Script re-execute just that one low-level operation, cheap enough to fit the script budget (the same single-instruction idea underlies BitVMX, Remark 6.1). The cost is that committing and disputing the trace is on-chain-heavy. BitVM3 removes the trace from the chain entirely: the challenger evaluates the garbled verifier off-chain (§13) and the only on-chain operation is a single hash equality, which is where the $\sim 1000\times$ reduction over BitVM2 comes from.

12.6 Structuring the Verifier Circuit

As it is the *verifier* that is garbled, the sole design pressure is the verifier’s gate count: every AND gate becomes a table the garbler commits and the challenger evaluates, so a smaller verifier means a smaller garbled circuit, a smaller setup, and a cheaper reveal. This is why the garbled verifier is Groth16’s; the choice is comparative, not a claim that the verifier is cheap.

Groth16’s verifier is a fixed set of elliptic-curve operations: the pairing check $e(A, B) = e(\alpha, \beta) \cdot e(vk_\varphi, \gamma) \cdot e(C, \delta)$ with $vk_\varphi = \sum_j [x_j] \psi_j$: the verifying key specialized to the statement φ , i.e. the fixed bases $\psi_j \in vk$ weighted by the claim’s public-input scalars x_j , and the only statement-dependent term in the check. In total, four pairings and one multi-scalar multiplication over BN254. As Boolean gates this is still *millions* of gates: a single BN254 pairing is a Miller loop and final exponentiation over $\mathbb{F}_{p^{12}}$, thousands of ~ 254 -bit field multiplications deep.⁴⁰ Fewer than four are actually garbled: $e(\alpha, \beta)$ is a setup constant, and for the hard-coded public inputs of §12.3 so is $e(vk_\varphi, \gamma)$, leaving exactly the two proof-dependent pairings, $e(A, B)$ and $e(C, \delta)$.

Two properties make Groth16 the least-bad target rather than a small one: the gate count is *constant in the statement* being proved, and it is far smaller than any competing verifier.⁴¹ A STARK or PLONK verifier runs tens to hundreds of thousands of hash and field operations, larger still as a Boolean circuit. This comparison is specific to garbling: when the verifier is instead *executed* in script (the BitVM2 route of Remark 12.4), Bitcoin’s native hash opcodes may favour hash-heavy verifiers if the verifier’s hash function is among those Bitcoin Script supports natively; a garbled verifier is never executed on-chain, so only Boolean gate count matters, and a single hash invocation there is tens of thousands of gates.

The standard mitigation is *recursion*. A fast, transparent inner system (a STARK from Risc0 or SP1) proves the application statement; a single outer Groth16 proof then attests “I checked that inner proof”, and it is the outer Groth16 verifier, constant-size, that is compiled

⁴⁰The deployed BitVM line lowers this cost by *verifying* the pairing rather than computing it: the final exponentiation is replaced by a residue witness check and the Miller loop shrinks under precomputed line evaluations (Novakovic and Eagen, *On Proving Pairings*, ePrint 2024/640, <https://eprint.iacr.org/2024/640>).

⁴¹J. Groth, *On the Size of Pairing-based Non-interactive Arguments*, EUROCRYPT 2016. ePrint 2016/260, <https://eprint.iacr.org/2016/260>.

into the garbled circuit.⁴² The garbled object then never grows with the application, only with the outer verifier. The division of labour: the STARK carries the application claim, the Groth16 wrap attests that the STARK verifies, and the garbled circuit is the Groth16 verifier, whose input is that wrap proof $\pi = (A, B, C)$, never the STARK itself.

Curve choice is the remaining lever. BN254 is the de facto target because Ethereum’s precompiles fix it and because its ~ 254 -bit field keeps the pairing circuit smaller than BLS12-381’s ~ 381 -bit field; the trade is BN254’s thinner security margin (~ 100 bits under recent tower-field-NFS estimates) against BLS12-381’s ~ 120 .⁴³ Either way the garbled Groth16 verifier remains tens of GB even after the optimizations of §4 (the ≈ 40 GB of Remark 4.1 already assumes half-gates), which is what motivates the designs of §14: Glock and BABE keep the optimized Boolean garbler and restructure *what* is garbled, while Argo (§14.2) replaces the Boolean primitive itself with arithmetic-circuit garbling.

12.7 Hazards and Constraints

Garbling a verifier, rather than running one, brings its own hazards and constraints:

- **Public-input count, order, and encoding.** The term $\sum_j [x_j] \psi_j$ fixes one $\psi_j \in vk$ per public input, and the garbled circuit bakes in that count and order. A statement re-parameterized after garble time (an extra input, a reordering, a different field encoding of x_j) no longer matches the committed ψ_j and simply fails. For example, a 256-bit event digest does not fit in one ~ 254 -bit BN254 field element, so it enters as two limbs (two x_j); switching to a truncated single-element encoding, or batching several withdrawals under one Merkle root where a single digest stood before, changes the constraint system interpreting the inputs even when the input count is unchanged. Either change means a new circuit-specific (pk, vk) : a fresh ceremony, a fresh audit, a re-garbling, and a re-signed graph, not a configuration change.
- **Static vs. dynamic public inputs.** Garbling freezes the statement at setup: the efficient construction folds vk_φ into the circuit as a constant (the folding that lets §12.6 drop $e(vk_\varphi, \gamma)$ from the garbled pairings), so every x_j must be known at garble time. A bridge violates this: the peg-out output digest exists only once a user withdraws, long after peg-in. The BitVM3 paper meets this with enumeration and commitment: a garbled instance is prepared per anticipated claim slot (per checkpoint and peg-out instance, its statement folded in), and genuinely runtime values, such as the light client’s checkpoint block hash and timestamp, are bound by on-chain Winternitz commitments the relation is written against (§5). The witness-encryption variant instead rebinds at runtime: its ciphertext bakes the statement term in at setup, and the *deferred binding* split of §14.4 binds the dynamic scalar at evaluation.
- **Proof non-malleability.** A Groth16 proof is not unique: negating both A and B satisfies the verification equation unchanged, since $e(-A, -B) = e(A, B)$ by bilinearity, so $(-A, -B, C)$ is a second valid proof of the same statement (and full re-randomization with fresh scalars yields infinitely many more, so no canonical-form check closes the gap⁴⁴). The hazard is not soundness (a mauled proof still proves the same statement) but any surrounding logic that uses proof bytes as an identifier: replay protection keyed on a hash

⁴²Risc0, <https://dev.risczero.com>; SP1 (Succinct), <https://docs.succinct.xyz>. Both expose a Groth16 “wrap” of an inner STARK for cheap on-chain verification.

⁴³Curve comparison and the tower-NFS re-estimate: <https://hackmd.io/@benjaminion/bls12-381>.

⁴⁴Concretely: for a valid (A, B, C) and fresh $r, s \xleftarrow{\$} \mathbb{Z}_q^*$, the tuple $([r]A, [1/r]B + [s]\delta, C + [rs]A)$ verifies for the same statement: the left side becomes $e(A, B) \cdot e(A, \delta)^{rs}$ and the shifted C contributes the matching factor $e(A, \delta)^{rs}$ on the right. Only the public proof and $\delta \in vk$ are needed, so anyone can maul; negation is the case $r = -1, s = 0$. The output is distributed exactly as a fresh honest proof, since the prover’s own internal randomness is what is being re-sampled, which is why mauled proofs cannot be filtered.

of the proof, for instance, reads a mauled proof as a fresh claim and processes the same event twice. The graph must therefore bind the *statement* (via the public inputs and the covenant), never the proof bytes.

- **Verifying-key binding.** The audited object is one vk ; a garbled instance is only as sound as the guarantee that it garbles *that* vk , which is precisely what setup verification certifies (§10). The ceremony behind (pk, vk) remains a trust root with toxic waste to destroy (§12.4), but that hazard attaches to any Groth16 deployment, not to garbling specifically.
- **Cross-instance replay.** The public inputs are the only per-instance binding across UTXOs; without a per-UTXO scalar, a proof for one deposit replays against another (Remark 12.3).

13 BitVM3-core Protocol

The garbling primitive of §3 (**Gb, En, Ev, De**) and the extractable signature scheme of §8 form elements which can be composed into a Bitcoin bridge. The operator garbles the SNARK verifier circuit, proves the garbling well-formed (§10), and binds a signature scheme to its input encoding; posting a signature on a SNARK proof on-chain then releases input labels which can be evaluated on that garbled circuit. Any challenger can then run the garbled verifier off-chain, and if valid, the proof will produce an output wire which corresponds to a label which will decode to true, and if invalid the proof the output label will return a label which the decoder will identify as false, and that label can then be posted in a fraud-proof to slash the operator’s bond. Bitcoin script itself never executes the SNARK verifier: it only checks a hash equality, which is what keeps the construction within Bitcoin’s size (scripting) constraints..

Throughout this section the label reveal is the Schnorr adaptor realization of §9: labels committed as curve points $T = [\ell]G$, released by completing pre-signatures (one 65-byte adaptor signature per proof digit), chosen for on-chain cost: about 8 witness bytes per proof bit against roughly 25 for Winternitz and 94 for Lamport (BitVM3 §8.1). The notation **GS.Sign**/**GS.Extract** below is shorthand for the GES interface of §8, whose security properties (extractability, unforgeability) the realization must meet; it does not imply the hash-based instantiation.

Six artefacts are produced once at setup, per eligible operator (only the audited verifier circuit is shared across operators; Remark 11.6), and referenced throughout:

1. Garbled circuit GC of **SNARK.Vrfy**, a public list of encrypted truth tables (§3.2).
2. Consistency proof that GC and the committed labels are a faithful garbling of the public circuit, and that pk_{GS} commits the same encoding e (§10), so the self-garbling operator cannot encode a wrong verdict.
3. Decoding info d , mapping output-wire labels back to True/False (§2).
4. GES keypair (pk_{GS}, sk_{GS}) , extractable signatures keyed to the garbled circuit’s input encoding e (§8). (pk_{GS} is the public table of points $T_{i,b} = [L_{i,b}]G$ against which challengers decode released scalars back into labels and bits; $sk_{GS} = e$ itself, the encoding info returned by the same **Gb** call that produced GC and d above.)
5. Transaction graph $\{\text{Assert}, \text{Disprove}, \dots\}$, pre-signed at setup (§11.6).
6. Signer committee $S_1, \dots, S_N$, which presigns the transaction graph once, then deletes setup-time secrets (§13.1). (After deletion no one, the operator included, can sign fresh under the slot key: a valid Assert signature can only arise by scalar-adding a label onto a setup pre-signature; Remark 9.8.)

The protocol then runs in three phases, each expanded in the subsections that follow:

1. **Setup** (off-chain, once): operator runs **Gb** and **GS.KeyGen**, and proves GC well-formed against the agreed verifier circuit, i.e. the public vk and hard-coded public inputs (§12), publishing the certification in whichever form the chosen variant of §10 dictates (instance commitments and opened seeds, a setup proof, or an attestation); committee N -of- N presigns the transaction graph over a funding outpoint fixed in advance (Remark 11.5); $(GC, d, pk_{GS}, \text{Assert})$ and all pre-signatures are published: everyone sees them, but only the label-holder can complete one.
2. **Proving** (on-chain, per claim): operator builds the SNARK proof π and signs it into the label-revealing signature $\sigma_\pi \leftarrow \text{GS.Sign}(sk_{GS}, \pi)$: for each bit π_i , select that wire's matching label, so $\sigma_\pi = (L_{1,\pi_1}, L_{2,\pi_2}, \dots, L_{\ell,\pi_\ell}) = \text{En}(e, \pi)$; the signature is the garbled input. The operator posts the Assert transaction with σ_π as its witness.
3. **Disprove** (on-chain, conditional): a challenger extracts the proof's input labels $L_\pi \leftarrow \text{GS.Extract}(pk_{GS}, \sigma_\pi)$ from the posted signature, evaluates them through the garbled circuit to the output label $L_y \leftarrow \text{Ev}(GC, L_\pi)$, and decodes that label to the plaintext verdict $y \leftarrow \text{De}(d, L_y)$; if $y = \text{False}$ they post Disprove with the output label L_y as the fraud-proof witness: this $L_y = L^*$ is the preimage of the hash $H(L^*)$ committed in the challenge leaf, so the script check is a single hash equality.

13.1 Role Assignments

The protocol partitions roles cleanly across three party classes (per §2.2 and Fig. 6 of the BitVM3 paper). The *operator* (the prover P) plays the garbler: they run $\text{Gb}(1^\kappa, \text{Circuit}(\text{SNARK.Vrfy}))$ to obtain (GC, e, d) , run $\text{GS.KeyGen}(1^\kappa, e)$ to obtain (pk_{GS}, sk_{GS}) , and send $(GC, d, pk_{GS}, \text{Assert})$ to the verifiers. When claiming, the operator signs the SNARK proof $\sigma_\pi \leftarrow \text{GS.Sign}(sk_{GS}, \pi)$ and posts Assert on-chain with σ_π as witness.

The *challenger* (any verifier $V \in \mathcal{V}$) plays the evaluator: extracts $L_\pi \leftarrow \text{GS.Extract}(pk_{GS}, \sigma_\pi)$ from the on-chain signature, computes $L_y \leftarrow \text{Ev}(GC, L_\pi)$ and $y \leftarrow \text{De}(d, L_y)$ off-chain, and posts Disprove with L_y as fraud-proof witness if $y = \text{False}$. The role is permissionless: any signer, operator, or user may act as a challenger, and the protocol assumes at least one honest challenger is online at all times.

Remark 13.1 (No challenge bond). A spurious Disprove is not economically deterred but cryptographically impossible: its witness L^* is obtainable only when the proof actually evaluates False (Theorem 2.5), so challenging carries no bond and costs only the transaction fee. Challenge bonds as anti-spam belong to interactive dispute designs (BitVM2), which the one-shot Disprove eliminates.

The *signer committee* $(S_1, \dots, S_N)$ does *not* garble; it N -of- N presigns the transaction graph at setup using **SIGHASH_ALL**, providing covenant emulation that prevents the operator from substituting different transactions. Setup integrity holds as long as one honest signer deletes their setup-time secret afterwards.

Finally, *Bitcoin* plays neither garbler nor evaluator: it only adjudicates the on-chain hash check $H(L_y) = H(L^*)$, never executing **Ev** or **De** itself.

13.2 Setup, Proving, Disprove

This subsection follows the data through the three phases of Fig. 6; the enumerate above is the summary, here we name the artefact each step produces or consumes.

13.2.1 Setup (off-chain, once)

The operator fixes the circuit $\text{Circuit}(\text{SNARK.Vrfy})$ for the statement to be proved and runs $(GC, e, d) \leftarrow \text{Gb}(1^\kappa, \cdot)$, then $(pk_{GS}, sk_{GS}) \leftarrow \text{GS.KeyGen}(1^\kappa, e)$, binding the signature scheme to

the input encoding. The public bundle (GC, d, pk_{GS}) goes to the verifiers: pk_{GS} commits to both labels of every input wire (as points $T_{i,b} = [L_{i,b}]G$; §9.6) and d commits to the output-wire labels, so the dispute can later reduce to a hash equality. Since the function $f = \text{SNARK.Vrfy}$ is public, the operator also posts a one-time consistency proof that GC and the committed labels are a faithful garbling of it; this is what stops a malicious operator from garbling a circuit that computes the wrong verdict, a guarantee the garbling *authenticity* property alone does not provide. In parallel the signer committee $S_1, \dots, S_N$ presigns the transaction graph (Assert, Disprove, payout, and timeout transactions) N -of- N under SIGHASH_ALL, emulating a covenant that pins which transaction may spend which output; at least one honest signer then deletes their share, so the graph can never be re-signed. In the deployed adaptor realization, setup also *publishes* the per-digit pre-signatures, each bound to one label statement $T = [L]G$ (Example 13.5). This is where spendability is fixed: the pre-signatures are public, but completing one requires its label, and once the slot key's shares are deleted (Remark 9.8) completions are the *only* valid signatures that can ever exist. Holding the labels e is what it means to be able to spend the Assert.

Remark 13.2 (What pk_{GS} is for). Three consumers, none of them Bitcoin consensus (the script sees only pk_{slot}):

1. *Setup verification*: the consistency proof binds the table to the circuit, showing the points $T_{i,b}$ commit the same encoding e that GC was garbled with (§10).
2. *Extraction*: a challenger decodes a posted signature against the table: the recovered scalar must satisfy $[s - \tilde{s}]G = T_{i,b}$, and *which* of the wire's two points matches determines the bit b . Without the table, released scalars are meaningless random numbers; with it they become “wire i , bit b ”, i.e. a well-defined garbled input.
3. *Equivocation evidence*: both points of every wire are committed upfront, so a second opening on any wire is provable cheating (slashable).

Remark 13.3 (Recipe: constructing the Disprove gadget). The Disprove path (§11.6) is assembled from pieces defined separately; collected here as one construction, then read off as the three behaviours it guarantees.

1. *Garble the public verifier*. Take $f = \text{SNARK.Vrfy}$, a single-output circuit ($m = 1$; Definition 1.2), and run $(GC, e, d) \leftarrow \text{Gb}(1^\kappa, f)$. Its one output wire carries a label pair $(L_{\text{out}}^{\text{False}}, L_{\text{out}}^{\text{True}})$, two uniform κ -bit strings (Definition 3.1). Nothing about the gates is special: the “gadget” lives entirely in *which output label is committed where*.
2. *Commit the boundary asymmetrically*. pk_{GS} commits *both* labels of every *input* wire, so a signature on π releases exactly $\text{En}(e, \pi)$ (§8). At the *output*, publish only $h := H(L_{\text{out}}^{\text{False}})$ into the Disprove challenge leaf $\text{Hashlock}(h)$ (§11.6). The True label is never hash-committed on-chain: d decodes it off-chain so an honest challenger knows *not* to disprove, but no Bitcoin script ever needs its preimage. Only the reject outcome arms a spend.
3. *Certify well-formedness*. Prove GC , the committed labels, and d are a faithful garbling of the *agreed public* f (§10); otherwise a malicious garbler could commit $H(L_{\text{out}}^{\text{False}})$ of a circuit that outputs True on every input. Garbling authenticity alone does not cover a dishonest *garbler*.

The behaviour this produces, matching the three requirements:

- *The accepting outcome yields nothing spendable*. A π with verdict True evaluates to $L_{\text{out}}^{\text{True}}$, whose hash is not h ; by authenticity (Theorem 2.5) no evaluator can produce $L_{\text{out}}^{\text{False}}$ from it. No Disprove exists, and the operator times out and reclaims.

- *Every rejecting outcome hands over the false preimage.* Any π with verdict False evaluates *deterministically* to $L_{\text{out}}^{\text{False}} = L^*$, the exact preimage of h ; the challenger drops it in the Disprove witness and the hashlock opens (slash).
- *Undesignated labels return nothing.* Feed any label not from a committed pair and trial decryption finds no row ending in 0^t , so Ev/De yields $\perp$ (§3.2, Theorem 2.5). On the input side, a slot’s CHECKSIGVERIFY completes *only* with its committed label (Remark 13.6), so the operator cannot Assert without releasing genuine $\text{En}(e, \pi)$.

A caveat on framing: **SNARK.Vrfy** is not “one input True, all others False” – it accepts *every* valid proof and rejects the rest. The gadget pins a single committed π per Assert, and the trap fires on *that* π ’s output-wire verdict, not on a unique accepting input.

13.2.2 Proving (on-chain, per claim)

To assert a claim φ (for instance a destination-chain burn that should release pegged funds), the operator builds a SNARK proof π with $\text{SNARK.Vrfy}(\pi) = \text{True}$, signs it as $\sigma_\pi \leftarrow \text{GS.Sign}(sk_{GS}, \pi)$, and posts the Assert transaction with σ_π in its witness. By extractability (§8) this single act publishes $L_\pi = \text{En}(e, \pi)$, exactly one label per input wire, the active half of e selected by π . In the deployed adaptor realization this “signing” involves no key at all: per digit, the operator performs one scalar addition on a published pre-signature, $s = \tilde{s} + L$. Completing with any scalar other than the committed label fails the Schnorr verification equation (Remark 13.6), so only genuine input labels of the garbled circuit produce a spendable Assert. The operator’s bond sits in the output that Disprove or the timeout will spend. The bond is distinct from the deposit: the depositor’s pegged funds sit in the bridge reserve and move only via the withdrawal transaction, which requires the surviving Assert as a connector input, while the slashable output is the operator’s own collateral, so lying costs the operator its stake, never the depositor’s funds.

What a claim commits, and where:

- *Public inputs ϕ :* an anchor commitment to destination-chain state (the light-client root the relation verifies inclusion against), the peg-out txid (pinning which deposit is reimbursed; Remark 12.3), and a session identifier binding the proof to this garbled instance.
- *SNARK witness (private):* the burn transaction, its block header, and Merkle inclusion proofs, establishing that the claimed event actually occurred under the anchor.
- *On-chain Winternitz commitments:* the light client’s checkpoint block hash and timestamp, advancing destination-chain state accountably, since committing two conflicting values is slashable equivocation (Remark 6.1). A hash-based OTS is the carrier because Bitcoin Script is stateless and CHECKSIG signs only transactions: with the OTS public key embedded in the script, hash opcodes verify the revealed preimages, making committed *data* script-checkable.
- *The pre-signed graph:* the payout destination itself, fixed at setup so a valid claim cannot redirect funds (Remark 11.5).

The relation itself is the paper’s Algorithm 4 ($\mathcal{R}_{\text{eth-bridge}}$): verify a chain of sync-committee handoffs from C_{anchor} to a committee finalizing block B , verify the burn’s inclusion in B , and check the burn references the deposit (txid || sid); the Bitcoin-side checkpoint chain is a separate relation (Algorithm 3, $\mathcal{R}_{\text{btc-lc}}$).

13.2.3 Disprove (on-chain, conditional, within the challenge window)

Any challenger reads σ_π off chain, extracts $L_\pi \leftarrow \text{GS.Extract}(pk_{GS}, \sigma_\pi)$, and evaluates the garbled verifier off chain: $L_y \leftarrow \text{Ev}(GC, L_\pi)$, then $y \leftarrow \text{De}(d, L_y)$. If $y = \text{False}$ the proof was

invalid, so the challenger posts Disprove carrying the recovered output label L_y as witness; the script checks only the hash equality $H(L_y) = H(L^{\text{False}})$ against the committed false-output label, and on success slashes the operator's bond. If no valid Disprove appears before the timeout expires, the claim finalises and the operator collects. Bitcoin script therefore runs one hash comparison, never Ev or De.

13.2.4 Anatomy of the Assert Broadcast

The Assert template is fixed at setup: its input spends the bonded output, its outputs (the challenge/timeout taptree) are pinned by the committee's N -of- N SIGHASH_ALL signatures. What varies at broadcast time is only the witness.

Remark 13.4 (How one locking script accepts 2^{1024} witnesses). The bonded output's tapleaf is, schematically,

$\langle pk_{slot} \rangle$ CHECKSIGVERIFY CODESEPARATOR $\langle pk_{slot} \rangle$ CHECKSIGVERIFY $\dots$ (128 repetitions).

We write pk_{slot} , not pk_{op} : the operator must never hold this key's secret, or they could fresh-sign a consensus-valid Assert that completes no pre-signature and reveals no labels. It is a joint key whose shares are deleted after the pre-signatures are made (Remark 9.8); the paper writes it as the operator's key, leaving this arrangement unspecified. Script execution, performed by every validating node's interpreter when the transaction is relayed or mined, pops one item off the witness stack per CHECKSIGVERIFY and demands a valid BIP340 signature under pk_{slot} ; any failure invalidates the transaction: this is what *enforces* that all 128 slots are present and genuine. The count derives from the proof: $|\pi| = 128$ bytes = 1024 bits, one garbled input wire per bit, grouped 8 bits per slot, one signature per digit. Two facts make the same script accept many different fillings. (i) A signature check verifies against a *message*, not against a listed set of allowed signatures: any (R, s) satisfying the verification equation (9.1) for the slot's sighash passes. Both completions $\tilde{\sigma}_i^0 + L_i^0$ and $\tilde{\sigma}_i^1 + L_i^1$ are exactly such valid signatures. The script cannot and need not distinguish them; the *pre-signatures* (off-chain artefacts) are what tie each acceptable signature to one label. (ii) The sighash never commits to witness data (§11.3), so the committee's covenant signatures pin inputs and outputs while leaving the witness free: one pre-signed template, $(2^8)^{128} = 2^{1024}$ admissible witnesses, one per possible proof value $\pi \in \{0, 1\}^{1024}$, chosen at broadcast. For that freedom to be *sound*, each slot must sign a *distinct* message, so no signature can be moved between slots; making the 128 messages distinct under a single key is exactly the job of OP_CODESEPARATOR (Remark 13.7). Cost note: 128×65 signature bytes ≈ 8.3 kB; the full Assert witness is ≈ 9.2 kB, ≈ 2.4 kB after the segwit discount; at 8 bits per 65-byte signature (≈ 8 bytes per proof bit) this is the $\sim 3\times$ saving over Winternitz's ~ 25 bytes per bit (§7). The single key also keeps the *script* small: it is pushed once, and the optimized leaf is $33 + 3(N - 1) + 1$ bytes (BitVM3 §8.1), each additional slot costing 3 bytes rather than a fresh 33-byte key push.

The remark above explains the *flexibility*: why one fixed template admits a filling per proof value (the sighash commits to no witness data, and a signature check accepts any signature valid for its message, not a listed set). The next three pieces explain the *binding*: how the filling actually chosen is pinned down, first per wire (which signature releases which label), then per value (only the committed label completes a slot), then per position (a completed signature is spendable only in its own slot).

Example 13.5 (One-wire reveal). Fix input wire i of the garbled verifier with label pair (L_i^0, L_i^1) , operator key x with $P = [x]G$, and message m the Assert sighash.

1. *Commit* (setup): publish the statements $T_i^b = [L_i^b]G$ and pre-signatures $\tilde{\sigma}_i^b \leftarrow \text{PreSign}(x, m, T_i^b)$ for $b \in \{0, 1\}$. Neither pre-signature verifies as a signature; each becomes one only by adding the discrete log of its statement, $s_i^b = \tilde{s}_i^b + L_i^b$.

2. *Prove* (claim time): the SNARK prover outputs π ; set $b := \pi_i$ (say $b = 1$).
3. *Complete*: $\sigma_i \leftarrow \text{Adapt}(\tilde{\sigma}_i^1, L_i^1) = (R + T_i^1, \tilde{s}_i^1 + L_i^1)$, possible only for the operator since L_i^1 is secret; place σ_i in witness slot i . All 128 slots filled this way form the one witness consensus accepts.
4. *Extract* (any challenger): $L_i^1 = s_i - \tilde{s}_i^1$; collected over all wires this is exactly $\text{En}(e, \pi)$, complementary labels untouched.

Completing *both* pre-signatures of a wire (equivocation) reuses the nonce and leaks the operator's signing key (§6.2, §9.4).

Remark 13.6 (Only the committed label completes the signature). The binding runs the other way too: a slot spends *only* with its committed label. The pre-signature $\tilde{\sigma}_i^b$ fixes the statement $T_i^b = [L_i^b]G$ at setup, and completing it adds a scalar t with $s = \tilde{s}_i^b + t$; by Proposition 9.3 the result $(R + T_i^b, s)$ satisfies the Schnorr equation $[s]G = (R + T_i^b) + [e]P$ iff $[t]G = T_i^b$, that is $t = L_i^b$. Any other scalar yields an (R, s) that fails verification, so the slot's `OP_CHECKSIGVERIFY` rejects (clause (i) above) and the Assert does not spend. The label is carried inside the tweak T_i^b , which enters the challenge $e = H(R + T_i^b \parallel P \parallel m)$, so a wrong label is a wrong tweak and a wrong challenge; the operator cannot post a valid Assert without releasing the genuine $\text{En}(e, \pi)$.

Remark 13.7 (Why slots need distinct messages: reordering and replay). Remark 13.6 pins which *value* completes a slot; nothing so far pins which *position* a completed signature may occupy. A Schnorr check binds a signature to a *message*, never to a position in the witness. That message is the slot's sighash, and consensus accepts *any* (R, s) satisfying the verification equation (9.1) for it; the interpreter has no notion of “this signature was produced for slot i ”. The witness is attacker-supplied data, and `CHECKSIGVERIFY` merely pops the next stack item and checks it against the current message: reading the slots *in order* is not the same as checking that item k belongs to slot k . Nothing in the script ties a signature to a slot except which message it verifies against.

Suppose the 128 slots were *not* separated by `OP_CODESEPARATOR`. Each is $\langle pk_{slot} \rangle$ `CHECKSIGVERIFY` under the same key over the same transaction, so all 128 sighash messages are byte-for-byte identical; call the common value m . Every completed digit-signature verifies against m , hence against *every* slot. Whoever assembles the witness (the operator) can then *reorder* (place the 128 signatures in a permuted order) or *replay* (drop one digit's signature into many slots), and consensus still accepts: every check passes, since every signature verifies against the one common m .

Why a cheating operator wants this freedom: suppose no valid proof exists (the claimed event never happened). Filling the slots honestly is self-defeating, since evaluation would hand challengers the false-output label L^* (the paragraph below). A scrambled witness escapes this. Extraction is positional: the challenger recovers slot i 's label as $s_i - \tilde{s}_i^b$ against slot i 's own pre-signatures, so a misplaced signature (one that actually completes slot j) yields a scalar that is no label at all, and a replayed signature releases one wire's label while other wires' labels never surface. Either way the challenger holds no well-formed $\text{En}(e, \pi)$: the recovered non-labels match no ciphertext row, trial decryption fails (§3.2), `Ev` aborts before any output label, and in particular L^* , is recovered. No Disprove can be posted, the timeout passes, and the operator collects on an invalid claim.

`OP_CODESEPARATOR` closes this by making the message position-specific. The opcode's standard description reads “ignore this and everything preceding when deciding what to sign when signature-checking”: in legacy script the signed script truncates to what follows the last executed separator. Tapscript refines this: the *whole* leaf stays committed via its leaf hash, and BIP342 instead folds the opcode position of the last executed separator into the sighash, so slot i , sitting after the i -th separator, signs a message m_i with the m_i pairwise distinct. A signature for m_i now verifies *only* at slot i : reordering or replay fails the very check it was

trying to pass, and a consensus-valid Assert necessarily releases the genuine, complete $\mathbf{En}(e, \pi)$ for a single well-defined π . Valid-on-chain then implies evaluable-off-chain, hence disprovable. One key still suffices: it is the message that is made position-specific, not the key, which is why 128 distinct keys would be the redundant alternative.

Proposition 13.8 (Position-binding is necessary and sufficient for extractability). Write $m_1, \dots, m_{128}$ for the slot sighashes of the Assert.

1. If the slots are not separated, so $m_1 = \dots = m_{128} = m$, there exists a consensus-valid Assert witness whose extraction returns $\mathbf{En}(e, \pi)$ for *no* π .
2. If the m_d are pairwise distinct and no fresh signature under the slot key can be produced after setup (Remark 9.8), every consensus-valid Assert witness extracts to $\mathbf{En}(e, \pi)$ for a unique π .

Proof. (i) Let σ be any single completed signature, valid for the common message m , and fill all 128 witness slots with copies of σ . Every CHECKSIGVERIFY checks its popped item against m and passes, so the transaction is consensus-valid. Extraction at slot d is positional: the challenger matches the posted nonce point against slot d 's published candidates $R_{d,v} + T_{d,v}$. For d other than σ 's slot of origin no candidate matches (nonces are fresh per pair, so a cross-slot coincidence has negligible probability), and subtracting any $\tilde{s}_{d,v}$ yields a scalar t with $[t]G \neq T_{d,v}$: no key, no decrypted row, and Ev never runs. A valid Assert has revealed no encoding, so extractability fails.

(ii) With distinct messages, the signature at slot d verifies only against m_d . By the setup assumption the only producible signatures for m_d are completions of slot d 's published pre-signatures, so the nonce-point match identifies a unique v_d , and subtraction yields W with $[W]G = T_{d,v_d}$, a genuine key (Remark 13.6). All 128 slots extract, and the recovered values assemble $\mathbf{En}(e, \pi)$ for exactly the π whose digits are $(v_1, \dots, v_{128})$. $\square$

Remark 13.9 (What separates an honest Assert from a dishonest one). Nothing on-chain: both are the same template, valid by consensus, and extraction works identically on both. The separation happens off-chain, at evaluation: the challenger feeds the extracted labels into the public garbled tables and runs Ev gate by gate, obtaining the output label L_{out}^v for $v = \text{SNARK.Vrfy}(\varphi, \pi)$. *Ending 1, honest:* the destination event occurred, π verifies, $v = \text{True}$. The evaluator holds $L_{\text{out}}^{\text{True}}$, but the challenge leaf's hashlock demands the preimage of $h = H(L_{\text{out}}^{\text{False}})$, and authenticity (§2.2) says no evaluator can produce the label of the *other* output value. No Disprove exists; after t blocks the operator's timeout path opens. *Ending 2, dishonest:* the event never happened, so by SNARK soundness no valid proof exists: whatever 128 digits the operator committed, $v = \text{False}$, and evaluation *hands the challenger* $L_{\text{out}}^{\text{False}} = L^*$: precisely the hashlock preimage. One Disprove later, the bond is slashed. The operator chooses its ending at completion time; the circuit merely executes the consequence. Lying is never prevented, only made losing: the same broadcast that authorises the claim deterministically arms its own refutation.

13.3 Why the Pieces Compose

Four properties established earlier line up with the four things the protocol needs; it is their conjunction that makes the dispute sound.

1. *Extractability gives availability.* The GES guarantee (§8) is that a signature on π is $\mathbf{En}(e, \pi)$, so σ_π , the one on-chain object the operator must post anyway, already carries the input labels every challenger needs, with no separate label-delivery step and no oblivious transfer (Remark 3.9).

2. *Correctness, with the consistency proof, catches false claims.* The setup consistency proof fixes GC as a faithful garbling of the public SNARK.Vrfy , and correctness then gives $\text{De}(d, \text{Ev}(GC, \text{En}(e, \pi))) = \text{SNARK.Vrfy}(\pi)$. So an invalid π evaluates to False, an honest challenger recovers the False label and disproves it, while a valid π evaluates to True and survives to the timeout.
3. *Authenticity blocks false slashing.* By authenticity (Theorem 2.5), an evaluator holding (GC, L_π) cannot produce any output label other than the genuine $\text{Ev}(GC, L_\pi)$. So no malicious challenger can forge the False label for a valid proof: the check $H(L_y) = H(L^{\text{False}})$ fires only when the proof really is invalid, and an honest operator cannot be framed.
4. *Non-equivocation gives enforceability.* The labels double as the one-time-signature secrets (§6), so the operator can open each input wire to only one bit; revealing both labels of a wire is equivocation, which is itself the slashable evidence (§6.2). In the clean model this non-equivocation is exactly that Lamport preimage leak; under the deployed Schnorr adaptor reveal (§9) the same single-opening guarantee instead comes from the N -of- N presigned graph binding one pre-signature per proof. The committee’s N -of- N presigning and SIGHASH_ALL covenant emulation stop the operator from swapping in a different circuit or redirecting the bonded output, binding the off-chain verdict to a fixed on-chain transaction graph.

In sum: extractability makes the verdict computable by anyone, correctness-with-consistency makes a false claim catchable, authenticity stops an honest claim being slashed, and non-equivocation plus the covenant bind the verdict to a fixed transaction graph. Bitcoin only ever adjudicates a hash equality, yet the dispute it settles is the full SNARK-verification result.

14 Related Designs and Security History

The clean model of §13 is the paper’s abstraction. The systems being built diverge from it, partly for efficiency and partly because the original garbling primitive was broken. This section maps that landscape along two axes: the *garbling primitive* and the *setup-verification and label-reveal mechanism*.

14.1 The RSA Garbling Scheme and Its Break

The first BitVM3 whitepaper instantiated the garbling scheme with RSA: the two labels on a wire are related by modular exponentiation $x \mapsto x^a \bmod N$, and RSA’s multiplicative homomorphism replaces the symmetric encryption of Yao’s tables. This was broken by Futoransky, Larotonda, and Barbàra (ePrint 2025/1291): because a wire’s outgoing exponents a, b can be coprime, a malicious evaluator runs the extended Euclidean algorithm to find k_1, k_2 with $ak_1 + bk_2 = 1$ and recovers the secret label as $x = (x^a)^{k_1} (x^b)^{k_2} \bmod N$, *without* knowing $\varphi(N)$. This forges wire labels the evaluator was never given, breaking authenticity (Theorem 2.5) on a minimal two-AND-gate counterexample. The scheme was retracted; GOAT’s “label forward propagation” optimization built over it inherits the same break.

14.2 Argo: the Size Frontier

The replacement direction is *Argo* (Eagen and Lai, ePrint 2026/049), which encodes wires as elliptic-curve points carrying a homomorphic MAC and garbles *arithmetic* circuits directly, collapsing millions of Boolean gates into single field operations. The claimed reduction is a further $\sim 1000\times$ in garbled-circuit size (a Groth16 verifier from tens of GB to an estimated ~ 25 MB). It is the current frontrunner but is not yet fully specified as a Bitcoin deployment; two other designs chase the same goal (keeping Bitcoin from running the pairing verifier) by different means, treated next.

14.3 Glock: Designated-Verifier SNARK-in-Circuit

Glock (Eagen, Alpen Labs)⁴⁵ keeps the garbled-verifier shape but shrinks what must be garbled by making the check *designated*: rather than a universally-checkable Groth16 verifier, it garbles a designated-verifier SNARK check keyed to the party holding the setup secret. The disprove condition then reduces to a compact predicate rather than a full pairing verifier, and the deployed cut-and-choose machinery (with Mosaic) certifies the garbling honest. It trades the universality of the check for a smaller on-chain footprint.

14.4 BABE: Witness Encryption

BABE⁴⁶ removes the on-chain verifier entirely and replaces it with *witness encryption*: a ciphertext only a valid proof can open. Running the Groth16 identity $e(A, B) = e(\alpha, \beta) \cdot e(vk_\varphi, \gamma) \cdot e(C, \delta)$ (the proof elements A, B, C and bases ψ_j of §12.1), with $vk_\varphi = \sum_j [x_j] \psi_j$, directly on Bitcoin would need four BN254 pairings: hundreds of kilobytes of script, over the limits.

Instead a designated verifier, at setup, samples a secret scalar r and a message msg , garbles a circuit computing $r \cdot A$, and publishes

$$ct_2 = r \cdot \delta, \quad ct_3 = \text{msg} \oplus H(Y^r), \quad Y = e(\alpha, \beta) \cdot e(vk_\varphi, \gamma).$$

An operator holding a valid proof evaluates the garbled circuit to obtain $ct_1 = r \cdot A$, forms

$$Y^r = \frac{e(ct_1, B)}{e(C, ct_2)}, \quad \text{msg} = ct_3 \oplus H(Y^r),$$

and posts msg ; the Bitcoin script checks only $\text{RIPEMD160}(\text{SHA256}(\text{msg})) = h_{\text{msg}}$, a hashlock in place of the pairing verifier. The algebra makes msg recoverable *only* from a proof satisfying the identity, so the hashlock inherits Groth16's soundness. That the garbled circuit was built honestly is enforced by cut-and-choose: with $N = 181$ instances and $M = 7$ hidden, a cheating verifier must corrupt exactly the hidden set, an error of $1/\binom{181}{7} \approx 2^{-40}$ (§10.2.1).

The catch is that $vk_\varphi = \sum_j [x_j] \psi_j$ must be known at garble time, whereas a bridge peg-out has a public input (the destination-chain output digest) fixed only at proving time. GOAT's *deferred binding* splits the public input $x = (x_S, x_D)$ into a static part (the verifying-key hash, fixed at peg-in) and a dynamic part x_D , and splits the computation into three garbled circuits: one computing $r \cdot A$ as before; one computing $Q = x_D \cdot \psi_D + Z$ from the dynamic scalar with a blinding point Z ; and one computing $r \cdot Q$. The blinding Z stops the evaluator (who knows x_D and Q) from isolating $r \cdot \psi_D$ and forging a ciphertext for a public input of their choice, closing the soundness leak while deferring the binding of x_D to evaluation time.

Two clarifications keep the mechanism in its lane. First, x_D enters the second circuit as an ordinary garbled input, its labels released at claim time, so the dynamic scalar is bound late but never trusted: recovering msg still requires a valid proof whose statement includes that exact x_D . Deferred binding defers the *arithmetic* of binding; the SNARK still does the attesting. Second, this is a timing fix, not a trust fix: how the relation authenticates destination-chain state (a light client's consensus assumptions, e.g. sync-committee honesty and weak-subjectivity checkpoints) is untouched, and any weakness there is inherited unchanged.

Unlike the adaptor-signature reveal of §13, BABE has no per-digit label reveal on-chain at all: the only on-chain object is the opened msg , and the proof never touches the chain in the clear. It buys a tiny on-chain footprint at the price of a designated-verifier trust split and a heavy off-chain setup.

⁴⁵Glock, ePrint 2025/1485, <https://eprint.iacr.org/2025/1485>; the cut-and-choose line it develops continues in Mosaic, ePrint 2026/812, <https://eprint.iacr.org/2026/812>.

⁴⁶BABE, Bitcoin Authenticated Binding Encryption, ePrint 2026/065, <https://eprint.iacr.org/2026/065>. The dynamic-public-input extension and the concrete construction summarized here are from GOAT Network, *Deferred Binding: Extending BABE for Dynamic Public Inputs in GOAT BitVM3*, <https://hackmd.io/@goatresearch/HkKp2g1Zf1> (code: <https://github.com/GOATNetwork/bitvm2-gc>). Distinct from the Babylon staking network of Remark 6.11, whose EOTS slashing shares only the equivocation idea; the names are unrelated.

14.5 Comparison: Argo, Glock, BABE

The three share a goal, keeping Bitcoin from running the pairing verifier, but differ on both axes of §14:

	garbling primitive	on-chain reveal / check
Argo	EC-point wires, homomorphic MAC	label reveal (adaptor signature)
Glock	Boolean GC, designated-verifier check	compact disprove predicate
BABE	Boolean GC of $r \cdot A$	witness-encryption hashlock

Argo attacks the *size* of the garbled object (arithmetic wires collapse the Boolean blow-up); BABE attacks the *on-chain check* (a hashlock replaces the verifier); Glock sits between (a smaller garbled check via a designated verifier). Put differently, Glock and BABE change *what* is garbled while keeping the optimized Boolean garbler of §4, shrinking the circuit handed to it; Argo changes the garbler itself. Its wires carry field elements as MAC’d curve points, so one field multiplication is a single garbled gate rather than the $\sim 10^5$ Boolean gates of a 254-bit multiplier, and the bit-label optimizations (Free-XOR, half-gates) are not reused but replaced by its homomorphic structure. The distinction is one of representation, not of statement: all three garble the same computation, Glock and BABE in its *binary* representation (bits on wires, Boolean gates), Argo in its *field* representation. The Free-XOR analogy carries over: XOR *is* addition in $\mathbb{F}_2$, so Free-XOR is the binary case of linear operations costing nothing, and in the arithmetic setting the additively homomorphic MAC plays that role, leaving addition gates free and concentrating cost in multiplication. None is deployed: Argo is not yet fully specified for Bitcoin, and both Glock and BABE lean on cut-and-choose and a designated-verifier trust assumption still being hardened.

14.6 The GSV Lineage

The public **garbled-snark-verifier** reference implementation stays close to the deployed stack this document teaches: half-gate + Free-XOR garbling of the Groth16 verifier, cut-and-choose for setup verification, VSSS to reveal labels cheaply, and Schnorr adaptor signatures on-chain. Its own addition is *soldering*: a base evaluated instance carries the real input labels, and a succinct proof binds the per-wire deltas of every other evaluated instance to that base, so all evaluated instances provably receive the same logical input. It is a two-party “conditional revealing secret scheme” rather than BitVM3-core verbatim, but it composes the same primitives.

14.7 Lombard BSA: TEE-Attested, Sealed Labels

A more distant relative replaces the cryptographic honesty machinery with hardware. The Lombard BSA redesign garbles a *single* instance inside a trusted execution environment (an AWS Nitro enclave) and relies on remote *attestation* instead of cut-and-choose to convince verifiers the garbling was honest. Input labels are *ECIES-sealed* to a recipient public key, so the enclave operator that persists the circuit never learns them; the recipient later decrypts and reveals the label matching its proof on each wire. There is no hash-based one-time signature and no adaptor signature in the reveal path. This trades a cryptographic assumption for a hardware trust assumption, and is a genuinely different protocol, not an instantiation of BitVM3-core.

14.8 Reality Check

Three summary points. First, the garbling primitive is the least settled component: RSA is broken, Argo is the frontier but nascent, and no consensus winner has emerged. Second, the on-chain label-reveal has largely converged on Schnorr adaptor signatures, for the $\sim 3\text{--}10\times$ on-chain saving of §9. Third, BitVM3 is still research and testnet: the Bitcoin bridges live today

(Citrea, GOAT, BOB) run on BitVM2, and the BitVM3 line is being hardened (Glock, Mosaic, and the BOB cost-reduction work) rather than deployed.

(optional) Further Reading

A Bitwise XOR as an Encryption Primitive

The bitwise XOR operator $\oplus$ is its own inverse: for any bit strings A, B of equal length,

$$A \oplus A = 0^{|A|}, \quad (B \oplus A) \oplus A = B.$$

This makes XOR usable as encryption. Given a message M and a key K of the same length, define $C := M \oplus K$. Anyone who knows K recovers M by computing $C \oplus K = (M \oplus K) \oplus K = M$. Without K , C alone reveals nothing about M provided K is uniformly random (this is the *one-time pad*).

Toy example. Take 4-bit strings $M = 1010$ and $K = 1100$, chosen so the four bit positions cover all four input combinations $(m_i, k_i) \in \{(1, 1), (0, 1), (1, 0), (0, 0)\}$. Encryption:

$$C = M \oplus K = 1010 \oplus 1100 = 0110.$$

Decryption:

$$C \oplus K = 0110 \oplus 1100 = 1010 = M.$$

The same key K both encrypts and decrypts, since the second XOR cancels the first.

In the garbling scheme. In Definition 3.2, the role of K is played by the hash $H(K_1 \parallel K_2 \parallel \nu)$, and the role of M by the padded message $(M \parallel 0^t)$. Encryption: $C = H(\dots) \oplus (M \parallel 0^t)$. Decryption: the evaluator computes the same hash and XORs it against C , recovering $(M \parallel 0^t)$. The 0^t pad acts as a marker: a wrong hash yields a uniformly random plaintext whose last t bits are all zero only with probability 2^{-t} , so the marker is a checkable witness that the keys were correct.

Caveat: pseudorandom, not a true one-time pad. A genuine one-time pad needs a key that is *uniformly random* and as long as the message; its secrecy is then information-theoretic, holding against an unbounded adversary. The garbling scheme does not meet this: the pad $H(K_1 \parallel K_2 \parallel \nu)$ is the deterministic output of a hash, not fresh randomness. Its security is therefore *computational*: it rests on H behaving like a random function (and on preimage resistance, per Remark 8.6), so it holds only against a polynomially bounded adversary. The XOR cancellation is identical to the one-time pad; the secrecy guarantee is strictly weaker. Read this appendix as *OTP-style* encryption, not a true one-time pad.

A.1 Wide-label rows

The lookup rows of §9.7 reuse the same shape at a larger size, and the caveat above applies to them equally. Each bit-label is a κ -bit string of 32 bytes (Remark 3.5), and a digit spans 8 wires, one label each, so the plaintext is the concatenation $L_1 \parallel \dots \parallel L_8$ of $8 \times 32 = 256$ bytes; the key $H(W_{d,v})$ is only 32 bytes. A *keystream* bridges the length gap: a pseudo-random byte sequence expanded deterministically from the key to any required length, for instance $H(\text{key} \parallel 0) \parallel H(\text{key} \parallel 1) \parallel \dots$, the arbitrary-length analogue of the fixed-length pad $H(K_1 \parallel K_2 \parallel \nu)$ above. Then

$$\text{row}_{d,v} = (L_1 \parallel \dots \parallel L_8) \oplus \text{keystream}(H(W_{d,v})).$$

The challenger, holding W from extraction, regenerates the identical keystream, XORs it off, and splits the result into labels at fixed 32-byte offsets. The labels are never *derived* from W : they were concatenated and locked at setup, and $H(W)$ only reopens the box.

B Merkle Commitments

A *Merkle tree* commits to an ordered list of values $v_1, \dots, v_n$ with a single hash, the *root*, such that any one v_i can later be proved a member without revealing the rest. Place the values at the leaves as $H(v_i)$ and hash pairwise up the tree: each internal node is $H(\text{left} \parallel \text{right})$, up to one root R . The root is a binding commitment to the whole list (changing any v_i changes R except with negligible collision probability), yet is only 32 bytes.

Membership is proved with an *authentication path*: to show v_i is in the tree, supply the sibling hash at each level from leaf to root. The verifier recomputes $H(v_i)$, folds in each sibling in turn, and accepts iff it arrives back at R . The path is $\log_2 n$ hashes, so proofs are logarithmic in the list size, the property that makes Merkle roots the workhorse of blockchain inclusion proofs (an operator’s Ethereum burn proof is one such) and of Taproot script trees (§11.2).

Two design points bear on the security arguments in the body. First, *order and domain separation*. A bare $H(\text{left} \parallel \text{right})$ tree is malleable unless leaf and internal hashes are separated, since otherwise an internal node can be reinterpreted as a leaf (a second-preimage, “node-as-leaf” attack). BIP341 separates them with tagged hashes (**TapLeaf** versus **TapBranch**) and, unusually, *sorts* the two children before hashing each branch, so no left/right flag need be transmitted, the reason the control block of Remark 11.1 carries only sibling hashes, not directions. Second, *binding but not hiding*. A Merkle root pins down its committed set but conceals nothing once a leaf is opened, which is exactly what a Taproot script-path spend does when it reveals one tapleaf. This is why the taptree argument of Remark 11.2 rests on collision-resistance alone: there is nothing to hide, only a unique committed set to pin down.